

At the top, we import the path module and our view, `post_list`. Then we set a single route at the empty string, `""`, which matches the route URL of our website. We pass in the view, `post_list`, as the second argument and then add an optional “home” name that will come in handy shortly in our template.

We also should update our `django_project/urls.py` file so that it knows to forward all requests directly to the blog app.

Code

```
# django_project/urls.py
from django.contrib import admin
from django.urls import path, include # new

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("blog.urls")), # new
]
```

We’ve added `include` on the second line and a URL pattern using an empty string regular expression, `""`, indicating that URL requests should be redirected as is to `blog`’s URLs for further instructions.

Templates

With our URLs and views now complete, we’re only missing the third piece of the puzzle: templates. Let’s use template inheritance to avoid repeating code, starting with a `base.html` file and a `home.html` file that inherits from it. Later, when we add templates for creating and editing blog posts, they can also be inherited from `base.html`.

Start by adding our new templates directory.

Shell

```
(.venv) $ mkdir templates
```

Create two new templates in your text editor: `templates/base.html` and `templates/home.html`. Then update `django_project/settings.py` so Django knows to look there for our templates.

Code

```
# django_project/settings.py
TEMPLATES = [
    {
        ...
        "DIRS": [BASE_DIR / "templates"], # new
    },
]
```

맨 위에서 우리는 경로 모듈과 우리의 뷰인 `post_list`를 가져옵니다. 그런 다음 빈 문자열인 `""`에 단일 경로를 설정하여 우리의 웹사이트의 경로 URL과 일치시킵니다. 우리는 뷰인 `post_list`를 두 번째 인수로 전달하고, 템플릿에서 곧 유용하게 사용될 선택적 `"home"` 이름을 추가합니다.

우리는 또한 `django_project/urls.py` 파일을 업데이트하여 그것이 알 수 있도록 해야 합니다. 모든 요청을 블로그 앱으로 직접 전달합니다. 코드

```
# django_project/urls.py
from django.contrib import admin
from django.urls import path, include # new

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("blog.urls")), # new]
```

우리는 두 번째 줄에 `include`를 추가하고 빈 문자열 정규 표현식인 `""`를 사용하여 URL 요청이 블로그의 URL로 리디렉션되어 추가 지침을 받을 것임을 나타내는 URL 패턴을 추가했습니다.

템플릿

이제 우리의 URL과 뷰가 완성되었으니, 우리는 퍼즐의 세 번째 조각인 템플릿만 남았습니다. 코드 중복을 피하기 위해 템플릿 상속을 사용합니다. `base.html` 파일과 이를 상속하는 `home.html` 파일로 시작하겠습니다. 나중에 블로그 게시물을 생성하고 편집하기 위한 템플릿을 추가할 때, 이들도 `base.html`에서 상속될 수 있습니다.

새 템플릿 디렉토리를 추가하는 것부터 시작합니다. 셸

```
(. venv) $ mkdir templates
```

텍스트 편집기에서 두 개의 새 템플릿을 만듭니다: `templates/base.html` 및 `templates/home.html`. 그런 다음 `django_project/settings.py`를 업데이트하여 Django가 우리의 템플릿을 찾도록 합니다.

코드

```
# django_project/settings.py
TEMPLATES = [
    {
        ...
        "DIRS": [BASE_DIR / "templates"], # 새로 만들기
```

```
    },  
    ...  
]
```

And update the `base.html` template as follows.

Code

```
<!-- templates/base.html -->  
<html>  
  
<head>  
  <title>Django blog</title>  
</head>  
  
<body>  
  <header>  
    <h1><a href="{% url 'home' %}">Django blog</a></h1>  
  </header>  
  <div>  
    {% block content %}  
    {% endblock content %}  
  </div>  
</body>  
</html>
```

The link to `url 'home'` means that we expect a URL with the name “home” to power our homepage. The code between `{% block content %}` and `{% endblock content %}` is designed to be filled by other templates. Speaking of which, here is the code for `home.html`.

Code

```
<!-- templates/home.html -->  
{% extends "base.html" %}  
  
{% block content %}  
{% for post in posts %}  
<div class="post-entry">  
  <h2><a href="">{{ post.title }}</a></h2>  
  <p>Author: {{ post.author }}</a>  
  <p>{{ post.body }}</p>  
</div>  
{% endfor %}  
{% endblock content %}
```

At the top, we note that this template extends `base.html` and then wraps our desired code with content blocks. We use the Django Templating Language to set up a simple *for loop* for each blog post. We are looping over the context dictionary `posts`, which we defined in our view, and naming each item as `post`.

```
    },  
    ...  
]
```

기본.html 템플릿을 다음과 같이 업데이트하세요.코드

```
<!-- templates/base.html -->  
<html>  
  
<head>  
  <title>Django 블로그</title>  
</head>  
  
<body>  
  <header>  
    <h1><a href="{% url 'home' %}">Django 블로그</a></h1>  
  </header>  
  <div>  
    {% block content %}{%  
    endblock content %}</div>  
  
</body>  
  
</html>
```

'home'라는 URL 링크는 우리가 홈페이지를 위한 “home”이라는 이름의 URL을 기대한다는 것을 의미합니다. {% block content %}와 {% endblock content %} 사이의 코드는 다른 템플릿에 의해 채워지도록 설계되었습니다. 그에 대해 말하자면, 여기 home.html의 코드가 있습니다.

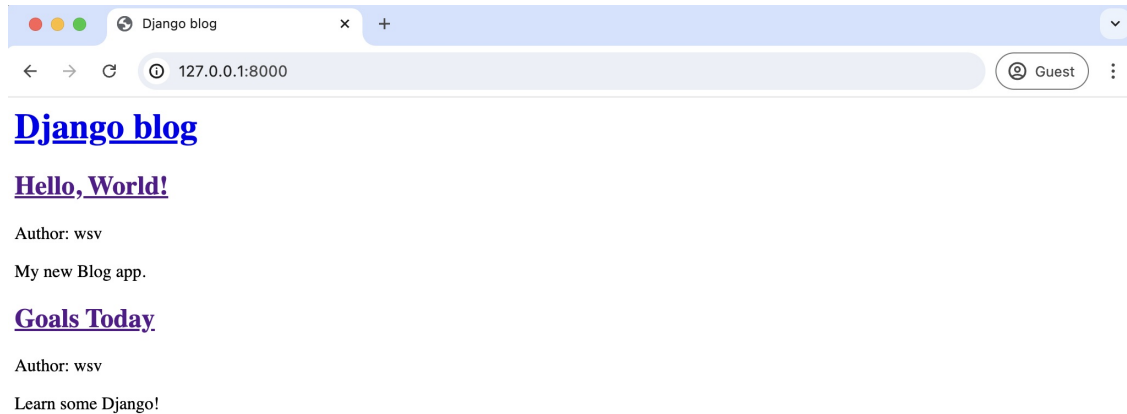
코드

```
<!-- templates/home.html -->  
{% extends "base.html" %}  
  
{% block content %}{% for post  
in posts %}  
<div class="post-entry"><h2><a href="">  
  {{ post.title }}</a></h2><p>저자:  
  {{ post.author }}</a><p>{{ post.body }}  
</p>  
</div>  
{% endfor %}{% endblock  
content %}
```

맨 위에서 이 템플릿이 base.html을 확장하고 원하는 코드를 콘텐츠 블록으로 감싸고 있음을 알 수 있습니다. 우리는 Django 템플릿 언어를 사용하여 각 블로그 게시물에 대한 간단한 for 루프를 설정합니다. 우리는 뷰에서 정의한 컨텍스트 사전 posts를 반복하고 각 항목을 post로 명명하고 있습니다.

Then, we can use dot notation to display fields like `post.title`, `post.author`, and `post.body`.

If you start the Django server again with `python manage.py runserver` and refresh the homepage, we can see it is working.



Blog Homepage with Two Posts

But it looks terrible. Let's fix that by adding some styling.

Static Files

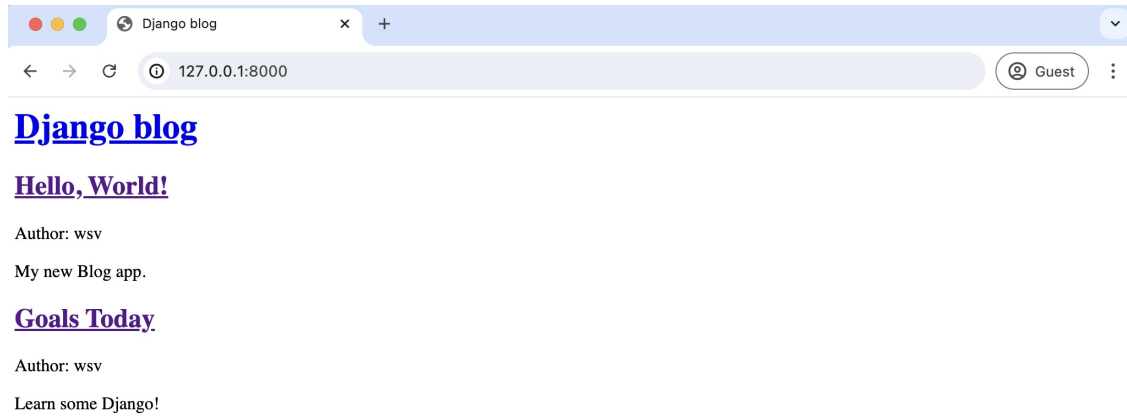
[Static files](#) are the Django community's term for additional files commonly served on websites, such as CSS, fonts, images, and JavaScript. Even though we haven't added any to our project yet, we are already relying on core Django static files—custom CSS, fonts, images, and JavaScript—to power the look and feel of the Django admin.

In production, things are more complex, which we will cover properly in the deployment section of this book. The central concept to understand is that it is far more efficient to combine all static files across a Django project into a single location in production. If you look near the bottom of the existing `django_project/settings.py` file, there is already a configuration for [STATIC_URL](#), which refers to the *URL location* of all static files in production. In other words, if our website had the URL `example.com`, all static files would be available in `example.com/static`.

Code

그럼, 우리는 dot 표기법을 사용하여 post.title, post.author, post.body와 같은 필드를 표시할 수 있습니다.

python manage.py runserver로 Django 서버를 다시 시작하고 홈페이지를 새로 고치면, 작동하는 것을 볼 수 있습니다.



두 개의 게시물이 있는 블로그 홈페이지

하지만 보기 흉합니다. 스타일을 추가하여 이를 수정합시다.

정적 파일

정적 파일은 CSS, 글꼴, 이미지 및 JavaScript와 같이 웹사이트에서 일반적으로 제공되는 추가 파일에 대한 Django 커뮤니티의 용어입니다. 아직 프로젝트에 추가하지 않았지만, 우리는 이미 Django 관리자의 모양과 느낌을 제공하기 위해 기본 Django 정적 파일인 사용자 정의 CSS, 글꼴, 이미지 및 JavaScript에 의존하고 있습니다.

프로덕션에서는 상황이 더 복잡해지며, 이는 이 책의 배포 섹션에서 제대로 다룰 것입니다. 이해해야 할 중심 개념은 프로덕션에서 Django 프로젝트의 모든 정적 파일을 단일 위치로 결합하는 것이 훨씬 더 효율적이라는 것입니다. 기존 `django_project/settings.py` 파일의 하단 근처를 보면, 프로덕션에서 모든 정적 파일의 URL 위치를 참조하는 `STATIC_URL`에 대한 구성이 이미 있습니다. 다시 말해, 우리의 웹사이트가 `example.com`이라는 URL을 가졌다면, 모든 정적 파일은 `example.com/static`에 있습니다.

```
# django_project/settings.py
STATIC_URL = "static/"
```

For local development, we don't have to worry about static files because the web server—run via the `runserver` command—will automatically find and serve them for us. It is worth mentioning that modern web browsers implement quite aggressive caching, so if you have made changes and don't see them on your local web project it is possible you need to clear your web browser cache. A good way to check if this is the issue is to try a different web browser, for example Safari rather than Chrome, if you fail to see a static file change materialize on the page.

The first question when adding a new static file is where to place it. By default, Django will look within each app for a folder called “static”; in other words, a folder called `blog/static/`. If you recall, this is similar to how templates are treated.

As Django projects grow in complexity over time and have multiple apps, it is often simpler to reason about static files if they are stored in a single, project-level directory instead. That is the approach we will take here.

Quit the local server with `Control+c` and create a new static directory in the same folder as the `manage.py` file.

Shell

```
(.venv) $ mkdir static
```

[STATICFILES_DIRS](#) defines the additional locations the built-in `staticfiles` app will traverse looking for static files beyond an `app/static` folder. We need to add our project-level static folder to this configuration.

Code

```
# django_project/settings.py
STATIC_URL = "static/"
STATICFILES_DIRS = [BASE_DIR / "static"] # new
```

Next, create a `css` directory within `static`.

Shell

```
(.venv) $ mkdir static/css
```

```
# django_project/settings.py
STATIC_URL = "static/"
```

로컬 개발의 경우, 웹 서버가 runserver 명령어를 통해 실행되므로 정적 파일에 대해 걱정할 필요가 없습니다. 웹 서버가 자동으로 정적 파일을 찾아 제공할 것입니다. 현대 웹 브라우저는 꽤 공격적인 캐싱을 구현하므로, 변경 사항을 만들었지만 로컬 웹 프로젝트에서 보이지 않는 경우 웹 브라우저 캐시를 지워야 할 수도 있습니다. 이 문제가 발생하는지 확인하는 좋은 방법은 Chrome 대신 Safari와 같은 다른 웹 브라우저를 시도해 보는 것입니다. 정적 파일 변경 사항이 페이지에 나타나지 않는 경우입니다.

새로운 정적 파일을 추가할 때 첫 번째 질문은 어디에 배치할 것인가입니다. 기본적으로 Django는 각 앱 내에서 "static"이라는 폴더를 찾습니다. 즉, blog/static/이라는 폴더입니다. 기억하신다면, 이는 템플릿이 처리되는 방식과 유사합니다.

Django 프로젝트가 시간이 지남에 따라 복잡성이 증가하고 여러 앱을 가지게 되면, 정적 파일을 단일 프로젝트 수준 디렉토리에 저장하는 것이 더 간단한 경우가 많습니다. 이것이 우리가 여기서 취할 접근 방식입니다.

Control+c로 로컬 서버를 종료하고 manage.py 파일과 동일한 폴더에 새로운 정적 디렉토리를 만듭니다.
Shell

```
(.venv) $ mkdir static
```

STATICFILES_DIRS는 내장된 staticfiles 앱이 앱/static 폴더 외부에서 정적 파일을 찾기 위해 탐색할 추가 위치를 정의합니다. 이 구성에 프로젝트 수준의 정적 폴더를 추가해야 합니다.

코드

```
# django_project/settings.py
STATIC_URL = "static/"
STATICFILES_DIRS = [BASE_DIR / "static"]
```

새로 만들기

다음으로, static.Shell 내에 css 디렉토리를 만듭니다.

```
(.venv) $ mkdir static/css
```

Add a new file within your text editor in this directory called `static/css/base.css`. What should we put in our file? How about changing the title to red?

Code

```
/* static/css/base.css */
header h1 a {
    color: red;
}
```

The last step is adding the static files to our templates by adding `{% load static %}` to the top of `base.html`. Because our other templates inherit from `base.html`, we only have to add this once. Include a new line at the bottom of the `<head></head>` code that explicitly references our new `base.css` file.

Code

```
<!-- templates/base.html -->
{% load static %}
<html>

<head>
    <title>Django blog</title>
    <link rel="stylesheet" href="{% static 'css/base.css' %}">
</head>
...
```

Phew! That was a pain, but it's a one-time hassle. We can add static files to our static directory, which will automatically appear in all our templates.

Start the server again with `python manage.py runserver` and look at our updated homepage at `http://127.0.0.1:8000/`.

이 디렉토리 내에서 static/css/base.css라는 이름의 새 파일을 텍스트 편집기에서 추가하세요. 파일에 무엇을 넣어야 할까요? 제목을 빨간색으로 바꾸는 것은 어떨까요?

코드

```
/* static/css/base.css */
header h1 a
{color: red;
}
```

마지막 단계는 {% load static %}를 base.html의 맨 위에 추가하여 정적 파일을 템플릿에 추가하는 것입니다. 다른 템플릿이 base.html을 상속 받기 때문에 이 작업은 한 번만 하면 됩니다. <head></head> 코드의 맨 아래에 새 base.css 파일을 명시적으로 참조하는 새 줄을 포함하세요.

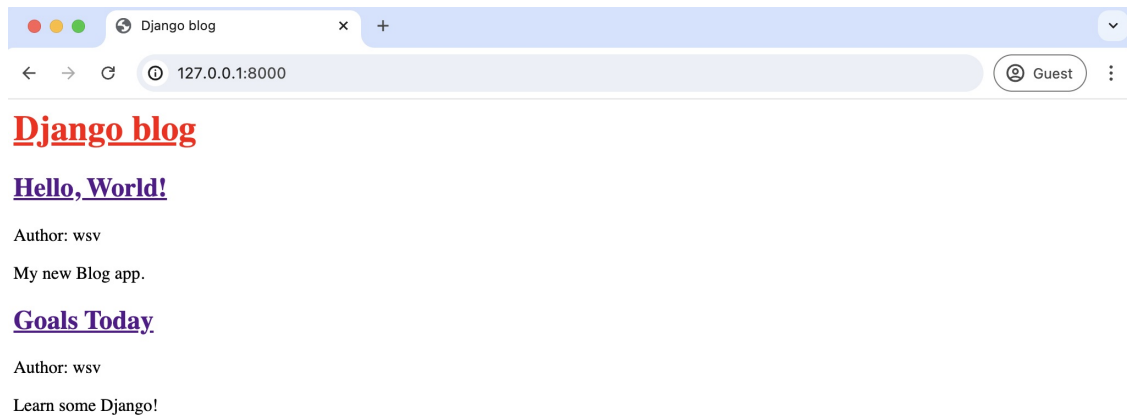
코드

```
<!-- templates/base.html -->
{% load static %}
<html>

<head>
  <title>Django 블로그</title>
  <link rel="stylesheet" href="{% static 'css/base.css' %}">
</head>
...
```

휴! 힘들었지만, 한 번의 번거로움이었습니다. 정적 파일을 ourstatic 디렉토리에 추가하면 모든 템플릿에 자동으로 나타납니다.

python manage.py runserver로 서버를 다시 시작하고
http://127.0.0.1:8000/에서 업데이트된 홈페이지를 확인하세요.



Blog Homepage with Red Title

If you see an error, `TemplateSyntaxError` at `/`, you forgot to add the `{% load static %}` line at the top. Even after all my years of using Django, I still make this mistake all the time! Fortunately, Django’s error message says, “Invalid block tag on line 4: ‘static’. Did you forget to register or load this tag?”. A pretty accurate description of what happened, no?

Even with this new styling, we can still do a little better. Let’s add a custom font and some more CSS. Since this book is not on CSS, we can insert the following between `<head></head>` tags to add [Source Sans Pro](https://fonts.googleapis.com/css?family=Source+Sans+Pro:400), a free font from Google.

Code

```
<!-- templates/base.html -->
{% load static %}
<html>

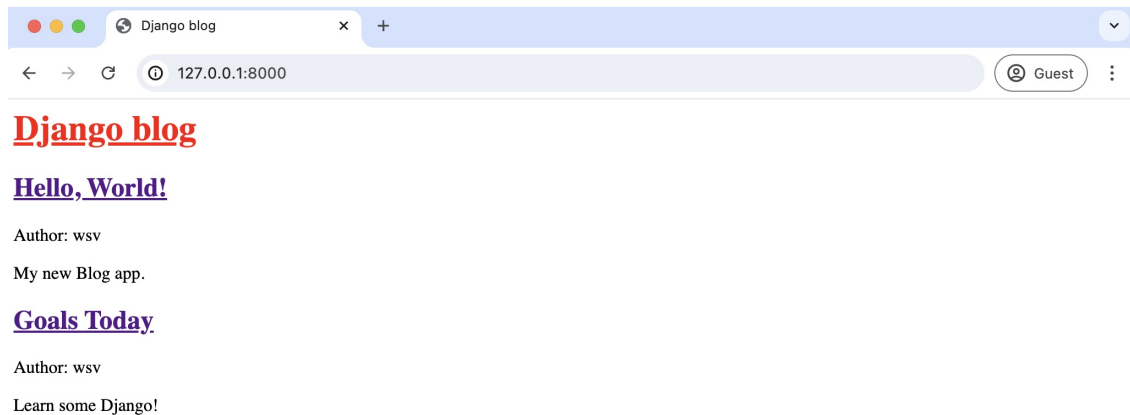
<head>
  <title>Django blog</title>
  <link href="https://fonts.googleapis.com/css?family=Source+Sans+Pro:400"
    rel="stylesheet">
  <link href="{% static 'css/base.css' %}" rel="stylesheet">
</head>
...
```

Then, update our CSS file by copying and pasting the following code:

Code

```
/* static/css/base.css */
body {
  font-family: 'Source Sans Pro', sans-serif;
  font-size: 18px;
}

header {
```



블로그 홈페이지와 빨간 제목

오류가 발생하면, `TemplateSyntaxError at /, {% loadstatic %}` 줄을 맨 위에 추가하는 것을 잊으셨습니다. Django를 사용한 지 여러 해가 지났지만, 저는 여전히 이 실수를 자주 합니다! 다행히도 Django의 오류 메시지는 "라인 4에서 잘못된 블록 태그: 'static'. 이 태그를 등록하거나 로드하는 것을 잊으셨나요?"라고 말합니다. 무슨 일이 있었는지에 대한 꽤 정확한 설명이죠?

이 새로운 스타일링에도 불구하고, 우리는 여전히 조금 더 나아질 수 있습니다. 사용자 정의 글꼴과 더 많은 CSS를 추가해 봅시다. 이 책은 CSS에 관한 것이 아니므로, 다음을 삽입할 수 있습니다

`<head></head>` 태그 사이에 [Source Sans Pro, Google의 무료 글꼴](https://fonts.googleapis.com/css?family=Source+Sans+Pro:400)을 추가하기 위해. 코드

```
<!-- templates/base.html -->
{% load static %}
<html>

<head>
  <title>Django 블로그</title>
  <link href="https://fonts.googleapis.com/css?family=Source+Sans+Pro:400" rel="stylesheet">
  <link href="{% static 'css/base.css' %}" rel="stylesheet">
</head>
...
```

그런 다음, 다음 코드를 복사하여 붙여넣어 CSS 파일을 업데이트합니다:

코드

```
/* static/css/base.css */
body {
  font-family: 'Source Sans Pro', sans-serif;
  font-size: 18px;}

header {
```

```
border-bottom: 1px solid #999;
margin-bottom: 2rem;
display: flex;
}

header h1 a {
color: red;
text-decoration: none;
}

.nav-left {
margin-right: auto;
}

.nav-right {
display: flex;
padding-top: 2rem;
}

.post-entry {
margin-bottom: 2rem;
}

.post-entry h2 {
margin: 0.5rem 0;
}

.post-entry h2 a,
.post-entry h2 a:visited {
color: blue;
text-decoration: none;
}

.post-entry p {
margin: 0;
font-weight: 400;
}

.post-entry h2 a:hover {
color: red;
}
```

Refresh the homepage at <http://127.0.0.1:8000/>; you should see the following.

```
border-bottom: 1px solid #999;  
margin-bottom: 2rem;  
display: flex; }
```

```
header h1 a  
  {color: red;  
   text-decoration: none; }
```

```
.nav-left {  
  margin-right: auto; }
```

```
.nav-right {display:  
  flex; padding-top:  
  2rem;  
}
```

```
.post-entry {  
  margin-bottom: 2rem; }
```

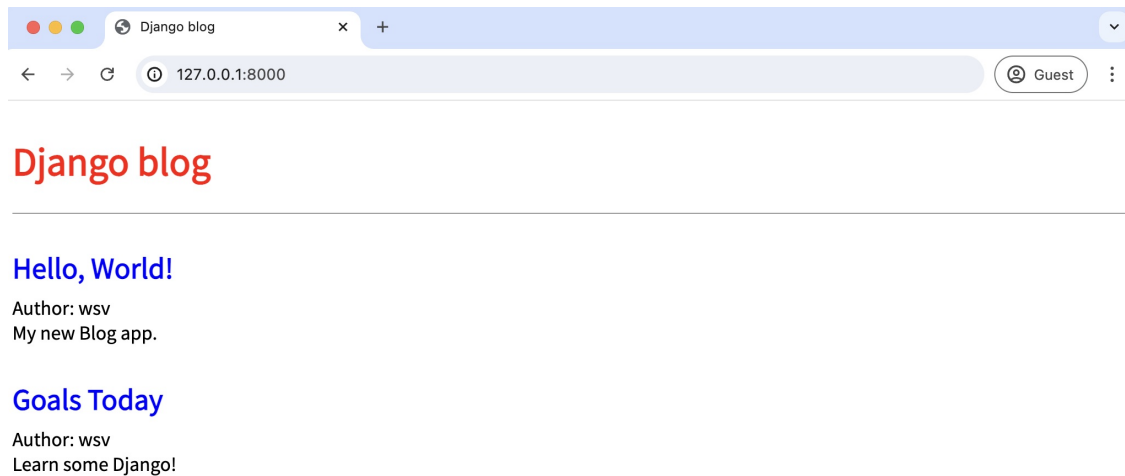
```
.post-entry h2  
  {margin: 0.5rem  
} 0;
```

```
.post-entry h2 a,  
.post-entry h2 a:visited {color:  
  blue;  
  text-decoration: none; }
```

```
.post-entry p  
  {margin: 0;  
   font-weight: 400; }
```

```
.post-entry h2 a: hover {color:  
  red;  
}
```

홈페이지를 새로 고치세요 <http://127.0.0.1:8000/>; 다음을 볼 수
있어야 합니다.



Blog Homepage with CSS

Individual Blog Pages

Now, we can add functionality for individual blog pages. We need to create a new view, URL, and template to do that. I hope you're noticing a pattern in Django development!

Start with the view. At the top of our views file, import the shortcut function [get_object_or_404\(\)](#), which calls the [get](#) QuerySet method to return an object or raises a [Http404](#) error if unsuccessful.

We will name the view function `post_detail` as it represents the detailed view of a blog post. It accepts two parameters: the first, `request`, is an instance of the `HttpRequest` object; the second, `pk`, is a parameter extracted from the URL that identifies the specific blog post to be displayed by its primary key.

Let's take a moment to focus on that last sentence, as it gets at the heart of understanding this function. To specify an individual blog post, we need a way to say that out of the list of all blog posts in the database, we should choose this specific one. The easiest and default way is to use the primary key ID associated with each record. For example, the first entry has a primary key (PK) of 1, the second of 2, and so on automatically set by the Django ORM.

Within the body of the `post_detail` function there are two lines. First, we call `get_object_or_404` on the `Post` model and specify that the `pk` primary key



Django blog

Hello, World!

Author: wsv
My new Blog app.

Goals Today

Author: wsv
Learn some Django!

CSS가 포함된 블로그 홈페이지

개별 블로그 페이지

이제 개별 블로그 페이지에 대한 기능을 추가할 수 있습니다. 이를 위해 새로운 뷰, URL 및 템플릿을 만들어야 합니다. Django 개발에서 패턴을 인식하고 계시길 바랍니다!

뷰부터 시작합니다. 우리의 views 파일 상단에 `get_object_or_404()`라는 단축 함수를 가져와서, 이는 `get_queryset` 메서드를 호출하여 객체를 반환하거나 실패할 경우 `Http404` 오류를 발생시킵니다.

우리는 이 뷰 함수의 이름을 `post_detail`로 지을 것입니다. 이는 블로그 게시물의 상세 보기를 나타냅니다. 두 개의 매개변수를 받습니다: 첫 번째는 `request`로, `HttpRequest` 객체의 인스턴스입니다; 두 번째는 `pk`로, URL에서 추출된 매개변수로 특정 블로그 게시물을 기본 키로 식별합니다.

마지막 문장에 잠시 집중해 봅시다. 이는 이 함수의 이해의 핵심을 다룹니다. 개별 블로그 게시물을 지정하기 위해, 데이터베이스의 모든 블로그 게시물 목록 중에서 이 특정 게시물을 선택해야 한다고 말할 방법이 필요합니다. 가장 쉽고 기본적인 방법은 각 레코드와 연결된 기본 키 ID를 사용하는 것입니다. 예를 들어, 첫 번째 항목은 기본 키(PK)가 1이고, 두 번째는 2이며, 이렇게 Django ORM에 의해 자동으로 설정됩니다.

`post_detail` 함수의 본문 내에는 두 줄이 있습니다. 첫째, 우리는 `Post` 모델에서 `get_object_or_404`를 호출하고 `pk` 기본 키를 지정합니다.

matches the parameter pk from the URL. If we want the URL /1, our function will call the blog post with a primary key of 1. When you see this in action, it will make more sense. We set the variable post to equal this specific blog post. Then we use the render() function to return a response, where the first argument is the request variable, the second is the template post_detail.html, and the third is the template context where we create a variable post that matches the post from the previous line.

Code

```
# blog/views.py
from django.shortcuts import render, get_object_or_404 # new
from .models import Post

def post_list(request):
    posts = Post.objects.all()
    return render(request, 'home.html', {'posts': posts})

def post_detail(request, pk): # new
    post = get_object_or_404(Post, pk=pk)
    return render(request, "post_detail.html", {"post": post})
```

This pattern for accessing a list of information from the database or a single item is repeated repeatedly in web development with Django, so don't worry if it's a little confusing at the moment.

With our view created, the next two steps are adding a URL route and a template. In the blog/urls.py file, import the new view, post_detail, and add a URL route at post/<int:pk>/. This pattern will look a bit strange initially as it is our first use of a Django [path converter](#). Up to this point, we have hardcoded our URL routes, but it is more common to use variables. In this case, we specify that individual posts will start at post/, but then we use int to specify that the captured value from the URL should be treated as an integer and the variable's name passed to the view is pk. The second argument is the view name, post_detail, and we add the optional third argument of a name that is also post_detail.

Code

```
# blog/urls.py
from django.urls import path
from .views import post_list, post_detail # new

urlpatterns = [
    path("post/<int:pk>/", post_detail, name="post_detail"), # new
    path("", post_list, name="home"),
]
```

URL의 매개변수 pk와 일치합니다. URL /1을 원할 경우, 우리의 함수는 기본 키가 1인 블로그 게시물을 호출합니다. 이것을 실제로 보면 더 이해가 될 것입니다. 우리는 변수 post를 이 특정 블로그 게시물과 같게 설정합니다. 그런 다음 render() 함수를 사용하여 응답을 반환하는데, 첫 번째 인자는 요청 변수, 두 번째는 템플릿 post_detail.html, 세 번째는 이전 줄의 게시물과 일치하는 변수 post를 생성하는 템플릿 컨텍스트입니다.

코드

```
# blog/views.py
from django.shortcuts import render, get_object_or_404 # new
from .models import Post

def post_list(request):
    posts = Post.objects.all()
    return render(request, 'home.html', {'posts': posts})

def post_detail(request, pk): # new
    post = get_object_or_404(Post, pk=pk)
    return render(request, 'post_detail.html', {'post': post})
```

데이터베이스에서 정보 목록이나 단일 항목에 접근하는 이 패턴은 Django로 웹 개발을 할 때 반복적으로 사용되므로, 현재 약간 혼란스러워도 걱정하지 마세요.

뷰가 생성되었으므로 다음 두 단계는 URL 경로와 템플릿을 추가하는 것입니다. blog/urls.py 파일에서 새로운 뷰인 post_detail을 가져오고 post/<int:pk>/에 URL 경로를 추가합니다. 이 패턴은 Django 경로 변환기를 처음 사용하는 것이기 때문에 처음에는 약간 이상하게 보일 수 있습니다. 지금까지 우리는 URL 경로를 하드코딩했지만, 변수를 사용하는 것이 더 일반적입니다. 이 경우 개별 게시물이 post/에서 시작한다고 지정하지만, int를 사용하여 URL에서 캡처된 값을 정수로 처리해야 한다고 지정하고 뷰에 전달되는 변수의 이름은 pk입니다. 두 번째 인수는 뷰 이름인 post_detail이며, 선택적 세 번째 인수로 이름을 추가하는데, 이 이름도 post_detail입니다.

코드

```
# blog/urls.py
from django.urls import path
from .views import post_list, post_detail # new

urlpatterns = [
    path("post/<int:pk>/", post_detail, name="post_detail"), path("", # 새
    post_list, name="home"),
]
```

The last step is adding a new template file called `templates/post_detail.html` in your text editor. Then type in the following code:

Code

```
<!-- templates/post_detail.html -->
{% extends "base.html" %}

{% block content %}
<div class="post-entry">
  <h2>{{ post.title }}</h2>
  <p>{{ post.body }}</p>
</div>
{% endblock content %}
```

At the top, we specify that this template inherits from `base.html`. The template context variable `post` contains information from this particular blog post, and we can display individual fields using `post.title` and `post.body`.

If you start the server with `python manage.py runserver`, you'll see a dedicated page for our first blog post at `http://127.0.0.1:8000/post/1/`.



Blog Post One Detail

Woohoo! You can also go to `http://127.0.0.1:8000/post/2/` to see the second entry.

마지막 단계는 텍스트 편집기에서 templates/post_detail.html이라는 새 템플릿 파일을 추가하는 것입니다. 그런 다음 다음 코드를 입력하세요:

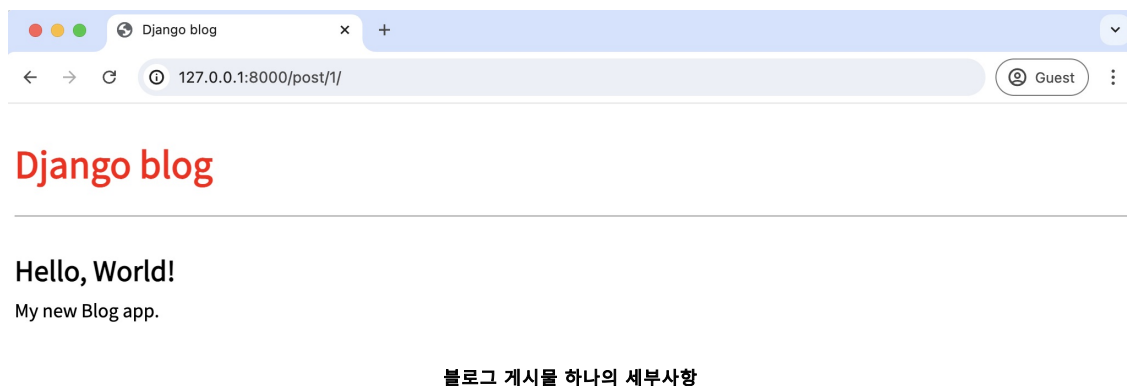
코드

```
<!-- templates/post_detail.html -->
{% extends "base.html" %}

{% block content %}
<div class="post-entry"><h2>
  {{ post.title }}</h2><p>
  {{ post.body }}</p>
</div>
{% endblock content %}
```

맨 위에서 이 템플릿이 base.html을 상속한다고 지정합니다. 템플릿 컨텍스트 변수 post는 이 특정 블로그 게시물의 정보를 포함하고 있으며, post.title과 post.body를 사용하여 개별 필드를 표시할 수 있습니다.

python manage.py runserver로 서버를 시작하면 http://127.0.0.1:8000/post/1/에서 첫 번째 블로그 게시물 전용 페이지를 볼 수 있습니다.



우와! http://127.0.0.1:8000/post/2/에 가서 두 번째 항목을 볼 수도 있습니다.



Django blog

Goals Today

Learn some Django!

Blog Post Two Detail

To make our lives easier, we should update the link on the homepage so we can directly access individual blog posts from there. Swap out the current empty link, `<a href="">` with `<a href="{% url 'post_detail' post.pk %}">`.

Code

```
<!-- templates/home.html -->
{% extends "base.html" %}

{% block content %}
{% for post in posts %}
<div class="post-entry">
  <h2><a href="{% url 'post_detail' post.pk %}">{{ post.title }}</a></h2>
  <p>{{ post.body }}</p>
</div>
{% endfor %}
{% endblock content %}
```

We start by using Django's `url` template tag and specifying the URL pattern name of `post_detail`. If you look at `post_detail` in our URLs file, it expects to be passed an argument `pk` representing the primary key for the blog post. Fortunately, Django has already created and included this `pk` field on our `post` object, but we must pass it into the URL by adding it to the template as `post.pk`.

To confirm everything works, refresh the main page at `http://127.0.0.1:8000/` and click on the title of each blog post to confirm the new links work.

get_absolute_url()

Currently, we are using the `url` template tag, which means that every time we want to display an individual blog post in this template or other templates, we



Django blog

Goals Today

Learn some Django!

블로그 게시물 두 번째 세부사항

우리의 삶을 더 쉽게 만들기 위해, 우리는 홈페이지의 링크를 업데이트하여 그곳에서 개별 블로그 게시물에 직접 접근할 수 있도록 해야 합니다. 현재 비어 있는 링크, `<a href="">`를 `<a href="{% url 'post_detail' post.pk %}">`로 교체하세요. 코드

```
<!-- templates/home.html -->
{% extends "base.html" %}

{% block content %}{% for post
in posts %}<div class="post-
entry">
  <h2><a href="{% url 'post_detail' post.pk %}">{{ post.title }}</a></h2>
  <p>{{ post.body }}</p>
</div>
{% endfor %}{% endblock
content %}
```

우리는 Django의 url 템플릿 태그를 사용하고 post_detail의 URL 패턴 이름을 지정하는 것으로 시작합니다. 우리의 URLs 파일에서 post_detail을 보면, 블로그 게시물의 기본 키를 나타내는 인수 pk를 전달받기를 기대합니다. 다행히도 Django는 이미 이 pk 필드를 우리의 post 객체에 생성하고 포함시켰지만, 우리는 이를 URL에 전달하기 위해 템플릿에 post.pk로 추가해야 합니다.

모든 것이 작동하는지 확인하려면 `http://127.0.0.1:8000/`에서 메인 페이지를 새로 고치고 각 블로그 게시물의 제목을 클릭하여 새 링크가 작동하는지 확인하세요.

get_absolute_url()

현재 우리는 url 템플릿 태그를 사용하고 있습니다. 이는 이 템플릿이 나 다른 템플릿에서 개별 블로그 게시물을 표시할 때마다 우리가

must repeat the pattern `{% url 'post_detail' post.pk %}`. If the URL pattern changes, we need to update every template where the URL is constructed, which increases the risk of errors.

A better approach is to use the built-in [get_absolute_url\(\)](#) method, which tells Django how to calculate the canonical URL for our model object. In the `blog/models.py` file, add a new method for getting the `get_absolute_url`.

Code

```
# blog/models.py
from django.db import models
from django.urls import reverse # new

class Post(models.Model):
    title = models.CharField(max_length=200)
    author = models.ForeignKey(
        "auth.User",
        on_delete=models.CASCADE,
    )
    body = models.TextField()

    def __str__(self):
        return self.title

    def get_absolute_url(self): # new
        return reverse("post_detail", kwargs={"pk": self.pk})
```

At the top, we import [reverse\(\)](#), a utility function for our URLs. Then, we define `get_absolute_url` using `self` as the first parameter referring to the model instance on which the method is called. This is a standard practice in Python for instance methods. The `reverse()` function accepts the URL name and keyword arguments or “kwargs.” In this case, we are setting the variable `pk` to equal the primary key of our model instance.

We don’t need to update the migrations files because we aren’t changing the database schema here. Migrations are only required when changes to the model affect the database schema, such as adding or removing fields, changing field types, or modifying relationships between models.

In the template file, update the `href` link to now use `post.get_absolute_url`.

Code

```
<!-- templates/home.html -->
{% extends "base.html" %}

{% block content %}
{% for post in posts %}
```

패턴 {% url 'post_detail' post.pk %}를 반복해야 합니다. URL 패턴이 변경되면, URL이 구성된 모든 템플릿을 업데이트해야 하므로 오류의 위험이 증가합니다.

더 나은 접근 방식은 내장된 `get_absolute_url()` 메서드를 사용하는 것입니다. 이 메서드는 Django에게 모델 객체의 표준 URL을 계산하는 방법을 알려줍니다. 다음의 `blog/models.py` 파일에서 `get_absolute_url`을 가져오는 새로운 메서드를 추가합니다. 코드

```
# blog/models.py
from django.db import models
from django.urls import reverse # new

class Post(models.Model):
    title = models.CharField(max_length=200)
    author = models.ForeignKey("auth.User", on_delete=models.CASCADE,
    )
    body = models.TextField()

    def __str__(self):
        return self.title

    def get_absolute_url(self): # new
        return reverse("post_detail", kwargs={"pk": self.pk})
```

맨 위에서, 우리는 URL을 위한 유틸리티 함수인 `reverse()`를 가져옵니다. 그런 다음, `self`를 첫 번째 매개변수로 사용하여 `get_absolute_url`을 정의합니다. 이는 메서드가 호출되는 모델 인스턴스를 참조합니다. 이는 Python의 인스턴스 메서드에서 표준 관행입니다. `reverse()` 함수는 URL 이름과 키워드 인수 또는 "kwargs"를 받습니다. 이 경우, 우리는 변수 `pk`를 모델 인스턴스의 기본 키와 같게 설정하고 있습니다.

우리는 데이터베이스 스키마를 변경하지 않기 때문에 마이그레이션 파일을 업데이트할 필요가 없습니다. 마이그레이션은 모델의 변경이 데이터베이스 스키마에 영향을 미칠 때만 필요합니다. 예를 들어, 필드를 추가하거나 제거하거나, 필드 유형을 변경하거나, 모델 간의 관계를 수정하는 경우입니다.

템플릿 파일에서 `href` 링크를 이제 `post.get_absolute_url` 코드를 사용하도록 업데이트합니다.

```
<!-- templates/home.html -->
{% extends "base.html" %}

{% block content %}{% for post
in posts %}
```

```
<div class="post-entry">
  <h2><a href="{ post.get_absolute_url }">{{ post.title }}</a></h2> <!-- new -->
  <p>{{ post.body }}</p>
</div>
{% endfor %}
{% endblock content %}
```

URL paths can and do change over a project's lifetime. With the previous method, if we changed the post detail view and URL path, we'd have to go through all our HTML and templates to update the code, a very error-prone and hard-to-maintain process. By using `get_absolute_url()` instead, we have one place, the `models.py` file, where the canonical URL is set, so our templates don't have to change.

Refresh the main page at `http://127.0.0.1:8000/` and click on the title of each blog post to confirm the links still work as expected.

Tests

Our *Blog* project has added new functionality we have not seen or tested before this chapter. The `Post` model has multiple fields; we have a user for the first time, and there is a list view of all blog posts and a detailed view for each blog post. There is quite a lot to test!

First, we can set up our test data and check the content of the `Post` model. Here's how that might look.

Code

```
# blog/tests.py
from django.contrib.auth import get_user_model
from django.test import TestCase

from .models import Post

class BlogTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        cls.user = get_user_model().objects.create_user(
            username="testuser", email="test@email.com", password="secret"
        )

        cls.post = Post.objects.create(
            title="A good title",
            body="Nice body content",
            author=cls.user,
        )
```

```
<div class="post-entry">
  <h2><a href="{{ post.get_absolute_url }}">{{ post.title }}</a></h2> <!-- new --><p>{{ post.body
  }}</p>
</div>
{% endfor %}{% endblock
content %}
```

URL 경로는 프로젝트의 수명 동안 변경될 수 있습니다. 이전 방법을 사용하면, 게시물 세부 정보 보기와 URL 경로를 변경할 경우, 모든 HTML과 템플릿을 통해 코드를 업데이트해야 하므로 매우 오류가 발생하기 쉽고 유지 관리하기 어려운 과정이 됩니다. 대신 `get_absolute_url()`을 사용하면, 정식 URL이 설정된 `models.py` 파일이라는 한 곳이 있으므로 템플릿을 변경할 필요가 없습니다.

`http://127.0.0.1:8000/`에서 메인 페이지를 새로 고치고 각 블로그 게시물의 제목을 클릭하여 링크가 여전히 예상대로 작동하는지 확인하세요.

테스트

우리 블로그 프로젝트는 이 장에서 보거나 테스트하지 않은 새로운 기능을 추가했습니다. 포스트 모델에는 여러 필드가 있으며, 우리는 처음으로 사용자를 가지고 있고, 모든 블로그 게시물의 목록 보기와 각 블로그 게시물에 대한 세부 보기 기능이 있습니다. 테스트할 것이 꽤 많습니다!

먼저, 테스트 데이터를 설정하고 포스트 모델의 내용을 확인할 수 있습니다. 다음은 그 모습입니다.

코드

```
# blog/tests.py
from django.contrib.auth import get_user_model
from django.test import TestCase

from .models import Post

class BlogTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        cls.user = get_user_model().objects.create_user(
            username="testuser",
            email="test@email.com", password="secret"
        )

        cls.post = Post.objects.create(
            title="A good title", body="Nice body content",
            author=cls.user,
        )
```

```
def test_post_model(self):
    self.assertEqual(self.post.title, "A good title")
    self.assertEqual(self.post.body, "Nice body content")
    self.assertEqual(self.post.author.username, "testuser")
    self.assertEqual(str(self.post), "A good title")
    self.assertEqual(self.post.get_absolute_url(), "/post/1/")
```

At the top, we import `get_user_model()` to refer to our User and then added TestCase and the Post model. Our class BlogTests contains setup data for both a test user and a test post. Currently, all the tests are focused on the Post model, so we name our test `test_post_model`. It checks that all three model fields return the expected values. Our model also has new tests for the `__str__` and `get_absolute_url` methods.

Go ahead and run the tests.

Shell

```
(.venv) $ python manage.py test
Found 1 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.
-----
Ran 1 test in 0.088s

OK
Destroying test database for alias 'default'...
```

What else to add? We now have two types of pages: a homepage that lists all blog posts and a detail page for each blog post containing its primary key in the URL. In the previous two chapters, we implemented tests to check that:

- expected URLs exist and return a 200 status code
- URL names work and return a 200 status code
- the correct template name is used
- the correct template content is outputted

All four tests need to be included. We could have eight new unit tests, four for each of our two pages, or we could combine them a bit. There isn't a right or wrong answer here so long as tests are implemented to test functionality, and it is clear from their names what went wrong if an error arises.

Here is one way to add these checks to our code:

Code

```
def test_post_model(self):
    self.assertEqual(self.post.title, "A good title")
    self.assertEqual(self.post.body, "Nice body content")
    self.assertEqual(self.post.author.username, "testuser")
    self.assertEqual(str(self.post), "A good title")
    self.assertEqual(self.post.get_absolute_url(), "/post/1/")
```

맨 위에서 `get_user_model()`을 가져와서 우리의 User를 참조한 다음, TestCase와 Post 모델을 추가했습니다. 우리의 클래스 BlogTests는 테스트 사용자와 테스트 게시물에 대한 설정 데이터를 포함합니다. 현재 모든 테스트는 Post 모델에 집중되어 있으므로, 우리의 테스트 이름을 `test_post_model`로 지정합니다. 이는 세 개의 모델 필드가 예상 값을 반환하는지 확인합니다. 우리의 모델은 또한 `__str__` 및 `get_absolute_url` 메서드에 대한 새로운 테스트를 가지고 있습니다.

계속해서 테스트를 실행하세요.

Shell

```
(.venv) $ python manage.py test테스트
1개를 찾았습니다.
별칭 'default'에 대한 테스트 데이터베이스를 생성하는
중...시스템 검사가 문제를 식별하지 못했습니다 (0 무시
됨)..
```

```
-----
-1개의 테스트를 0.088초에 실행했습니다
```

확인

별칭 'default'에 대한 테스트 데이터베이스를 파괴하는 중...

추가할 것이 더 무엇이 있을까요? 이제 우리는 두 가지 유형의 페이지가 있습니다: 모든 블로그 게시물을 나열하는 홈페이지와 각 블로그 게시물의 기본 키가 포함된 상세 페이지입니다. 이전 두 장에서는 다음을 확인하기 위해 테스트를 구현했습니다:

- 예상 URL이 존재하고 200 상태 코드를 반환합니다
- URL 이름이 작동하고 200 상태 코드를
- 반환합니다. 올바른 템플릿 이름이 사용됩
- 니다. 올바른 템플릿 내용이 출력됩니다.

모든 네 가지 테스트가 포함되어야 합니다. 우리는 두 페이지 각각에 대해 네 개의 새로운 단위 테스트를 추가하여 총 여덟 개의 테스트를 만들 수 있거나, 약간 결합할 수도 있습니다. 기능을 테스트하기 위해 테스트가 구현되는 한 정답이나 오답은 없습니다. 오류가 발생할 경우 무엇이 잘못되었는지 이름에서 명확하게 알 수 있습니다.

다음은 이러한 검사를 코드에 추가하는 한 가지 방법입니다:

코드

```

# blog/tests.py
from django.contrib.auth import get_user_model
from django.test import TestCase
from django.urls import reverse # new

from .models import Post

class BlogTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        cls.user = get_user_model().objects.create_user(
            username="testuser", email="test@email.com", password="secret"
        )

        cls.post = Post.objects.create(
            title="A good title",
            body="Nice body content",
            author=cls.user,
        )

    def test_post_model(self):
        self.assertEqual(self.post.title, "A good title")
        self.assertEqual(self.post.body, "Nice body content")
        self.assertEqual(self.post.author.username, "testuser")
        self.assertEqual(str(self.post), "A good title")
        self.assertEqual(self.post.get_absolute_url(), "/post/1/")

    def test_url_exists_at_correct_location_listview(self): # new
        response = self.client.get("/")
        self.assertEqual(response.status_code, 200)

    def test_url_exists_at_correct_location_detailview(self): # new
        response = self.client.get("/post/1/")
        self.assertEqual(response.status_code, 200)

    def test_post_listview(self): # new
        response = self.client.get(reverse("home"))
        self.assertEqual(response.status_code, 200)
        self.assertContains(response, "Nice body content")
        self.assertTemplateUsed(response, "home.html")

    def test_post_detailview(self): # new
        response = self.client.get(reverse("post_detail",
            kwargs={"pk": self.post.pk}))
        no_response = self.client.get("/post/100000/")
        self.assertEqual(response.status_code, 200)
        self.assertEqual(no_response.status_code, 404)
        self.assertContains(response, "A good title")
        self.assertTemplateUsed(response, "post_detail.html")

```

First, we check that the URL exists at the proper location for both views. Then we import [reverse](#) at the top and create `test_post_listview` to confirm that the named URL is used, returns a 200 status code, contains the expected content, and uses the `home.html` template. For `test_post_detailview`, we must pass in the

```

# 블로그/tests.py
from django.contrib.auth import get_user_model
from django.test import TestCase
from django.urls import reverse # new
from django.models import Post

class BlogTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        cls.user = get_user_model().objects.create_user(username="testuser",
            email="test@email.com", password="secret"
        )

        cls.post = Post.objects.create(title="좋은 제목", body="멋진 본문 내용",
            author=cls.user, )

    def test_post_model(self):
        self.assertEqual(self.post.title, "A good title")
        self.assertEqual(self.post.body, "Nice body content")
        self.assertEqual(self.post.author.username, "testuser")
        self.assertEqual(str(self.post), "A good title")
        self.assertEqual(self.post.get_absolute_url(), "/post/1/")

    def test_url_exists_at_correct_location_listview(self):
        # new
        response = self.client.get("/")
        self.assertEqual(response.status_code, 200)

    def test_url_exists_at_correct_location_detailview(self):
        # new
        response = self.client.get("/post/1/")
        self.assertEqual(response.status_code, 200)

    def test_post_listview(self):
        # new
        response = self.client.get(reverse("home"))
        self.assertEqual(response.status_code, 200)
        self.assertContains(response, "Nice body content")
        self.assertTemplateUsed(response, "home.html")

    def test_post_detailview(self):
        # new
        response = self.client.get(reverse("post_detail", kwargs={"pk": self.post.pk}))
        no_response = self.client.get("/post/100000/")
        self.assertEqual(response.status_code, 200)
        self.assertEqual(no_response.status_code, 404)
        self.assertContains(response, "A good title")
        self.assertTemplateUsed(response, "post_detail.html")

```

먼저, 두 뷰 모두에 대해 URL이 올바른 위치에 존재하는지 확인합니다. 그런 다음, 맨 위에 `reverse`를 가져오고 `test_post_listview`를 생성하여 명명된 URL이 사용되고 200 상태 코드를 반환하며 예상된 콘텐츠를 포함하고 `home.html` 템플릿을 사용하는지 확인합니다. `test_post_detailview`의 경우, 우리는 다음을 전달해야 합니다.

pk of our test post to the response. The same template is used, and we add new tests for what we *don't want* to see. For example, we don't want a response at the URL `/post/100000/` because we have not created that many posts yet! And we don't want a 404 HTTP status response either. It is always good to sprinkle in examples of incorrect tests that *should* pass through failure using the `no_response` method to ensure your tests aren't mindlessly passing for some reason.

Run the new tests to confirm everything is working.

Shell

```
(.venv) $ python manage.py test
Found 5 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.....
-----
Ran 5 tests in 0.095s

OK
Destroying test database for alias 'default'...
```

A common mistake when testing URLs is failing to include the preceding slash `/`. For example, if `test_url_exists_at_correct_location_detailview` is checked in the response for `"post/1/"` that would throw a 404 error. However, if you check ``"/post/1/"`, it will be a 200 status response.

Git

Now is also a good time for our first Git commit. Initialize our directory and review all the added content by checking the status.

Shell

```
(.venv) $ git init
(.venv) $ git status
```

Oops, we do not want to include the `.venv` directory and the SQLite database! There might also be a `__pycache__` directory. To remove all three, in your text editor, create a project-level `.gitignore` file—in the same top directory as `manage.py`—and add these three lines.

`.gitignore`

```
.venv/
__pycache__/
db.sqlite3
```

우리 테스트 게시물의 pk를 응답에 추가합니다. 동일한 템플릿이 사용되며, 우리가 보고 싶지 않은 것에 대한 새로운 테스트를 추가합니다. 예를 들어, 우리는 /post/100000/ URL에서 응답을 원하지 않습니다. 왜냐하면 아직 그만큼의 게시물을 생성하지 않았기 때문입니다! 그리고 404 HTTP 상태 응답도 원하지 않습니다. 항상 실패를 통해 통과해야 하는 잘못된 테스트의 예를 섞어주는 것이 좋습니다. no_response 메서드를 사용하여 테스트가 어떤 이유로든 무작정 통과하지 않도록 합니다.

새로운 테스트를 실행하여 모든 것이 작동하는지 확인합니다.

셸

```
(.venv) $ python manage.py test 5개의
테스트를 찾았습니다.
별칭 'default'에 대한 테스트 데이터베이스를 생성하는 중...
시스템 검사가 문제를 식별하지 않았습니다 (0 무시
됨).....
```

```
-----
- 0.095초에 5개의 테스트를 실행했습니다.
```

확인

```
별칭 'default'에 대한 테스트 데이터베이스를 파괴하는 중...
```

URL을 테스트할 때 흔히 발생하는 실수는 앞에 슬래시 /를 포함하지 않는 것입니다. 예를 들어, test_url_exists_at_correct_location_detailview가 "post/1/"에 대한 응답에서 확인되면 404 오류가 발생합니다. 그러나 "/post/1/"를 확인하면 200 상태 응답이 됩니다.

Git

지금은 첫 번째 Git 커밋을 하기 좋은 시점입니다. 우리의 디렉토리를 초기화하고 상태를 확인하여 추가된 모든 내용을 검토합니다. 셸

```
(.venv) $ git init(.venv)
$ git status
```

이런, 우리는 .venv 디렉토리와 SQLite 데이터베이스를 포함하고 싶지 않습니다! __pycache__ 디렉토리도 있을 수 있습니다. 이 세 가지를 모두 제거하려면 텍스트 편집기에서 프로젝트 수준의 .gitignore 파일을 생성하세요—manage.py와 같은 최상위 디렉토리에 위치시키고 이 세 줄을 추가합니다.

.gitignore

```
.venv/__pycac
he__/db.sqlite3
```

Run `git status` again to confirm the `.venv` directory is no longer included. Then, add the rest of our work along with a commit message.

Shell

```
(.venv) $ git status
(.venv) $ git add -A
(.venv) $ git commit -m "initial commit"
```

Conclusion

We've now built a basic blog application from scratch! We can create, edit, or delete the content using the Django admin, which will display a list of all posts on the homepage and individual pages for each post. We looked at static files for the first time and added some styling with CSS. We also learned about the best practice of using `get_absolute_url` in our templates and made substantial progress on our test suite.

In the next section, we'll switch to generic class-based views and add forms to create, update, and delete blog posts so we don't have to use the Django admin for these changes.

다시 `git status`를 실행하여 `.venv` 디렉토리가 더 이상 포함되지 않는지 확인하세요. 그런 다음, 나머지 작업과 커밋 메시지를 추가하세요.

셸

```
(.venv) $ git status
(.venv) $ git add -A
(.venv) $ git commit -m "초기 커밋"
```

결론

이제 우리는 처음부터 기본 블로그 애플리케이션을 구축했습니다! Django 관리자를 사용하여 콘텐츠를 생성, 편집 또는 삭제할 수 있으며, 이는 홈페이지와 각 게시물의 개별 페이지에 모든 게시물 목록을 표시합니다. 우리는 처음으로 정적 파일을 살펴보고 CSS로 약간의 스타일링을 추가했습니다. 또한 템플릿에서 `get_absolute_url`을 사용하는 모범 사례에 대해 배우고 테스트 스위트에서 상당한 진전을 이루었습니다.

다음 섹션에서는 일반 클래스 기반 뷰로 전환하고 블로그 게시물을 생성, 업데이트 및 삭제하기 위한 양식을 추가하여 이러한 변경을 위해 Django 관리자를 사용할 필요가 없도록 할 것입니다.

Chapter 7: Forms

In this chapter, we'll continue working on our *Blog* application by switching to class-based views and adding forms so a user can create, edit, or delete any of their blog entries. HTML forms are one of the more complicated and error-prone aspects of web development. Any time you accept user input, there are significant security concerns since users are uploading information to your database. All forms must be properly rendered, validated, and saved to the database. Writing this code by hand would be time-consuming and complex, so Django comes with powerful [built-in Forms](#) and [generic editing views](#) for common tasks like displaying, creating, updating, or deleting a form.

ListView and DetailView

Currently, we are using function-based views for our page listing all blog posts and powering individual posts. We could continue down this page and create function-based views for create, edit, and delete functionality; however, doing so requires a lot more code and is error-prone compared to using generic class-based views designed explicitly for the job. For this reason, we will switch to using generic class-based views throughout the rest of the book.

Let's look at the current `blog/views.py` file.

Code

```
# blog/views.py
from django.shortcuts import render, get_object_or_404 # new
from .models import Post

def post_list(request):
    posts = Post.objects.all()
    return render(request, 'home.html', {'posts': posts})

def post_detail(request, pk): # new
    post = get_object_or_404(Post, pk=pk)
    return render(request, "post_detail.html", {"post": post})
```

Switching over to generic class-based views is relatively straightforward. Here is the updated code for the views:

Code

7장: 양식

이 장에서는 클래스 기반 뷰로 전환하고 사용자가 블로그 항목을 생성, 편집 또는 삭제할 수 있도록 양식을 추가하여 블로그 애플리케이션 작업을 계속 진행하겠습니다. HTML 양식은 웹 개발에서 가장 복잡하고 오류가 발생하기 쉬운 측면 중 하나입니다. 사용자의 입력을 수락할 때마다 사용자가 정보를 데이터베이스에 업로드하므로 상당한 보안 문제가 발생합니다. 모든 양식은 적절하게 렌더링되고, 검증되며, 데이터베이스에 저장되어야 합니다. 이 코드를 수작업으로 작성하는 것은 시간 소모가 크고 복잡하므로 Django는 양식을 표시, 생성, 업데이트 또는 삭제하는 것과 같은 일반적인 작업을 위한 강력한 ~~내장 양식~~ 및 ~~일반 편집 뷰~~를 제공합니다.

ListView 및 DetailView

현재 우리는 모든 블로그 게시물을 나열하고 개별 게시물을 처리하기 위해 함수 기반 뷰를 사용하고 있습니다. 이 페이지를 계속 진행하여 생성, 편집 및 삭제 기능을 위한 함수 기반 뷰를 만들 수 있지만, 그렇게 하려면 훨씬 더 많은 코드가 필요하고 작업을 위해 명시적으로 설계된 일반 클래스 기반 뷰를 사용하는 것에 비해 오류가 발생하기 쉽습니다. 이러한 이유로 우리는 책의 나머지 부분에서 일반 클래스 기반 뷰를 사용하도록 전환할 것입니다.

현재 블로그/views.py 파일을 살펴보겠습니다.코드

```
# blog/views.py
from django.shortcuts import render, get_object_or_404 # new
from .models import Post

def post_list(request):
    posts = Post.objects.all()
    return render(request, 'home.html', {'posts': posts})

def post_detail(request, pk): # new
    post = get_object_or_404(Post, pk=pk)
    return render(request, 'post_detail.html', {'post': post})
```

제네릭 클래스 기반 뷰로 전환하는 것은 상대적으로 간단합니다. 다음은 뷰에 대한 업데이트된 코드입니다:

Code

```
# blog/views.py
from django.views.generic import ListView, DetailView
from .models import Post

class BlogListView(ListView):
    model = Post
    template_name = "home.html"

class BlogDetailView(DetailView):
    model = Post
    template_name = "post_detail.html"
```

At the top, we import the generic views `ListView` and `DetailView`, along with our model, `Post`. We used `ListView` previously in the Message Board app, but [DetailView](#) is new. It is used for detail pages and has a format extremely similar to `ListView`. We pass in the generic class-based view and define the model and template for both views. That's it.

Next up is the URLs file. We change the views imports to match our new ones, `BlogListView` and `BlogDetailView`. Then, we update the second argument in both routes, specifying the view name and adding `as_view()` to transform the class into a callable view function.

Code

```
# blog/urls.py
from django.urls import path
from .views import BlogListView, BlogDetailView # new

urlpatterns = [
    path("post/<int:pk>/", BlogDetailView.as_view(), name="post_detail"), # new
    path("", BlogListView.as_view(), name="home"), # new
]
```

The final update is to our template. Previously, in our function-based list and detail views, we named the context object `posts` and `post` respectively. But for the GCBV `ListView`, you need to know that the default naming pattern is `<model>_list`; since our model is `Post`, the context object will be `post_list`. The updated code looks as follows:

Code

```
<!-- templates/home.html -->
{% extends "base.html" %}

{% block content %}
{% for post in post_list %} <!-- new -->
<div class="post-entry">
    <h2><a href="{ post.get_absolute_url }">{{ post.title }}</a></h2>
```

```
# blog/views.py
from django.views.generic import ListView, DetailView
from .models import Post
```

```
class BlogListView(ListView):
    model = Post
    template_name = "home.html"
```

```
class BlogDetailView(DetailView):
    model = Post
    template_name = "post_detail.html"
```

맨 위에서, 우리는 일반 뷰인 `ListView`와 `DetailView`를 가져오고, 모델인 `Post`를 가져옵니다. 우리는 이전에 Message Board 앱에서 `ListView`를 사용했지만, `DetailView`는 새롭습니다. 이것은 상세 페이지에 사용되며 `ListView`와 매우 유사한 형식을 가지고 있습니다. 우리는 일반 클래스 기반 뷰를 전달하고 두 뷰 모두에 대한 모델과 템플릿을 정의합니다. 그게 전부입니다.

다음은 URLs 파일입니다. 우리는 뷰 가져오기를 새로운 것인 `BlogListView`와 `BlogDetailView`에 맞게 변경합니다. 그런 다음, 두 경로의 두 번째 인수를 업데이트하여 뷰 이름을 지정하고 `as_view()`를 추가하여 클래스를 호출 가능한 뷰 함수로 변환합니다.

코드

```
# blog/urls.py
from django.urls import path
from .views import BlogListView, BlogDetailView # new

urlpatterns = [
    path("post/<int:pk>/", BlogDetailView.as_view(), name="post_detail"), # new
    path("", BlogListView.as_view(), name="home"), # new
]
```

최종 업데이트는 우리의 템플릿입니다. 이전에, 함수 기반의 리스트 및 상세 보기에서 우리는 컨텍스트 객체를 각각 `posts`와 `post`라고 명명했습니다. 그러나 GCBV `ListView`의 경우, 기본 명명 패턴이 `<model>_list`라는 것을 알아야 합니다; 우리의 모델이 `Post`이므로, 컨텍스트 객체는 `post_list`가 됩니다. 업데이트된 코드는 다음과 같습니다:

코드

```
<!-- templates/home.html -->
{% extends "base.html" %}

{% block content %}
{% for post in post_list %} <!-- new -->
<div class="post-entry">
  <h2><a href="{ post.get_absolute_url }">{{ post.title }}</a></h2>
```

```
<p>{{ post.body }}</p>
</div>
{% endfor %}
{% endblock content %}
```

Do you want to change the context object name? You can! Django almost always provides a way to customize behavior. The attribute to update is called [context_object_name](#). The update below would switch it back to `posts` as opposed to `post_list`. We won't implement this change in the book, but you are welcome to do so on your own.

Code

```
# blog/views.py
...
class BlogListView(ListView):
    model = Post
    template_name = "home.html"
    context_object_name = 'posts' # Change the context object name to 'posts'
...
```

The context object name in a `DetailView` also has a default value of either the model name (so `post` in this case) or `object`. That means we could leave our current template as is.

Code

```
<!-- templates/post_detail.html -->
{% extends "base.html" %}

{% block content %}
<div class="post-entry">
  <h2>{{ post.title }}</h2>
  <p>{{ post.body }}</p>
</div>
{% endblock content %}
```

Or we could change `post` to `object` like this:

Code

```
<!-- templates/post_detail.html -->
{% extends "base.html" %}

{% block content %}
<div class="post-entry">
  <h2>{{ object.title }}</h2>
  <p>{{ object.body }}</p>
</div>
{% endblock content %}
```

```
<p>{{ post.body }}</p>
</div>
{% endfor %}{% endblock
content %}
```

컨텍스트 객체 이름을 변경하시겠습니까? 가능합니다! Django는 거의 항상 동작을 사용자 정의할 수 있는 방법을 제공합니다. 업데이트할 속성은 context_object_name이라고 합니다. 아래의 업데이트는 `post_list` 대신 `posts`로 다시 전환할 것입니다. 이 변경 사항은 책에서 구현하지 않겠지만, 원하신다면 직접 하실 수 있습니다.

코드

```
# blog/views.py
...
class BlogListView(ListView): model =
    Post
    template_name = "home.html"
    context_object_name = 'posts' ... # 'posts'로 컨텍스트 객체 이름 변경
```

DetailView의 컨텍스트 객체 이름은 기본값으로 `themodel` 이름(이 경우 `post`) 또는 `object`를 가집니다. 즉, 현재 템플릿을 그대로 두어도 됩니다.

코드

```
<!-- templates/post_detail.html -->
{% extends "base.html" %}

{% block content %}
<div class="post-entry"><h2>
    {{ post.title }}</h2><p>
    {{ post.body }}</p>
</div>
{% endblock content %}
```

또는 `post`를 `object`로 이렇게 변경할 수 있습니다:코드

```
<!-- templates/post_detail.html -->
{% extends "base.html" %}

{% block content %}
<div class="post-entry"><h2>
    {{ object.title }}</h2><p>
    {{ object.body }}</p>
</div>
{% endblock content %}
```

If you refresh an individual blog post page in your web browser, you'll see that both work. Which naming pattern you prefer is a personal preference. I like to use the model name, so we'll use `post` going forward in our detail view templates. If you don't like these built-in names for the template context object, you can override the existing [context_object_name](#) variable in your `DetailView`.

Mixins

If you were paying close attention, you noticed that the `context_object_name` attribute for `ListView` was part of [MultipleObjectMixin](#) whereas for `DetailView` it was part of [SingleObjectMixin](#). Why the difference?

A mixin is a type of multiple inheritance where you can add specific functionality to a class by including additional classes. Mixins are used in class-based views (CBVs) and generic class-based views (GCBVs) to add reusable behaviors or functionality without duplicating code. But you need to know what you're doing to use them effectively.

Covering Django mixins fully is beyond the scope of a beginner book, but it is worth briefly explaining how the inheritance structure works. To do this, we will rely on [Classy Class-Based Views](#), a website dedicated to describing the full methods and attributes for each Django GCBV. It is an invaluable resource if you decide to work with GCBVs. Looking at the entry for [ListView](#), you can see its Ancestors listed at the top of the page. The hierarchy of classes used by `ListView` starts from the bottom.

1. `ListView`
2. `MultipleObjectTemplateResponseMixin`
3. `TemplateResponseMixin`
4. `BaseListView`
5. `MultipleObjectMixin`
6. `ContextMixin`
7. `View`

The base class, [View](#), is used by all class-based views. You don't need to memorize all these mixins to use GCBVs. But over time as attempt to customize their behavior, you will likely find yourself looking for the specific attribute or method that needs to be overridden.

개별 블로그 게시물 페이지를 웹 브라우저에서 새로 고치면 두 가지 모두 작동하는 것을 볼 수 있습니다. 어떤 명명 패턴을 선호하는지는 개인적인 취향입니다. 저는 모델 이름을 사용하는 것을 좋아하므로, 앞으로 세부 보기 템플릿에서는 post를 사용하겠습니다. 템플릿 컨텍스트 객체에 대한 이러한 내장 이름이 마음에 들지 않으면, `DetailView`에서 기존 `context_object_name` 변수를 재정의할 수 있습니다.

믹스인

주의 깊게 지켜보았다면, `ListView`의 `context_object_name` 속성이 `MultipleObjectMixin`의 일부인 반면, `DetailView`의 경우 `SingleObjectMixin`의 일부라는 것을 알았을 것입니다. 왜 차이가 있을까요?

믹스인은 특정 기능을 추가하기 위해 추가 클래스를 포함하여 클래스에 특정 기능을 추가할 수 있는 다중 상속의 한 유형입니다. 믹스인은 클래스 기반 뷰(CBVs)와 일반 클래스 기반 뷰(GCBVs)에서 재사용 가능한 동작이나 기능을 추가하기 위해 사용되며, 코드를 중복하지 않고도 가능합니다. 하지만 효과적으로 사용하려면 무엇을 하고 있는지 알아야 합니다.

Django 믹스인을 완전히 다루는 것은 초보자 책의 범위를 넘어가지만, 상속 구조가 어떻게 작동하는지 간단히 설명할 가치가 있습니다. 이를 위해 우리는 [각 Django GCBV의 전체 메서드와 속성을 설명하는 웹사이트인 Classy Class-Based Views](#)에 의존할 것입니다. GCBV로 작업하기로 결정하면 귀중한 자원이 될 것입니다. `ListView`의 항목을 살펴보면 페이지 상단에 조상들이 나열되어 있는 것을 볼 수 있습니다. `ListView`에서 사용되는 클래스의 계층 구조는 아래에서 시작됩니다.

1. `ListView`
2. `MultipleObjectTemplateResponseMixin`
3. `TemplateResponseMixin`
4. `BaseListView`
5. `MultipleObjectMixin`
6. `ContextMixin`
7. `View`

기본 클래스인 `View`는 모든 클래스 기반 뷰에서 사용됩니다. GCBV를 사용하기 위해 이러한 믹스인을 모두 암기할 필요는 없습니다. 그러나 시간이 지나면서 그들의 동작을 사용자 정의하려고 시도할 때, 오버라이드해야 하는 특정 속성이나 메서드를 찾고 있는 자신을 발견하게 될 것입니다.

A proponent of function-based views would say that this is ridiculous; you should have all the code you are using visible. A fan of class-based views would counter that it is ridiculous to write the same boilerplate code repeatedly when you only want to change one or two lines. I fall in the latter camp, but you can certainly understand why some very experienced Django developers prefer function-based views.

CreateView

We now need a view where users can create a new blog post. [CreateView](#) is a generic class-based view designed for exactly this purpose. Here is the code we need in our views file.

Code

```
# blog/views.py
from django.views.generic import ListView, DetailView
from django.views.generic.edit import CreateView # new
from .models import Post

class BlogListView(ListView):
    model = Post
    template_name = "home.html"

class BlogDetailView(DetailView):
    model = Post
    template_name = "post_detail.html"

class BlogCreateView(CreateView): # new
    model = Post
    template_name = "post_new.html"
    fields = ["title", "author", "body"]
```

At the top we import `CreateView` from the `django.views.generic.edit` module. Then we create a class, `BlogCreateView`, that extends `CreateView`, specifying the model, `Post`, the template name, `post_new.html`, and the database fields we want visible on the form, which are `title`, `author`, and `body`. That's it! The function-based version of this view is much, much longer.

Then, update the URLs file to add a URL route for the create page.

Code

```
# blog/urls.py
from django.urls import path
from .views import BlogListView, BlogDetailView, BlogCreateView # new
```

함수 기반 뷰의 지지자는 이것이 터무니없다고 말할 것이다; 당신이 사용하는 모든 코드를 볼 수 있어야 한다. 클래스 기반 뷰의 팬은 단지 한 두 줄만 변경하고 싶을 때 같은 보일러플레이트 코드를 반복해서 작성하는 것이 터무니없다고 반박할 것이다. 나는 후자의 입장에 있지만, 일부 매우 경험이 많은 Django 개발자들이 함수 기반 뷰를 선호하는 이유를 확실히 이해할 수 있다.

CreateView

이제 사용자가 새 블로그 게시물을 생성할 수 있는 뷰가 필요하다. CreateView는 바로 이 목적을 위해 설계된 일반 클래스 기반 뷰이다. 여기 우리가 views 파일에 필요한 코드가 있다.

코드

```
# blog/views.py
from django.views.generic import ListView, DetailView
from django.views.generic.edit import CreateView
from .models import Post

class BlogListView(ListView):
    model = Post
    template_name = "home.html"

class BlogDetailView(DetailView):
    model = Post
    template_name = "post_detail.html"

class BlogCreateView(CreateView):
    model = Post
    template_name = "post_new.html"
    fields = ("title", "author", "body")
```

맨 위에서 `django.views.generic.edit` 모듈에서 `CreateView`를 가져옵니다. 그런 다음 `CreateView`를 확장하는 `BlogCreateView`라는 클래스를 생성하고, 모델인 `Post`, 템플릿 이름인 `post_new.html`, 그리고 양식에 표시할 데이터베이스 필드인 `title`, `author`, `body`를 지정합니다. 그게 전부입니다! 이 뷰의 함수 기반 버전은 훨씬 더 깁니다.

그런 다음, 생성 페이지에 대한 URL 경로를 추가하기 위해 `URLs` 파일을 업데이트합니다.

코드

```
# blog/urls.py
from django.urls import path
from .views import BlogListView, BlogDetailView, BlogCreateView
```

new

```
urlpatterns = [
    path("post/new/", BlogCreateView.as_view(), name="post_new"), # new
    path("post/<int:pk>/", BlogDetailView.as_view(), name="post_detail"),
    path("", BlogListView.as_view(), name="home"),
]
```

Simple, right? It's the same URL, views, and template pattern we've seen before. We import the new view, `BlogCreateView` and set the new path. It is located at `post/new/`, uses the view `BlogCreateView`, and has a URL name of `post_new`.

The only thing left is our template. Make a new template, `templates/post_new.html`, in the text editor. Then add the following code:

Code

```
<!-- templates/post_new.html -->
{% extends "base.html" %}

{% block content %}
<h1>New post</h1>
<form action="" method="post">{% csrf_token %}
    {{ form }}
    <input type="submit" value="Save">
</form>
{% endblock content %}
```

Let's break down what we've done:

- on the top line, we extended our base template.
- use HTML `<form>` tags with the *POST* method since we're *sending* data. If we were receiving data from a form, for example, in a search box, we would use GET.
- add a `{% csrf_token %}` which Django provides to protect our form from cross-site request forgery. You should use it for all your Django forms.
- then, to output our form data use `{{ form }}` to render the specified fields.
- finally, specify an input type of submit and assign the value "Save".

To view our work, start the server with `python manage.py runserver` and go to the create a new page at `http://127.0.0.1:8000/post/new`.

```
urlpatterns = [
    path("post/new/", BlogCreateView.as_view(), name="post_new"), # newpath("
    post/<int:pk>/", BlogDetailView.as_view(), name="post_detail"), path("",
    BlogListView.as_view(), name="home"),
]
```

간단하죠? 이전에 보았던 것과 동일한 URL, 뷰 및 템플릿 패턴입니다. 새로운 뷰인 BlogCreateView를 가져오고 새로운 경로를 설정합니다. 이는 post/new/에 위치하며, BlogCreateView 뷰를 사용하고 URL 이름은 post_new입니다.

남은 것은 우리의 템플릿입니다. 새로운 템플릿을 만드세요, templates/post_new.html, 텍스트 편집기에서. 그런 다음 다음 코드를 추가하세요:코드

```
<!-- templates/post_new.html -->
{% extends "base.html" %}

{% block content %}<h1>
새 게시물</h1>
<form action="" method="post">{% csrf_token %}{{ form
}}
<input type="submit" value="저장">
</form>
{% endblock content %}
```

우리가 한 일을 정리해 봅시다:

- 첫 번째 줄에서, 우리는 기본 템플릿을 확장했습니다.
- 데이터를 전송하고 있으므로 POST 방법을 사용하여 HTML <form> 태그를 사용합니다. 예를 들어, 검색 상자에서 양식으로부터 데이터를 수신하는 경우 GET을 사용합니다.
- 교차 사이트 요청 위조로부터 우리의 양식을 보호하기 위해 Django가 제공하는 {% csrf_token %}을 추가합니다. 모든 Django 양식에 대해 사용해야 합니다. 그런 다음, 양식 데이터를 출력하기 위해 {{ form }}을 사용하여 지정된 필드를 렌더링합니다. 마지막으로, 제출 유형의 입력을 지정하고 값 "저장"을 할당합니다.
-

작업을 보려면 python manage.py runserver로 서버를 시작하고 http://127.0.0.1:8000/post/new에서 새 페이지를 만듭니다.



Django blog

New post

Title:

Author:

Body:

Blog New Page

Try to create a new blog post and submit it by clicking the “Save” button.



Django blog

New post

Title:

Author:

Body:

Blog Third Post Save

Django blog

New post

Title:

Author:

Body:

블로그 새 페이지

새 블로그 게시물을 작성하고 "저장" 버튼을 클릭하여 제출해 보세요.

Django blog

New post

Title:

Author:

Body:

블로그 세 번째 게시물 저장

Upon completion, it will redirect to a detail page with the post at <http://127.0.0.1:8000/post/3/>. Success!



Django blog

3rd post

I wonder if this will work?

Blog Third Post Page

Rather than making a user guess where the create new post page is, let's add a link to our base template. It will take the form `<a href="{% url 'post_new' %}"></a>` where `post_new` is the name for our URL.

Your updated `templates/base.html` file should look as follows:

Code

```
<!-- templates/base.html -->
{% load static %}
<html>

<head>
  <title>Django blog</title>
  <link href="https://fonts.googleapis.com/css?family=Source+Sans+Pro:400" rel="stylesheet">
  <link href="{% static 'css/base.css' %}" rel="stylesheet">
</head>

<body>
  <div>
    <header>
      <!-- start new HTML... -->
      <div class="nav-left">
        <h1><a href="{% url 'home' %}">Django blog</a></h1>
      </div>
      <div class="nav-right">
        <a href="{% url 'post_new' %}">+ New Blog Post</a>
      </div>
      <!-- end new HTML... -->
    </header>
    {% block content %}
    {% endblock content %}
  </div>
</body>

</html>
```

완료되면 <http://127.0.0.1:8000/post/3/>의 게시물 세부 정보 페이지로 리디렉션됩니다. 성공!



Django blog

3rd post

I wonder if this will work?

블로그 세 번째 게시물 페이지

사용자가 새 게시물 생성 페이지가 어디인지 추측하게 하기보다는, 기본 템플릿에 링크를 추가합니다. 형식은 `<a href="{% url 'post_new' %}"></a>`가 되며, 여기서 `post_new`은 우리의 URL 이름입니다.

업데이트된 `templates/base.html` 파일은 다음과 같아야 합니다:코드

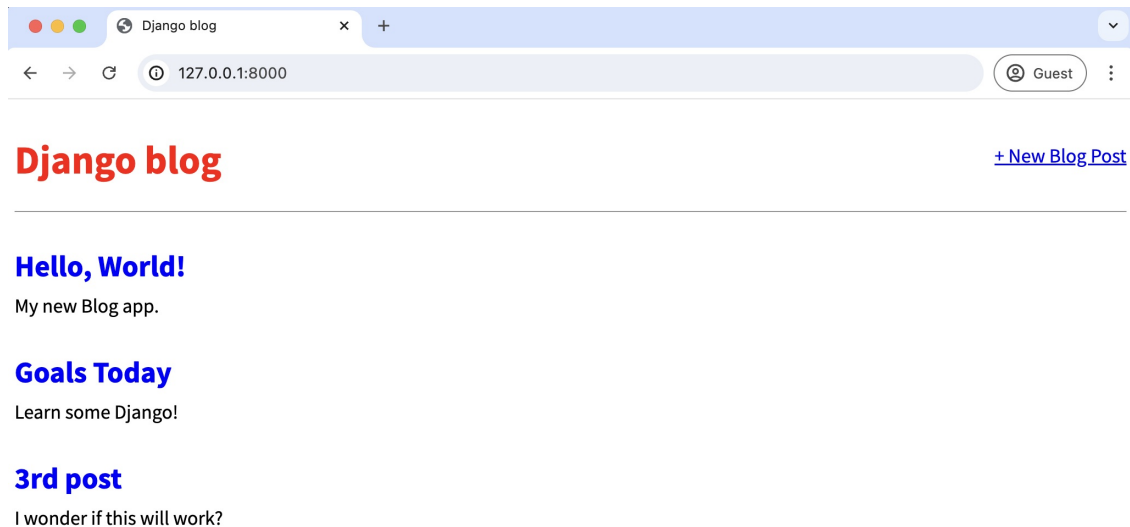
```
<!-- templates/base.html -->
{% load static %}
<html>

<head>
  <title>Django 블로그</title>
  <link href="https://fonts.googleapis.com/css?family=Source+Sans+Pro:400" rel="stylesheet">
  <link href="{% static 'css/base.css' %}" rel="stylesheet">
</head>

<body>
  <div>
    > <header>
      <!-- start new HTML... -->
      <div class="nav-left">
        <h1><a href="{% url 'home' %}">장고 블로그</a></h1>
      </div><div class="nav-right">
        <a href="{% url 'post_new' %}">+ 새 블로그 포스트</a>
      </div>
      <!-- end new HTML... -->
    </header>
    {% block content %}{%
    endblock content %}
  </div>
</body>

</html>
```

Navigate to the homepage and the “+ New Blog Post” link in the upper right-hand corner will be visible.



Homepage with New Button

Clicking it will redirect to the create new post page at `http://127.0.0.1:8000/post/new/`.

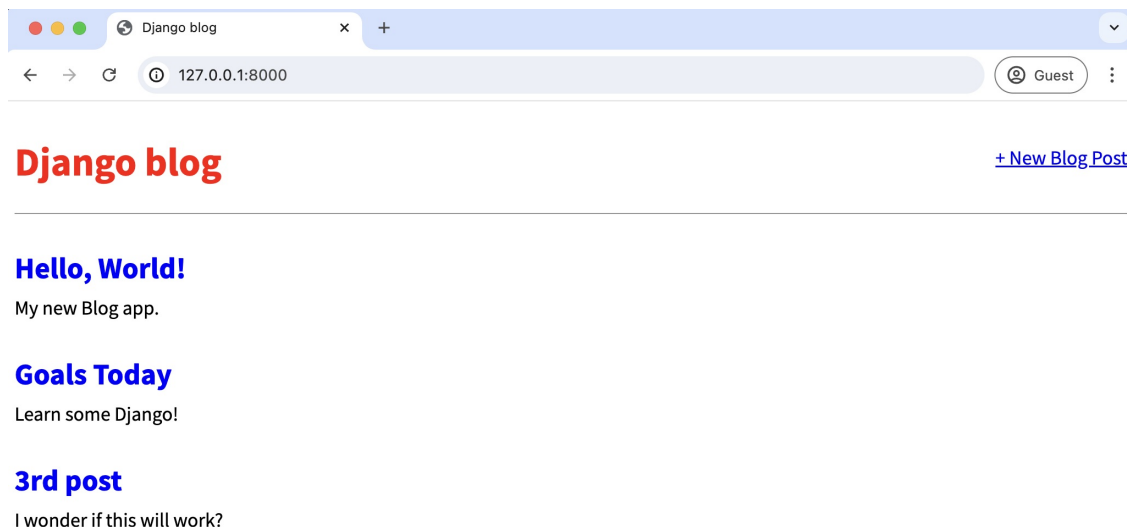
If we were to create this functionality using function-based views, we’d need a dedicated `blog/forms.py` file and a much longer view explaining how to handle the form, validate the data, and save it to the database. By using `CreateView` instead, we rely on built-in code to handle all these issues.

UpdateView

Our next task is to add a page with a form for editing existing blog posts. It turns out that the generic class-based view, [UpdateView](#), is designed explicitly for this. We will add it to our Blog, and the pattern of adding a new view, then a new URL path, and finally, a new template should feel familiar.

Let’s begin with the view. At the top of the `blog/views.py` file import `UpdateView` and then create a new class, `BlogUpdateView`, that extends it. Just as with `CreateView`, we only have to define three things: the model, template name, and fields we want displayed on the form.

홈페이지로 이동하면 오른쪽 상단 모서리에 “+ 새 블로그 게시물” 링크가 보입니다.



새 버튼이 있는 홈페이지

클릭하면 `http://127.0.0.1:8000/post/new/`의 새 게시물 작성 페이지로 리디렉션됩니다.

함수 기반 뷰를 사용하여 이 기능을 만들려면 전용 `blog/forms.py` 파일과 양식을 처리하고 데이터를 검증하며 데이터베이스에 저장하는 방법을 설명하는 훨씬 긴 뷰가 필요합니다. 대신 `CreateView`를 사용하면 이러한 모든 문제를 처리하는 내장 코드를 활용할 수 있습니다.

UpdateView

다음 작업은 기존 블로그 게시물을 편집하기 위한 양식이 있는 페이지를 추가하는 것입니다. 일반 클래스 기반 뷰인 `UpdateView`는 이를 위해 명시적으로 설계되었습니다. 이를 블로그에 추가할 것이며, 새 뷰를 추가한 다음 새 URL 경로를 추가하고 마지막으로 새 템플릿을 추가하는 패턴은 익숙할 것입니다.

뷰부터 시작하겠습니다. `blog/views.py` 파일의 상단에 `UpdateView`를 가져오고 이를 확장하는 새 클래스 `BlogUpdateView`를 생성합니다. `CreateView`와 마찬가지로 세 가지 항목만 정의하면 됩니다: 모델, 템플릿 이름 및 양식에 표시할 필드.

Code

```
# blog/views.py
from django.views.generic import ListView, DetailView
from django.views.generic.edit import CreateView, UpdateView # new

from .models import Post

class BlogListView(ListView):
    model = Post
    template_name = "home.html"

class BlogDetailView(DetailView):
    model = Post
    template_name = "post_detail.html"

class BlogCreateView(CreateView):
    model = Post
    template_name = "post_new.html"
    fields = ["title", "author", "body"]

class BlogUpdateView(UpdateView): # new
    model = Post
    template_name = "post_edit.html"
    fields = ["title", "body"]
```

Notice that we are not including “author” in the `fields` because we assume that the author of a post will not change during editing. We only want the title and body text to be available to update.

Next we need a new URL path for this update view. Update `blog/urls.py` by importing `BlogUpdateView` at the top and then create a new route with the URL pattern of `/post/pk/edit`. It will use `BlogUpdateView` as the view and have a URL named `post_edit`.

Code

```
# blog/urls.py
from django.urls import path
from .views import (
    BlogListView,
    BlogDetailView,
    BlogCreateView,
    BlogUpdateView, # new
)

urlpatterns = [
    path("post/new/", BlogCreateView.as_view(), name="post_new"),
    path("post/<int:pk>/", BlogDetailView.as_view(), name="post_detail"),
    path("post/<int:pk>/edit/", BlogUpdateView.as_view(), name="post_edit"), # new
```

코드

```
# blog/views.py
from django.views.generic import ListView, DetailView
from django.views.generic.edit import CreateView, UpdateView
from .models import Post

class BlogListView(ListView): model =
    Post
    template_name = "home.html"

class BlogDetailView(DetailView): model =
    Post
    template_name = "post_detail.html"

class BlogCreateView(CreateView): model
    = Post
    template_name = "post_new.html" fields = [
        "title", "author", "body" ]

class BlogUpdateView(UpdateView): # new
    model = Post
    template_name = "post_edit.html" fields =
        ["title", "body"]
```

저자(author)를 필드에 포함하지 않는 이유는 게시물의 저자가 편집 중에 변경되지 않을 것이라고 가정하기 때문입니다. 우리는 제목(title)과 본문(body) 텍스트만 업데이트할 수 있기를 원합니다.

다음으로 이 업데이트 뷰를 위한 새로운 URL 경로가 필요합니다. blog/urls.py를 업데이트하여 맨 위에 BlogUpdateView를 가져오고 /post/pk/edit의 URL 패턴으로 새로운 경로를 만듭니다. 이 경로는 BlogUpdateView를 뷰로 사용하고 post_edit이라는 이름의 URL을 가집니다.

Code

```
# blog/urls.py
from django.urls import path
from .views import
    (BlogListView,
     BlogDetailView,
     BlogCreateView,
     BlogUpdateView, # new
    )

urlpatterns = [
    path("post/new/", BlogCreateView.as_view(), name="post_new"), path("post/<int:pk>/", BlogDetailView.as_view(), name="post_detail"), path("post/<int:pk>/edit/", BlogUpdateView.as_view(), name="post_edit"),
    # 새로 만들기
```

```
    path("", BlogListView.as_view(), name="home"),
]
```

The third step is adding a new template for the update page. We know from our view that it should have the name `post_edit.html`, so add that file to your templates directory. It should have the following code.

Code

```
<!-- templates/post_edit.html -->
{% extends "base.html" %}

{% block content %}
<h1>Edit post</h1>
<form action="" method="post">{% csrf_token %}
    {{ form }}
    <input type="submit" value="Update">
</form>
{% endblock content %}
```

At the top, we use `extends` on the base template, `base.html`, and then sandwich this page's content between the content blocks. We again use HTML `<form>` `</form>` tags, Django's `csrf_token` for security, and give it the value "Update" on the submit button.

We could go directly to the URL for a specific post, for example, `127.0.0.1:8000/post/1/edit/` for the first blog post. However, a better approach is to add a link to the individual blog page in the `post_detail.html` template.

Code

```
<!-- templates/post_detail.html -->
{% extends "base.html" %}

{% block content %}
<div class="post-entry">
    <h2>{{ post.title }}</h2>
    <p>{{ post.body }}</p>
</div>
<!-- start new HTML... -->
<a href="{% url 'post_edit' post.pk %}">+ Edit Blog Post</a>
<!-- end new HTML... -->
{% endblock content %}
```

We've added a link using `<a href>...</a>` and the `{% url ... %}` tag. Within it, we've specified the target name of our URL, which will be called `post_edit`, and also passed the parameter needed, which is the primary key of the post: `post.pk`.

```
path("", BlogListView.as_view(), name="home"),
"]
```

세 번째 단계는 업데이트 페이지를 위한 새로운 템플릿을 추가하는 것입니다. ourview에서 post_edit.html이라는 이름을 가져야 한다는 것을 알고 있으므로, yourtemplates 디렉토리에 해당 파일을 추가하세요. 다음 코드를 포함해야 합니다.

코드

```
<!-- templates/post_edit.html -->
{% extends "base.html" %}

{% block content %}<h1>
게시물 수정</h1>
<form action="" method="post">{% csrf_token %}{% form
  %}
  <input type="submit" value="업데이트">
</form>
{% endblock content %}
```

맨 위에서 우리는 기본 템플릿인 base.html을 확장하고, 이 페이지의 내용을 콘텐츠 블록 사이에 삽입합니다. 우리는 다시 HTML <form> </form> 태그, 보안을 위한 Django의 csrf_token을 사용하고, 제출 버튼에 "Update"라는 값을 부여합니다.

우리는 특정 게시물의 URL로 직접 이동할 수 있습니다. 예를 들어, 첫 번째 블로그 게시물에 대한 127.0.0.1:8000/post/1/edit/입니다. 그러나 더 나은 접근 방식은 post_detail.html 템플릿에 개별 블로그 페이지에 대한 링크를 추가하는 것입니다.

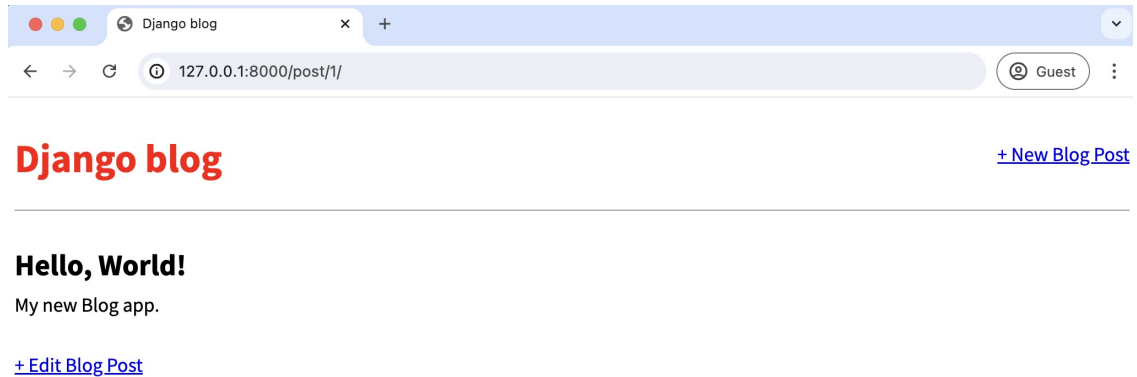
코드

```
<!-- templates/post_detail.html -->
{% extends "base.html" %}

{% block content %}
<div class="post-entry"><h2>
  {{ post.title }}</h2><p>
    {{ post.body }}</p>
</div>
<!-- start new HTML ... -->
<a href="{% url 'post_edit' post.pk %}">+ 블로그 게시물 편집</a>
<!-- end new HTML ... -->
{% endblock content %}
```

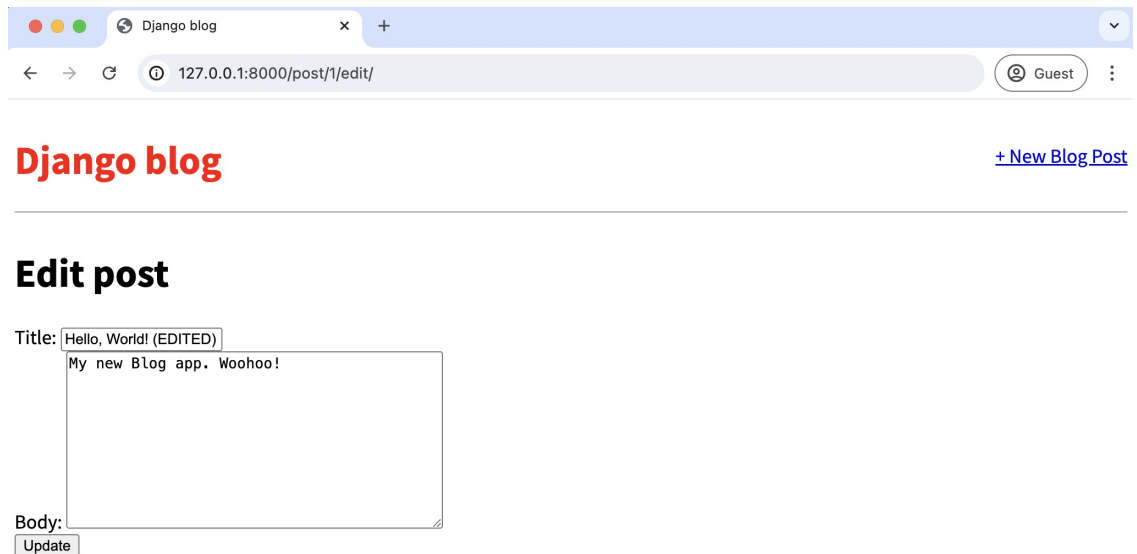
우리는 <a href>...를 사용하여 링크를 추가했으며 {% url ... %} 태그를 사용했습니다. 이 안에서 우리는 URL의 대상 이름인 post_edit를 지정했으며, 또한 필요한 매개변수인 게시물의 기본 키: post.pk를 전달했습니다.

Now, if you click on a blog entry, you'll see our new + Edit Blog Post hyperlink.



Blog Page with Edit Button

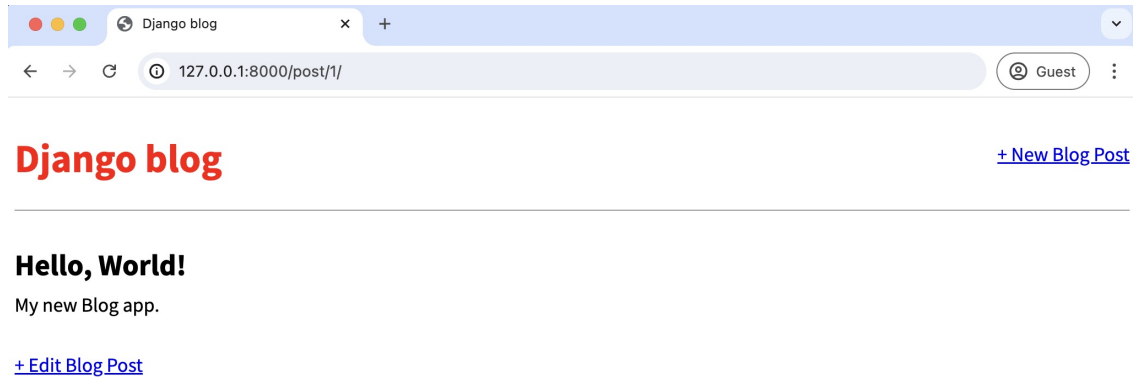
If you click on “+ Edit Blog Post” you’ll be redirected to /post/1/edit/ if it is your first blog post, hence the 1 in the URL. Note that the form is pre-filled with our database’s existing data for the post. Let’s make a change...



Blog Edit Page

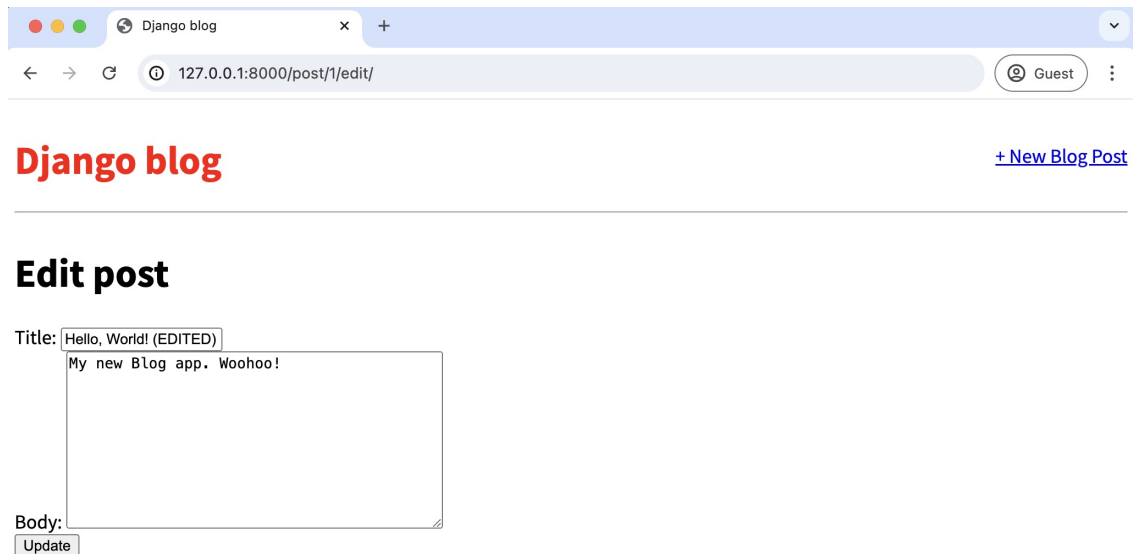
After clicking the “Update” button, we are redirected to the detail view of the post where the change is visible. Navigate to the homepage to see the change next to all the other entries.

이제 블로그 항목을 클릭하면 새로운 + 블로그 게시물 편집 하이퍼링크가 표시됩니다.



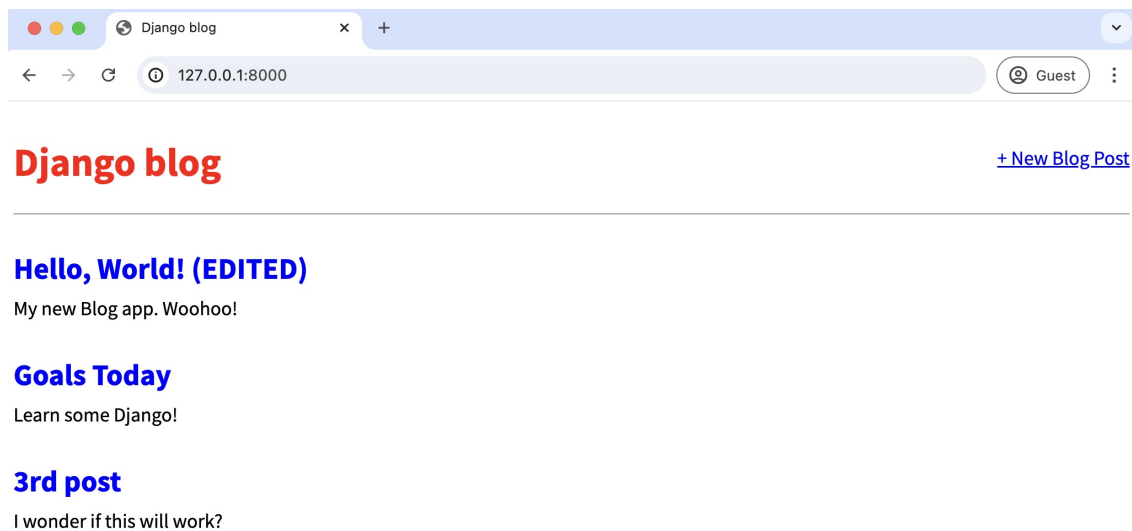
편집 버튼이 있는 블로그 페이지

“+ 블로그 게시물 편집”을 클릭하면 첫 번째 블로그 게시물인 경우 /post/1/edit/로 리디렉션됩니다. URL의 1을 참고하세요. 양식은 게시물에 대한 데이터베이스의 기존 데이터로 미리 채워져 있습니다. 변경해 보겠습니다...



블로그 편집 페이지

“업데이트” 버튼을 클릭한 후, 변경 사항이 표시되는 게시물의 세부 정보 보기로 리디렉션됩니다. 홈페이지로 이동하여 다른 모든 항목 옆에 있는 변경 사항을 확인하세요.



Blog Homepage with Edited Post

DeleteView

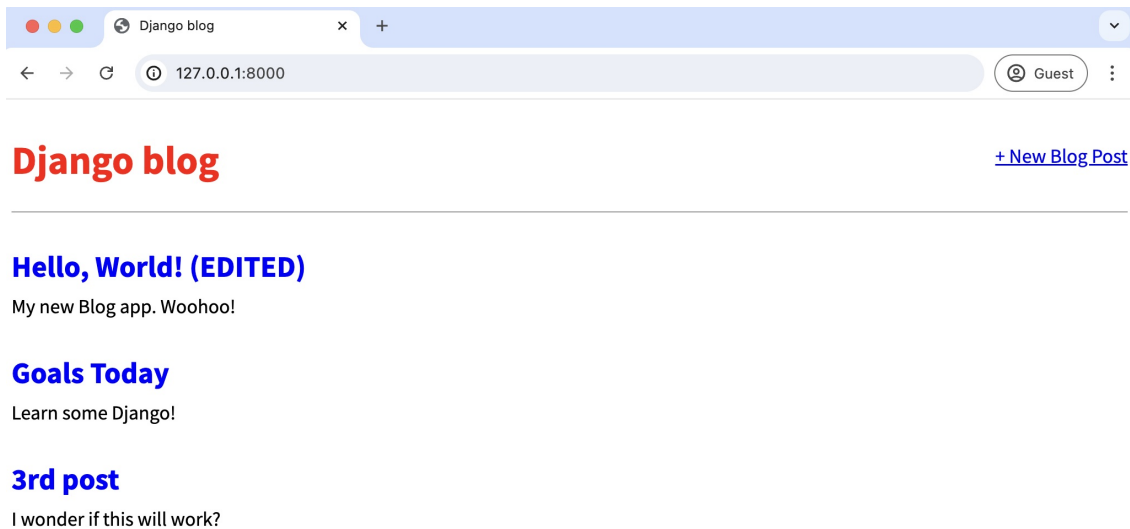
The final feature to add in this chapter is the ability to delete blog posts. We'll use yet another generic class-based view, [DeleteView](#), to do this and create the necessary view, URL, and template.

To begin, update the `blog/views.py` file by importing `DeleteView` and [reverse_lazy](#) at the top and then create a new view that subclasses `DeleteView`. Both `reverse` and `reverse_lazy` perform the same task: generating a URL based on an input like the URL name. The difference is when they are evaluated: `reverse` executes right away, so when `BlogDeleteView` is executed, immediately the `model`, `template_name`, and `success_url` methods are loaded. But the `success_url` needs to find out what the resulting URL path is associated with the URL name "home." It can't always do that in time! That's why we use `reverse_lazy` in this example: it delays the actual call to the URL dispatcher until the moment it is needed, not when our class `BlogDeleteView` is being evaluated.

Code

```
# blog/views.py
from django.views.generic import ListView, DetailView
from django.views.generic.edit import CreateView, UpdateView, DeleteView # new
from django.urls import reverse_lazy # new

from .models import Post
```



편집된 게시물이 있는 블로그 홈페이지

DeleteView

이번 장에 추가할 마지막 기능은 블로그 게시물을 삭제하는 기능입니다. 이를 위해 또 다른 제네릭 클래스 기반 뷰인 DeleteView를 사용하여 필요한 뷰, URL 및 템플릿을 생성하겠습니다.

먼저, `blog/views.py` 파일을 업데이트하여 `DeleteView`와 `reverse_lazy`를 상단에 임포트한 다음 `DeleteView`를 서브클래스하는 새로운 뷰를 생성합니다. `reverse`와 `reverse_lazy`는 모두 입력된 URL 이름과 같은 입력을 기반으로 URL을 생성하는 동일한 작업을 수행합니다. 차이점은 평가되는 시점입니다: `reverse`는 즉시 실행되므로 `BlogDeleteView`가 실행될 때 모델, `template_name` 및 `success_url` 메서드가 즉시 로드됩니다. 그러나 `success_url`은 URL 이름 "home"과 연결된 결과 URL 경로가 무엇인지 알아내야 합니다. 항상 제때 그렇게 할 수는 없습니다! 그래서 이 예제에서는 `reverse_lazy`를 사용합니다: 실제 URL 디스패처 호출을 필요할 때까지 지연시킵니다. 즉, 우리의 클래스 `BlogDeleteView`가 평가될 때가 아니라 필요할 때 호출됩니다.

코드

```
# blog/views.py
from django.views.generic import ListView, DetailView
from django.views.generic.edit import CreateView, UpdateView, DeleteView from django.urls # new
import reverse_lazy # new

from .models import Post
```

```
...
class BlogDeleteView(DeleteView): # new
    model = Post
    template_name = "post_delete.html"
    success_url = reverse_lazy("home")
```

DeleteView requires a model, template name, and success URL. We have supplied all three here. Remember, *after* a blog post is deleted, we have to *redirect* the user somewhere. In this case, that is our homepage at the URL name of “home”.

An astute reader might notice that both CreateView and UpdateView also have redirects yet we did not have to specify a success_url. Why is this? If available, Django, by default, will use the `get_absolute_url()` on the model object, a handy feature, but you would only know about it from reading a book like this or deeply reading the docs. More likely, with experience, you have used these GCBVs before and vaguely recall something about the redirect, consult the [docs on model forms](#) and then implement. There is no way to memorize everything you need to know as a Django developer; instead, with time, you’ll start to see the broad patterns and be able to look up the documentation for implementation details.

With our view complete, we can turn to the URL. It’s a similar pattern where we import the view, BlogDeleteView, set a URL path, specify the view, and include a URL name. A convention is to add `/delete/` to your URL path as we’ve done here.

Code

```
# blog/urls.py
from django.urls import path
from .views import (
    BlogListView,
    BlogDetailView,
    BlogCreateView,
    BlogUpdateView,
    BlogDeleteView, # new
)

urlpatterns = [
    path("post/new/", BlogCreateView.as_view(), name="post_new"),
    path("post/<int:pk>/", BlogDetailView.as_view(), name="post_detail"),
    path("post/<int:pk>/edit/", BlogUpdateView.as_view(),
        name="post_edit"),
    path("post/<int:pk>/delete/", BlogDeleteView.as_view(),
        name="post_delete"), # new
    path("", BlogListView.as_view(), name="home"),
]
```

```
...
class BlogDeleteView(DeleteView): # new
    model = Post
    template_name = "post_delete.html"
    success_url = reverse_lazy("home")
```

DeleteView는 모델, 템플릿 이름 및 성공 URL이 필요합니다. 우리는 여기서 세 가지를 모두 제공했습니다. 블로그 게시물이 삭제된 후 사용자를 어디론가 리디렉션해야 한다는 것을 기억하세요. 이 경우, 그것은 "home"이라는 URL 이름의 홈페이지입니다.

예리한 독자는 CreateView와 UpdateView도 리디렉션이 있지만 성공_url을 지정할 필요가 없다는 것을 알 수 있습니다. 왜 그럴까요? 사용 가능한 경우, Django는 기본적으로 모델 객체의 get_absolute_url()을 사용합니다. 이는 유용한 기능이지만, 이런 책을 읽거나 문서를 깊이 읽어야만 알 수 있습니다. 아마도 경험이 쌓이면서 이러한 GCBV를 사용해본 적이 있고 리디렉션에 대한 무언가를 희미하게 기억할 것입니다. 모델 폼에 대한 [문서를 참조한](#) 후 구현하게 됩니다. Django 개발자로서 알아야 할 모든 것을 암기할 수는 없습니다. 대신, 시간이 지나면서 넓은 패턴을 보기 시작하고 구현 세부 사항에 대한 문서를 찾아볼 수 있게 될 것입니다.

우리의 뷰가 완성되었으므로 URL로 넘어갈 수 있습니다. 뷰인 BlogDeleteView를 가져오고, URL 경로를 설정하고, 뷰를 지정하고, URL 이름을 포함하는 유사한 패턴입니다. 여기서처럼 URL 경로에 /delete/를 추가하는 것이 관례입니다.

Code

```
# blog/urls.py
from django.urls import path
from .views import
    (BlogListView,
     BlogDetailView,
     BlogCreateView,
     BlogUpdateView,
     BlogDeleteView, # new
    )

urlpatterns = [
    path("post/new/", BlogCreateView.as_view(), name="post_new"), path("post/
    <int:pk>/", BlogDetailView.as_view(), name="post_detail"), path("post/<int:pk>
    /edit/", BlogUpdateView.as_view(),
        name="post_edit"),
    path("post/<int:pk>/delete/", BlogDeleteView.as_view(), name="
        post_delete"), # new
    path("", BlogListView.as_view(), name="home"),
]
```

Lastly, we need to add a template file confirming the user's wish to delete the blog post. It will be called `templates/post_delete.html` and contain the following code:

Code

```
<!-- templates/post_delete.html -->
{% extends "base.html" %}

{% block content %}
<h1>Delete post</h1>
<form action="" method="post">{% csrf_token %}
  <p>Are you sure you want to delete "{{ post.title }}"?</p>
  <input type="submit" value="Confirm">
</form>
{% endblock content %}
```

Note we are using `post.title` here to display the title of our blog post. We could use `object.title` as it is also provided by `DetailView`.

We can add a link to delete blog posts on the individual blog page, `post_detail.html`.

Code

```
<!-- templates/post_detail.html -->
{% extends "base.html" %}

{% block content %}
<div class="post-entry">
  <h2>{{ post.title }}</h2>
  <p>{{ post.body }}</p>
</div>
<div>
  <p><a href="{% url 'post_edit' post.pk %}">+ Edit Blog Post</a></p>
  <!-- new HTML below here... -->
  <p><a href="{% url 'post_delete' post.pk %}">+ Delete Blog Post</a></p>
</div>
{% endblock content %}
```

If you start the server again with the command `python manage.py runserver` and refresh any individual post page, you'll see our "Delete Blog Post" link.

마지막으로, 사용자가 블로그 게시물을 삭제하겠다는 의사를 확인하는 템플릿 파일을 추가해야 합니다. 이 파일은 `templates/post_delete.html`이라고 불리며 다음 코드를 포함합니다:

코드

```
<!-- templates/post_delete.html -->
{% extends "base.html" %}

{% block content %}<h1>
게시물 삭제</h1>
<form action="" method="post">{% csrf_token %}
  <p>"{{ post.title }}"를 정말 삭제하시겠습니까?</p><input type="submit"
  value="확인">
</form>
{% endblock content %}
```

여기서 우리는 `post.title`을 사용하여 블로그 게시물의 제목을 표시하고 있습니다. `DetailView`에서 제공되므로 `object.title`을 사용할 수도 있습니다.

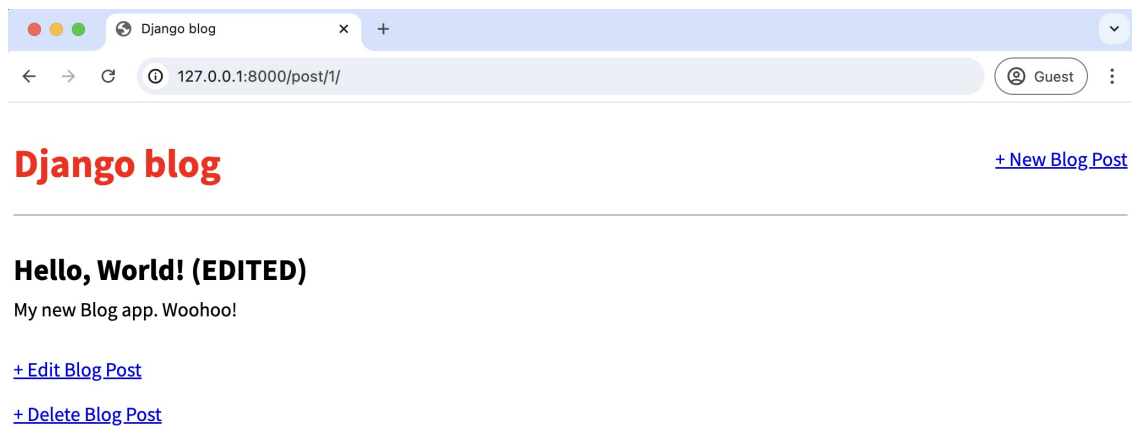
개별 블로그 페이지인 `post_detail.html`에 블로그 게시물을 삭제하는 링크를 추가할 수 있습니다.

코드

```
<!-- templates/post_detail.html -->
{% extends "base.html" %}

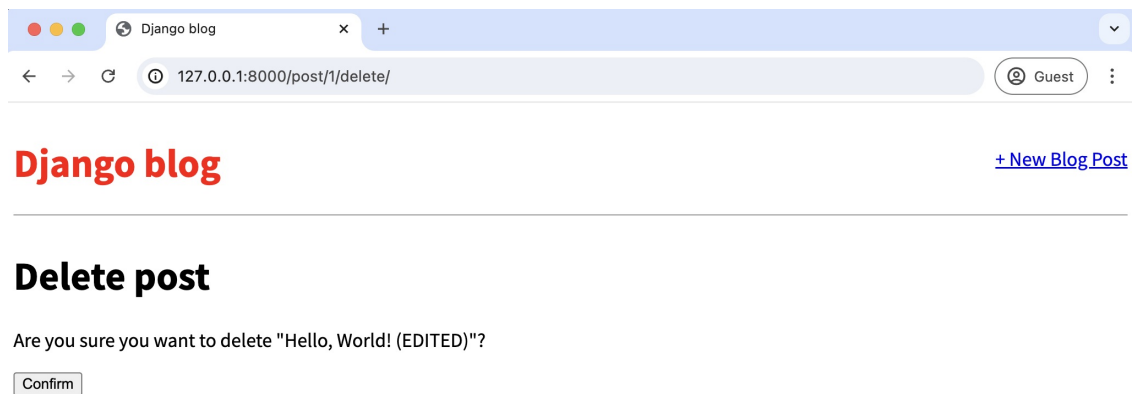
{% block content %}
<div class="post-entry"><h2>
  {{ post.title }}</h2><p>
  {{ post.body }}</p>
</div>
<div>
  <p><a href="{% url 'post_edit' post.pk %}">+ 블로그 게시물 수정</a></p>
  <!-- 아래에 새로운 HTML... -->
  <p><a href="{% url 'post_delete' post.pk %}">+ 블로그 게시물 삭제</a></p>
</div>
{% endblock content %}
```

서버를 다시 시작하고 `python manage.py runserver` 명령어로 개별 게시물 페이지를 새로 고치면 "블로그 게시물 삭제" 링크가 표시됩니다.



Blog Delete Post

Clicking on the link takes us to the delete page for the blog post, which displays the blog post's title.



Blog Delete Post Page

If you click the “Confirm” button, it redirects you to the homepage where the blog post has been deleted!



Django blog

[+ New Blog Post](#)

Hello, World! (EDITED)

My new Blog app. Woohoo!

[+ Edit Blog Post](#)[+ Delete Blog Post](#)

블로그 게시물 삭제

링크를 클릭하면 블로그 게시물의 제목이 표시되는 삭제 페이지로 이동합니다.



Django blog

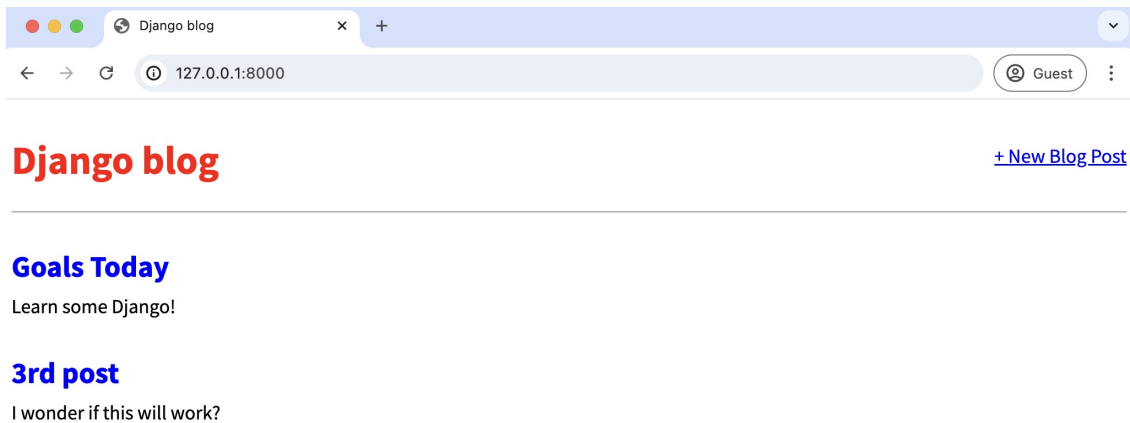
[+ New Blog Post](#)

Delete post

Are you sure you want to delete "Hello, World! (EDITED)"?

블로그 게시물 삭제 페이지

“확인” 버튼을 클릭하면 블로그 게시물이 삭제된 홈페이지로 리디렉션됩니다!



Homepage with Post Deleted

Tests

It's time for tests to make sure everything works now—and in the future—as expected. We've added new views for create, update, and delete, so that means three new tests:

- `def test_post_createview`
- `def test_post_updateview`
- `def test_post_deleteview`

Update your existing `tests.py` file with new tests below `test_post_detailview` as follows.

Code

```
# blog/tests.py
...
def test_post_createview(self): # new
    response = self.client.post(
        reverse("post_new"),
        {
            "title": "New title",
            "body": "New text",
            "author": self.user.id,
        },
    )
    self.assertEqual(response.status_code, 302)
    self.assertEqual(Post.objects.last().title, "New title")
    self.assertEqual(Post.objects.last().body, "New text")

def test_post_updateview(self): # new
    response = self.client.post(
```



Django blog

[+ New Blog Post](#)

Goals Today

Learn some Django!

3rd post

I wonder if this will work?

게시물이 삭제된 홈페이지

테스트

모든 것이 현재와 미래에 예상대로 작동하는지 확인하기 위한 테스트 시간입니다. 생성, 업데이트 및 삭제를 위한 새로운 뷰를 추가했으므로 세 가지 새로운 테스트가 필요합니다:

- `def test_post_createview`
- `def test_post_updateview`
- `def test_post_deleteview`

기존 `tests.py` 파일을 아래의 새로운 테스트로 업데이트하세요

`test_post_detailview`는 다

`# blog/tests.py`

```
...
def test_post_createview(self): # 새로 추가
    response = self.client.post(reverse(
        "post_new"),
        {
            "title": "새 제목", "body": "
            새 텍스트", "author":
            self.user.id,
        },
    )
    self.assertEqual(response.status_code, 302)
    self.assertEqual(Post.objects.last().title, "새 제목")
    self.assertEqual(Post.objects.last().body, "새 텍스트")

def test_post_updateview(self): # new
    response = self.client.post(
```

음과
같습니다
코드

```

        reverse("post_edit", args="1"),
        {
            "title": "Updated title",
            "body": "Updated text",
        },
    )
    self.assertEqual(response.status_code, 302)
    self.assertEqual(Post.objects.last().title, "Updated title")
    self.assertEqual(Post.objects.last().body, "Updated text")

def test_post_deleteview(self): # new
    response = self.client.post(reverse("post_delete", args="1"))
    self.assertEqual(response.status_code, 302)

```

For `test_post_createview`, we create a new response and check that the page has a 302 redirect status code and that the `last()` object created on our model matches the new response. Then `test_post_updateview` sees if we can update the initial post created in `setUpTestData` since that data is available throughout our entire test class. The last new test, `test_post_deleteview`, confirms that a 302 redirect occurs when deleting a post.

More tests can always be added later, such as for the templates, but at least we have some coverage of all our new functionality. Stop the local web server with `Control+c` and run these tests now. They should all pass.

Shell

```

(.venv) $ python manage.py test
Found 8 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.....
-----
Ran 8 tests in 0.101s

OK
Destroying test database for alias 'default'...

```

Conclusion

With a small amount of code, we've built a blog application that allows for creating, reading, updating, and deleting blog posts. While there are multiple ways to achieve this same functionality—we could have used function-based views or written our own class-based views—we've demonstrated how little code it takes in Django to make this happen.

Note, however, a potential security concern: currently *any* user can update or delete blog entries, not just the creator! Fortunately, Django has built-in features

```

        reverse("post_edit", args="1"), {
            "title": "업데이트된 제목", "
            body": "업데이트된 텍스트",
        },
    )
    self.assertEqual(response.status_code, 302)self.assertEqual
(Post.objects.last().title, "업데이트된 제목")self.assertEqual
(Post.objects.last().body, "업데이트된 텍스트")

def test_post_deleteview(self): # new
    response = self.client.post(reverse("post_delete", args="1"))self.assertEqual
(response.status_code, 302)

```

test_post_createview에서는 새로운 응답을 생성하고 페이지가 302 리디렉션 상태 코드를 가지고 있으며 모델에서 생성된 마지막 객체가 새로운 응답과 일치하는지 확인합니다. 그런 다음 test_post_updateview는 setUpTestData에서 생성된 초기 게시물을 업데이트할 수 있는지 확인합니다. 이 데이터는 전체 테스트 클래스에서 사용할 수 있습니다. 마지막 새로운 테스트인 test_post_deleteview는 게시물을 삭제할 때 302 리디렉션이 발생하는지 확인합니다.

더 많은 테스트는 나중에 추가할 수 있으며, 템플릿에 대한 테스트도 포함될 수 있지만, 적어도 우리는 모든 새로운 기능에 대한 일부 커버리지를 가지고 있습니다. 로컬 웹 서버를 중지하려면

Control+c를 누르고 지금 이 테스트를 실행하세요. 모두 통과해야 함

니다.Shell

```

(.venv) $ python manage.py testFound
8 test(s).
'default'에 대한 테스트 데이터베이스 생성 중... 시스템 점
검에서 문제가 발견되지 않았습니다 (0개 무시됨).....

```

-8개의 테스트를 0.101초에 실행했습니다

확인

'default'에 대한 테스트 데이터베이스 파괴 중...

결론

조금의 코드로 블로그 게시물을 생성, 읽기, 업데이트 및 삭제할 수 있는 블로그 애플리케이션을 구축했습니다. 동일한 기능을 달성하는 여러 방법이 있지만 - 함수 기반 뷰를 사용하거나 자체 클래스 기반 뷰를 작성할 수 있었지만 - Django에서 이를 실현하는 데 필요한 코드가 얼마나 적은 지 보여주었습니다.

그러나 잠재적인 보안 문제에 유의하십시오: 현재 모든 사용자가 블로그 항목을 업데이트하거나 삭제할 수 있으며, 작성자만이 아닙니다! 다행히도 Django에는 내장 기능이 있습니다.

to restrict access based on permissions, which we'll cover later in the book.

But for now, our blog application is working, and in the next chapter, we'll add user accounts so users can sign up, log in, and log out of the web app.

권한에 따라 접근을 제한하는 것, 이는 책의 후반부에서 다룰 것입니다.

하지만 지금은 우리의 블로그 애플리케이션이 작동하고 있으며, 다음 장에서는 사용자 계정을 추가하여 사용자가 웹 앱에 가입하고, 로그인하고, 로그아웃할 수 있도록 할 것입니다.

Chapter 8: User Accounts

One major area still needs to be added to our *Blog* application: user accounts. We have an author field but no way, currently, for a user to sign up, log in, log out, and so on. Implementing user authentication from scratch is famously hard and not advised! It is one area where you really don't want to make a mistake because of all the resulting security implications. Fortunately, Django has a powerful, battle-tested, built-in [user authentication system](#) that we can use and customize as needed.

Whenever you create a new project, by default, Django installs the auth app, which provides us with a [User object](#) containing:

- username
- password
- email
- first_name
- last_name

We will use this User object to implement login, logout, and signup in our blog application.

Log In

Django provides us with a default view for a login page via [LoginView](#). All we need to add is a URL pattern for the auth system, a login template, and a minor update to our `django_project/settings.py` file. Note that this code exists at the project-level and not within the blog app.

First, update the `django_project/urls.py` file. We'll place our login and logout pages at the `accounts/` URL: a one-line addition to the next-to-last line.

Code

```
# django_project/urls.py
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("accounts/", include("django.contrib.auth.urls")), # new
```

8장: 사용자 계정

아직 우리 블로그 애플리케이션에 추가해야 할 주요 부분이 하나 있습니다: 사용자 계정입니다. 저자 필드는 있지만 현재 사용자가 회원가입, 로그인, 로그아웃 등을 할 수 있는 방법이 없습니다. 사용자 인증을 처음부터 구현하는 것은 매우 어렵고 권장되지 않습니다! 보안과 관련된 모든 결과적인 문제 때문에 실수를 하고 싶지 않은 부분입니다. 다행히도 Django에는 강력하고 검증된 내장 사용자 인증 시스템이 있어 필요에 따라 사용하고 맞춤 설정할 수 있습니다.

새 프로젝트를 생성할 때마다 기본적으로 Django는 auth 앱을 설치하며, 이 앱은 다음을 포함하는 User 객체를 제공합니다:

- 사용자
- 이름비밀
- 번호이메일
- 일이름성
-

우리는 이 User 객체를 사용하여 블로그 애플리케이션에서 로그인, 로그아웃 및 회원가입을 구현할 것입니다.

로그인

Django는 LoginView를 통해 로그인 페이지에 대한 기본 뷰를 제공합니다. 우리가 추가해야 할 것은 인증 시스템을 위한 URL 패턴, 로그인 템플릿, 그리고 `django_project/settings.py` 파일에 대한 약간의 업데이트뿐입니다. 이 코드는 블로그 앱 내에 있는 것이 아니라 프로젝트 수준에 존재한다는 점에 유의하세요.

먼저, `django_project/urls.py` 파일을 업데이트합니다. 로그인과 로그아웃을 배치할 것입니다. `accounts/` URL의 페이지: 끝에서 두 번째 줄에 한 줄 추가. Code

```
# django_project/urls.py
...
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("accounts/", include("django.contrib.auth.urls")),  # new
```

```
    path("", include("blog.urls")),  
]
```

As the [LoginView](#) documentation notes, by default, Django will look within a templates directory called registration for a file called login.html for a login form. So we need to create a new directory within templates called registration and the requisite file within it. From the command line, type Control+c to quit our local server. Then, create the new directory.

Shell

```
(.venv) $ mkdir templates/registration
```

Then, with your text editor, create a new template file, templates/registration/login.html, filled with the following code:

Code

```
<!-- templates/registration/login.html -->  
{% extends "base.html" %}  
  
{% block content %}  
<h2>Log In</h2>  
<form method="post">{% csrf_token %}  
    {{ form }}  
    <button type="submit">Log In</button>  
</form>  
{% endblock content %}
```

We’re using HTML <form></form> tags and specifying the POST method since we’re sending data to the server (we’d use GET if we were requesting data, such as in a search engine form). We add {% csrf_token %} for security concerns to prevent a CSRF Attack and then include a “submit” button.

The final step is to specify *where* to redirect the user upon successful login. We can set this with the LOGIN_REDIRECT_URL setting. At the bottom of the django_project/settings.py file, add the following:

Code

```
# django_project/settings.py  
LOGIN_REDIRECT_URL = "home" # new
```

Now the user will be redirected to the “home” template, which is our homepage. And we’re actually done at this point! If you now start up the Django server again with python manage.py runserver and navigate to our login page at http://127.0.0.1:8000/accounts/login/, you’ll see the following:

```
path("", include("blog.urls")), ]
```

로그인 뷰 문서에서 언급했듯이, 기본적으로 Django는 loginform을 위한 login.html이라는 파일을 registration이라는 템플릿 디렉토리 내에서 찾습니다. 따라서 templates 내에 registration이라는 새 디렉토리를 만들고 그 안에 필요한 파일을 생성해야 합니다. 명령줄에서 Control+c를 입력하여 로컬 서버를 종료합니다. 그런 다음 새 디렉토리를 만듭니다.

셸

```
(. venv) $ mkdir templates/registration
```

그런 다음 텍스트 편집기를 사용하여 새 템플릿 파일을 생성합니다, templates/registration/login.html, 다음 코드를 채워 넣습니다:코드

```
<!-- templates/registration/login.html -->
{% extends "base.html" %}

{% block content %}<h2>
로그인</h2>
<form method="post">{% csrf_token %}{% form
    %}
    <button type="submit">로그인</button>
</form>
{% endblock content %}
```

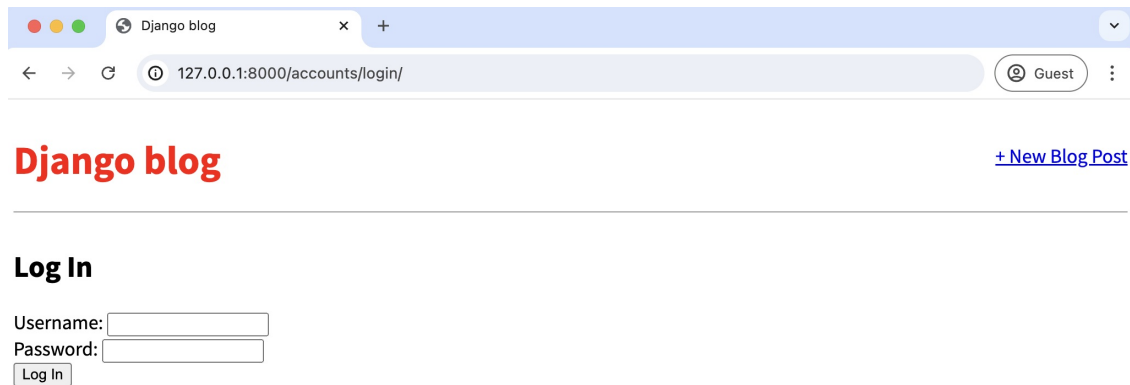
우리는 데이터를 서버로 전송하기 때문에 HTML <form></form> 태그를 사용하고 POST 방식을 지정합니다(검색 엔진 폼과 같이 데이터를 요청하는 경우에는 GET 방식을 사용합니다). 보안 문제로 CSRF 공격을 방지하기 위해 {% csrf_token %}을 추가하고, 그 다음에 "submit" 버튼을 포함합니다.

마지막 단계는 사용자가 로그인에 성공했을 때 어디로 리디렉션할지 지정하는 것입니다. 이는 LOGIN_REDIRECT_URL 설정으로 지정할 수 있습니다. 맨 아래에 있는...

django_project/settings.py 파일에 다음을 추가하세요:Code

```
# django_project/settings.pyLOGIN_REDIRECT_URL
= "home" # 새로 추가됨
```

이제 사용자는 우리의 홈페이지인 "home" 템플릿으로 리디렉션됩니다. 그리고 사실 이 시점에서 작업이 완료되었습니다! 이제 python manage.py runserver로 다시 Django 서버를 시작하고 http://127.0.0.1:8000/accounts/login/에 로그인 페이지로 이동하면 다음과 같은 화면을 볼 수 있습니다:



Login Page

Upon entering the login info for our superuser account, we are redirected to the homepage. Notice that we didn't add any *view* logic or create a database model because the Django auth system automatically provided both for us. Thanks, Django!

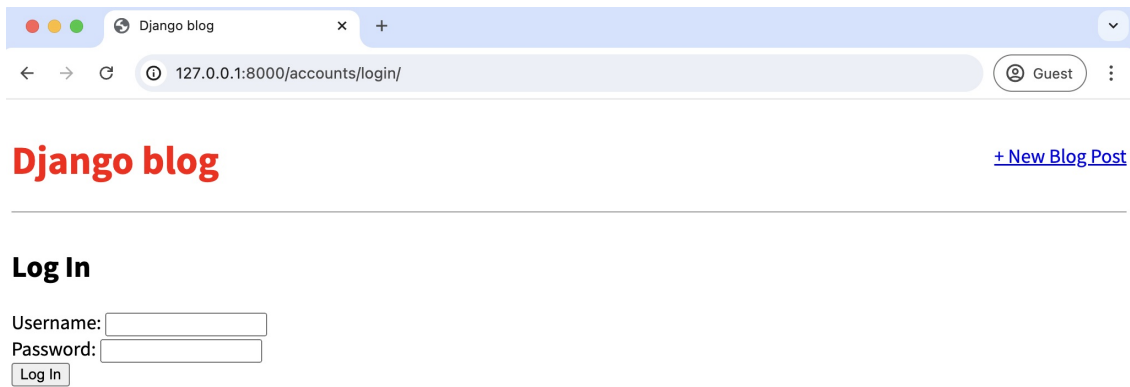
Updated Homepage

Let's update our `base.html` template so we display a message to users regardless of whether they are logged in. We can use the `is_authenticated` attribute for this.

For now, place this code in a prominent position. Later on, we can style it more appropriately. Update the `base.html` file with new code starting beneath the closing `</header>` tag.

Code

```
<!-- templates/base.html -->
...
<body>
  <div>
    <header>
      <div class="nav-left">
        <h1><a href="{% url 'home' %}">Django blog</a></h1>
      </div>
      <div class="nav-right">
        <a href="{% url 'post_new' %}">+ New Blog Post</a>
      </div>
    </header>
    <!-- start new HTML... -->
    {% if user.is_authenticated %}
    <p>Hi {{ user.username }}!</p>
    {% else %}
    <p>You are not logged in.</p>
    <a href="{% url 'login' %}">Log In</a>
```



Django blog

127.0.0.1:8000/accounts/login/

+ New Blog Post

Django blog

Log In

Username:

Password:

로그인 페이지

슈퍼유저 계정의 로그인 정보를 입력하면 홈페이지로 리디렉션됩니다. Django 인증 시스템이 자동으로 두 가지를 제공했기 때문에 뷰 로직을 추가하거나 데이터베이스 모델을 생성하지 않았습니다. 감사합니다, Django!

업데이트된 홈페이지

사용자가 로그인했는지 여부에 관계없이 메시지를 표시할 수 있도록 base.html 템플릿을 업데이트합니다. 이를 위해 is_authenticated 속성을 사용할 수 있습니다.

지금 이 코드를 눈에 띄는 위치에 배치하세요. 나중에 더 적절하게 스타일을 지정할 수 있습니다. </header> 태그 아래에서 시작하는 새로운 코드로 base.html 파일을 업데이트하세요.

코드

```
<!-- templates/base.html -->
...
<body>
  <div>
    > <header><div class="nav-
      left">
        <h1><a href="{% url 'home' %}">장고 블로그</a></h1>
      </div><div class="nav-
        right">
          <a href="{% url 'post_new' %}">+ 새 블로그 포스트</a>
        </div>
      </header>
      <!-- start new HTML ... -->
      {% if user.is_authenticated %}<p>안녕
      하세요 {{ user.username }}!</p>{%
      else %}
      <p>로그인하지 않았습니다.</p><a href="{% url
      'login' %}">로그인</a>
```

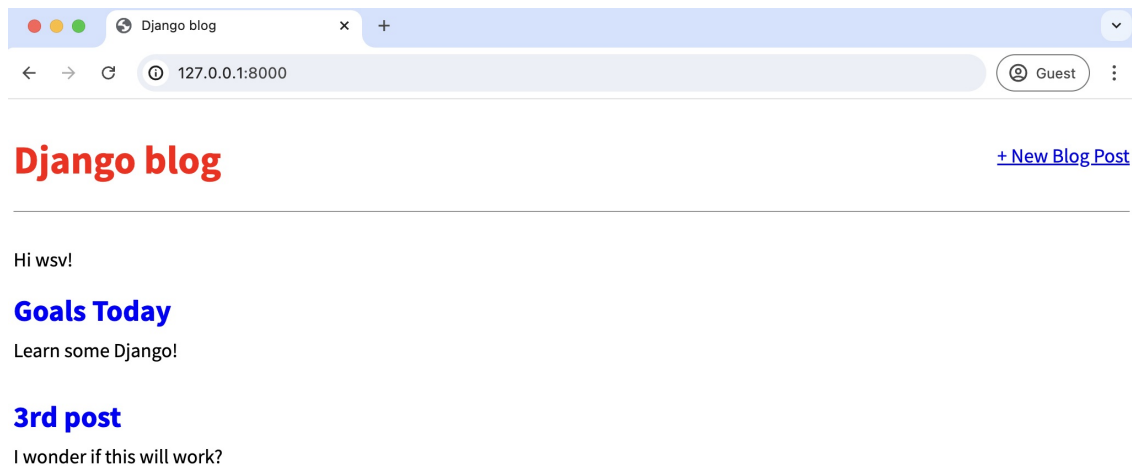
```

{% endif %}
<!-- end new HTML... -->
{% block content %}
{% endblock content %}
</div>
</body>

</html>

```

If the user is logged in, we say hello to them by name; if not, we provide a link to our newly created login page.



Homepage Logged In

It worked! My superuser name is wsv, which I see on the page.

Log Out Link

One of the major changes to Django 5.0, as noted in the [release notes](#), is the removal support for logging out via GET requests. Previously, you could add a logout link such as `<a href="{% url 'logout' %}">Log Out</a>` to a template file. But now a POST request via a form is required.

Add a logout button to the `base.html` file under the “Hi `{{ user.username }}`!” section.

Shell

```

<!-- templates/base.html -->
...
{% if user.is_authenticated %}
<p>Hi {{ user.username }}!</p>

```

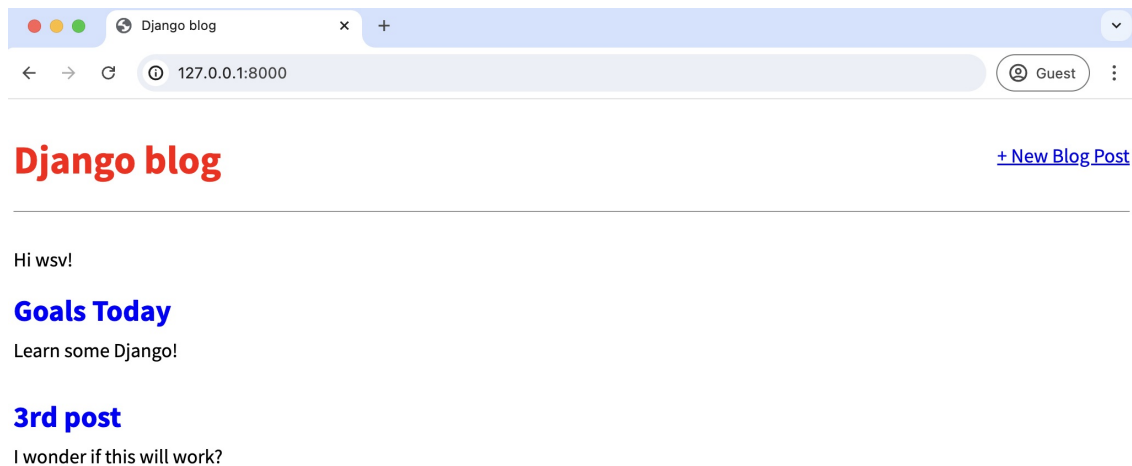
```

{% endif %}
<!-- end new HTML ... -->
{% block content %}{%
endblock content %}
</div>
</body>

</html>

```

사용자가 로그인하면 이름으로 인사합니다. 그렇지 않으면 새로 생성된 로그인 페이지로의 링크를 제공합니다.



홈페이지 로그인됨

작동했습니다! 내 슈퍼유저 이름은 wsv이며, 페이지에서 확인할 수 있습니다.

로그아웃 링크

Django 5.0에 대한 주요 변경 사항 중 하나는 [릴리스 노트에 언급된](#) 대로 GET 요청을 통한 로그아웃 지원이 제거된 것입니다. 이전에는 `<a href="{% url 'logout' %}" %}>로그아웃</a>`와 같은 로그아웃 링크를 템플릿 파일에 추가할 수 있었습니다. 그러나 이제는 양식을 통한 POST 요청이 필요합니다.

"Hi {{ user.username }}" 섹션 아래의 base.html 파일에 로그아웃 버튼을 추가하세요.

Shell

```

<!-- templates/base.html -->
...{% if user.is_authenticated %}<p>
안녕하세요 {{ user.username }}!
</p>

```

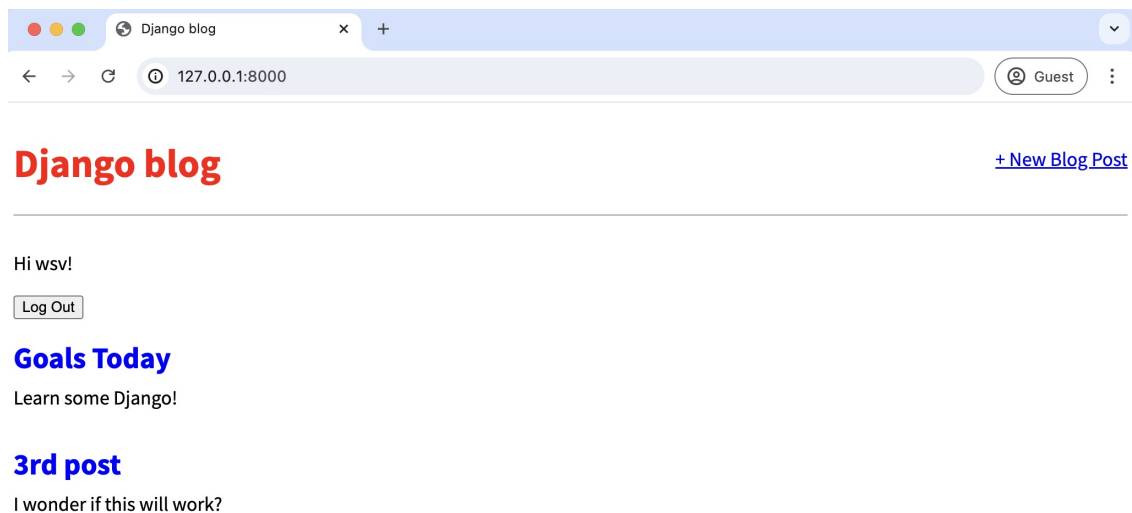
```
<form action="{% url 'logout' %}" method="post">
  {% csrf_token %}
  <button type="submit">Log Out</button>
</form>
{% else %}
...
```

The Django auth app provides us with the necessary view, so all we need to do is specify where to redirect a user when logging out. Update `django_project/settings.py` to provide a redirect link called `LOGOUT_REDIRECT_URL`. We can add it right next to our login redirect, so the bottom of the file should look as follows:

Code

```
# django_project/settings.py
LOGIN_REDIRECT_URL = "home"
LOGOUT_REDIRECT_URL = "home" # new
```

If you refresh the homepage, you'll see it now has a “log out” button for logged-in users.



Homepage Log Out Link

And clicking it takes you back to the homepage with a login link.

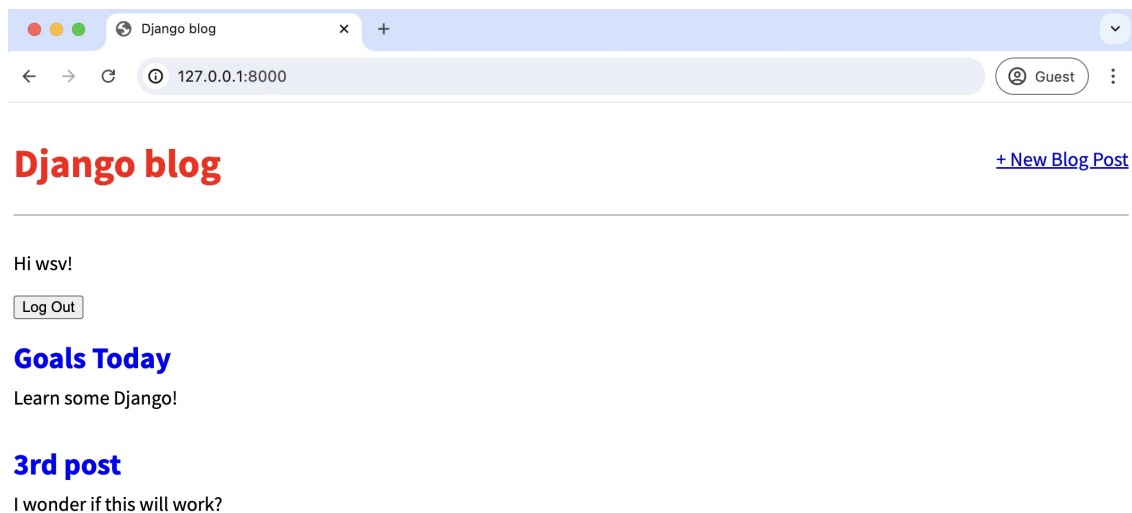
```
<form action="{% url 'logout' %}" method="post">
  {% csrf_token %}
  <button type="submit">로그 아웃</button>
</form>
{% else %}
{% ... %}
```

Django 인증 앱은 필요한 뷰를 제공하므로, 로그아웃할 때 사용자를 리디렉션할 위치만 지정하면 됩니다. `django_project/settings.py`를 업데이트하여 `LOGOUT_REDIRECT_URL`이라는 리디렉션 링크를 제공하세요. 로그인 리디렉션 바로 옆에 추가할 수 있으므로 파일의 하단은 다음과 같아야 합니다:

코드

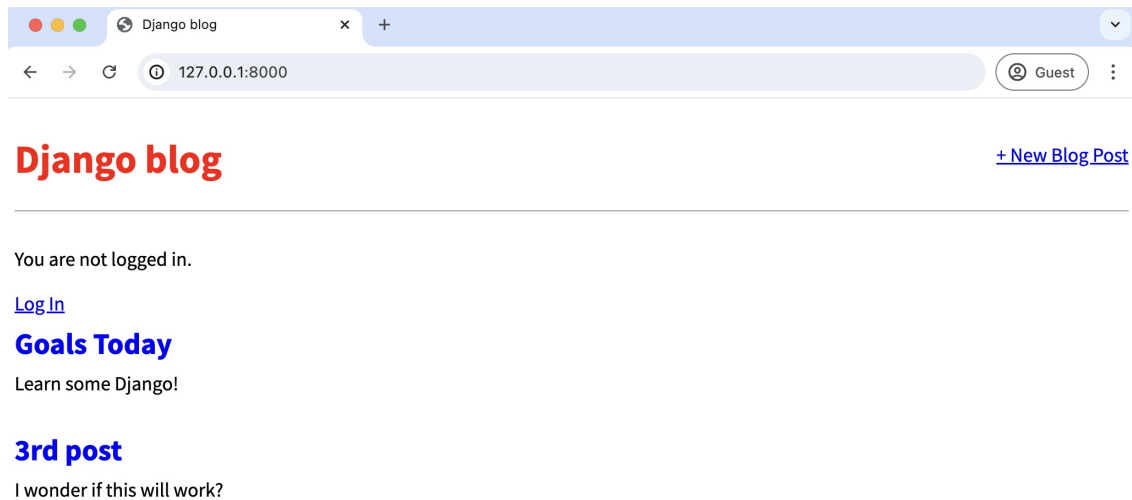
```
# django_project/settings.py
LOGIN_REDIRECT_URL = "home"
LOGOUT_REDIRECT_URL = "home" # 새로
```

홈페이지를 새로 고치면 로그인한 사용자에게 '로그 아웃' 버튼이 생긴 것을 볼 수 있습니다.



홈페이지 로그아웃 링크

그리고 클릭하면 로그인 링크가 있는 홈페이지로 돌아갑니다.



Homepage Logged Out

Try logging in and out several times with your user account.

Sign Up

Django does not provide a built-in view, URL, or template for signup. A common approach is to create a dedicated app, called `accounts` here, but also sometimes users in other projects, for this. That way, if we need to make further customizations to the user authentication process in the future, it is only contained in one place.

On the command line, stop the local server with `Control+c` and create a dedicated new app, `accounts`.

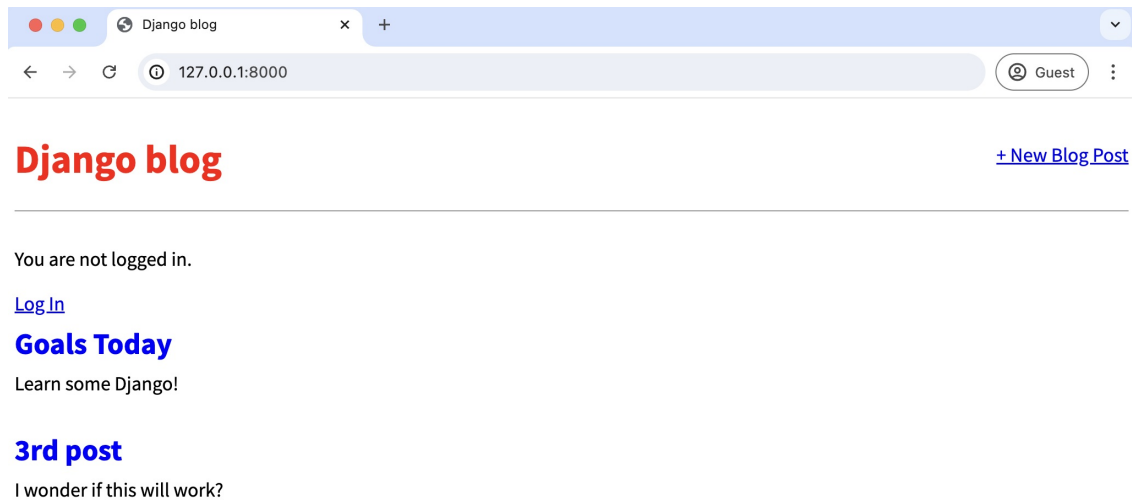
Shell

```
(.venv) $ python manage.py startapp accounts
```

Then, add the new app to the `INSTALLED_APPS` setting in our `django_project/settings.py` file so it is registered with Django.

Code

```
# django_project/settings.py
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
```



홈페이지 로그아웃됨

사용자 계정으로 여러 번 로그인하고 로그아웃해 보세요.

가입하기

Django는 회원가입을 위한 내장 뷰, URL 또는 템플릿을 제공하지 않습니다. 일반적인 방법은 여기서 accounts라고 하는 전용 앱을 만드는 것이지만, 때로는 다른 프로젝트의 사용자들을 위해서도 만듭니다. 이렇게 하면 향후 사용자 인증 프로세스에 추가적인 맞춤화가 필요할 때 한 곳에만 집중되어 관리할 수 있습니다.

명령줄에서 Control+c를 눌러 로컬 서버를 중지하고, 새로운 전용 앱인 accounts를 만드세요.

셸

```
(. venv) $ python manage.py startapp accounts
```

그런 다음, 새 앱을 INSTALLED_APPS 설정에 추가합니다. django_project/settings.py 파일이 Django에 등록되도록 합니다. Code

```
# django_project/settings.py
...
INSTALLED_APPS = [
    django.contrib.admin",
    django.contrib.auth",
    django.contrib.contenttypes",
    django.contrib.sessions",
    django.contrib.messages",
```

```
    "django.contrib.staticfiles",
    "blog",
    "accounts", # new
]
```

Next, add a new URL path in `django_project/urls.py` pointing to this new app directly **below** where we include the built-in auth app.

Code

```
# django_project/urls.py
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("accounts/", include("django.contrib.auth.urls")),
    path("accounts/", include("accounts.urls")), # new
    path("", include("blog.urls")),
]
```

The order of our `urls` matters here because Django reads this file from top to bottom. Therefore when we request the `/accounts/signup` url, Django will first look in `auth`, not find it, and **then** proceed to the `accounts` app.

In your text editor, create a file called `accounts/urls.py` and add the following code:

Code

```
# accounts/urls.py
from django.urls import path
from .views import SignUpView

urlpatterns = [
    path("signup/", SignUpView.as_view(), name="signup"),
]
```

We're using a not-yet-created view called `SignUpView`, which we already know is class-based since it is capitalized and has the `as_view()` suffix. Its path is just `signup/`, so the complete URL path will be `accounts/signup/`.

For the view, Django has a built-in form class, [UserCreationForm](#), that comes with three fields: `username`, `password1`, and `password2`. We can use it with `CreateView` to create our signup page. Replace the default `accounts/views.py` code with the following:

Code

```
"django.contrib.staticfiles", "blog",  
    "accounts", # 새로 추가  
]
```

다음으로, `django_project/urls.py`에 이 새로운 앱을 가리키는 새로운 URL 경로를 추가합니다. 내장된 인증 앱을 포함하는 바로 아래에 위치시킵니다.

코드

```
# django_project/urls.py  
from django.contrib import admin  
from django.urls import path, include  
  
urlpatterns = [  
    path("admin/", admin.site.urls),  
    path("accounts/", include("django.contrib.auth.urls")),  
    path("accounts/", include("accounts.urls")), # new  
    path("", include("blog.urls")),  
]
```

우리의 URL 순서는 중요합니다. Django는 이 파일을 위에서 아래로 읽기 때문입니다. 따라서 `/accounts/signup` URL을 요청할 때 Django는 먼저 `auth`를 확인하고 찾지 못한 후 `accounts` 앱으로 진행합니다.

텍스트 편집기에서 `accounts/urls.py`라는 파일을 만들고 다음 코드를 추가하세요:

코드

```
# accounts/urls.py  
from django.urls import path  
from .views import SignUpView  
  
urlpatterns = [  
    path("signup/", SignUpView.as_view(), name="signup"),  
]
```

우리는 아직 생성되지 않은 `SignUpView`라는 뷰를 사용하고 있으며, 이는 대문자로 시작하고 `as_view()` 접미사가 있기 때문에 클래스 기반임을 이미 알고 있습니다. 그 경로는 단지 `signup/`이므로 전체 URL 경로는 `accounts/signup/`이 됩니다.

뷰를 위해 Django는 `username`, `password1`, `password2`의 세 가지 필드가 있는 내장 폼 클래스인 `UserCreationForm`을 제공합니다. 우리는 `CreateView`와 함께 사용하여 가입 페이지를 만들 수 있습니다. 기본 `accounts/views.py` 코드를 다음으로 교체하세요:

코드

```
# accounts/views.py
from django.contrib.auth.forms import UserCreationForm
from django.urls import reverse_lazy
from django.views.generic import CreateView

class SignUpView(CreateView):
    form_class = UserCreationForm
    success_url = reverse_lazy("login")
    template_name = "registration/signup.html"
```

We're subclassing the generic class-based view `CreateView` in our `SignUpView` class and specifying `UserCreationForm`, the not-yet-created template `registration/signup.html`. We also use `reverse_lazy` to redirect the user to the login page upon successful registration.

Why use `reverse_lazy` here instead of `reverse`? The URLs are not loaded when the file is imported for generic class-based views, so we have to use the lazy form of `reverse` to load them later when they're available.

Create the file `templates/registration/signup.html` in your text editor and populate it with the code below.

Code

```
<!-- templates/registration/signup.html -->
{% extends "base.html" %}

{% block content %}
<h2>Sign Up</h2>
<form method="post">{% csrf_token %}
  {{ form.as_p }}
  <button type="submit">Sign Up</button>
</form>
{% endblock content %}
```

This format is very similar to what we've done before. We extend our base template at the top, place our logic between `<form></form>` tags with the added `as_p` to render it with paragraph tags, use the `csrf_token` for security, and include a submit button.

We're done! To test it out, start the local server with the `python manage.py runserver` command and navigate to `http://127.0.0.1:8000/accounts/signup/`.

```
# accounts/views.py
from django.contrib.auth.forms import UserCreationForm
from django.urls import reverse_lazy
from django.views.generic import CreateView
```

```
class SignUpView(CreateView):
    form_class = UserCreationForm
    success_url = reverse_lazy("login")
    template_name = "registration/signup.html"
```

우리는 SignUpView 클래스에서 일반 클래스 기반 뷰 CreateView를 서브 클래스화하고, 아직 생성되지 않은 템플릿 registration/signup.html을 지정하며 UserCreationForm을 사용하고 있습니다. 또한 성공적인 등록 후 사용자를 로그인 페이지로 리디렉션하기 위해 reverse_lazy를 사용합니다.

왜 여기서 reverse 대신 reverse_lazy를 사용할까요? 일반 클래스 기반 뷰에 대한 파일이 가져올 때 URL이 로드되지 않기 때문에 나중에 사용할 수 있을 때 로드하기 위해 reverse의 지연 형식을 사용해야 합니다.

텍스트 편집기에서 templates/registration/signup.html 파일을 만들고 아래 코드를 채워 넣으세요.

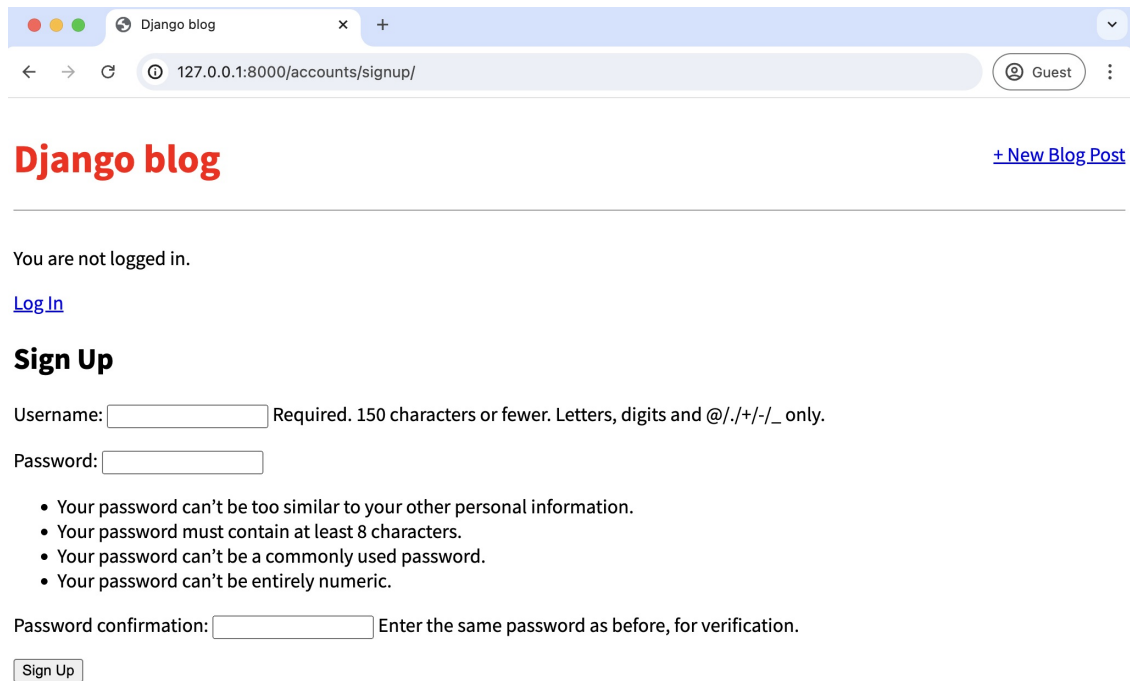
코드

```
<!-- templates/registration/signup.html -->
{% extends "base.html" %}

{% block content %}
<h2>
회원 가입</h2>
<form method="post">
    {{ csrf_token }}
    {{ form.as_p }}
    <button type="submit">회원 가입</button>
</form>
{% endblock content %}
```

이 형식은 우리가 이전에 했던 것과 매우 유사합니다. 우리는 맨 위에서 기본 템플릿을 확장하고, <form></form> 태그 사이에 논리를 배치하며, 단락 태그로 렌더링하기 위해 addedas_p를 사용하고, 보안을 위해 csrf_token을 사용하며, 제출 버튼을 포함합니다.

완료되었습니다! 테스트하려면 python manage.py runserver 명령으로 로컬 서버를 시작하고 http://127.0.0.1:8000/accounts/signup/로 이동하세요.



The screenshot shows a web browser window with the title "Django blog". The address bar displays "127.0.0.1:8000/accounts/signup/". The page content includes the "Django blog" header in red, a "+ New Blog Post" link, and a message "You are not logged in." with a "Log In" link. The "Sign Up" section contains a "Username:" label, a text input field, and a requirement note: "Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only." Below this is a "Password:" label, a text input field, and a list of four password requirements: "Your password can't be too similar to your other personal information.", "Your password must contain at least 8 characters.", "Your password can't be a commonly used password.", and "Your password can't be entirely numeric." A "Password confirmation:" label is followed by another text input field and the instruction "Enter the same password as before, for verification." At the bottom of the form is a "Sign Up" button.

Django blog [+ New Blog Post](#)

You are not logged in.
[Log In](#)

Sign Up

Username: Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

Password:

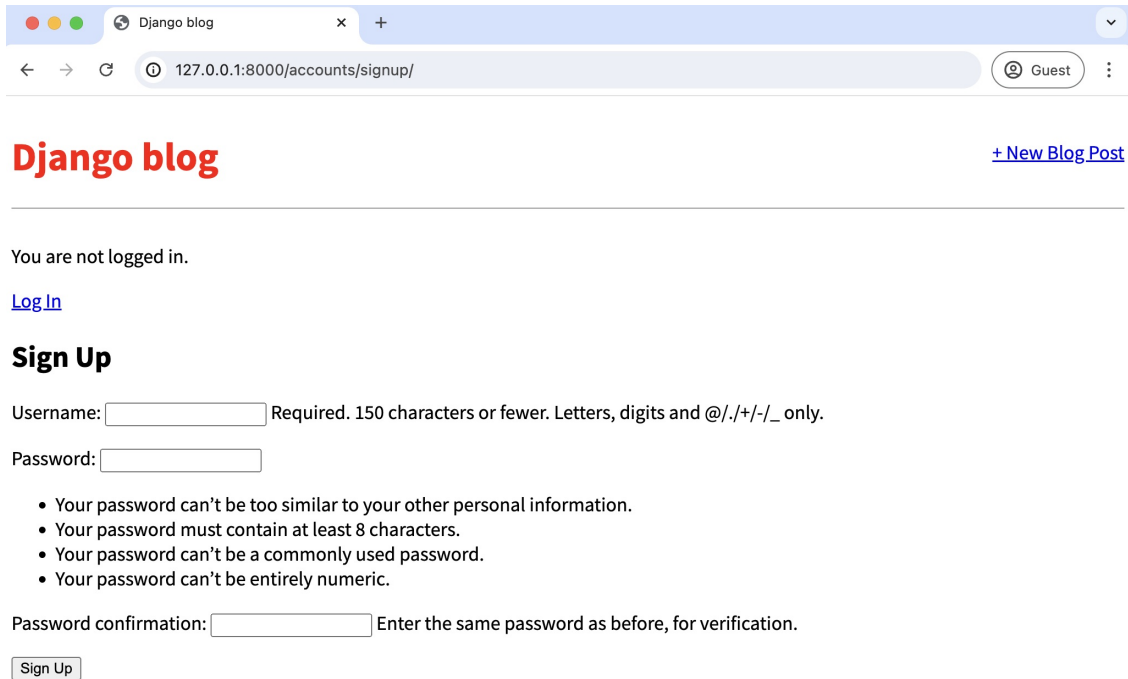
- Your password can't be too similar to your other personal information.
- Your password must contain at least 8 characters.
- Your password can't be a commonly used password.
- Your password can't be entirely numeric.

Password confirmation: Enter the same password as before, for verification.

Django Signup Page

Notice the large amount of extra text that Django includes by default. We can customize this using something like the built-in [messages framework](#), but for now, try out the form.

I created a new user called “william” and was redirected to the login page upon submission. After logging in successfully with my new username and password, I was redirected to the homepage with our personalized “Hi username” greeting.



The screenshot shows a web browser window with the title 'Django blog'. The address bar displays '127.0.0.1:8000/accounts/signup/'. The page content includes the 'Django blog' title in red, a '+ New Blog Post' link, and a message 'You are not logged in.' with a 'Log In' link. The 'Sign Up' section contains a 'Username' field with a required field message, a 'Password' field, a list of password requirements, a 'Password confirmation' field, and a 'Sign Up' button.

Django blog [+ New Blog Post](#)

You are not logged in.

[Log In](#)

Sign Up

Username: Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

Password:

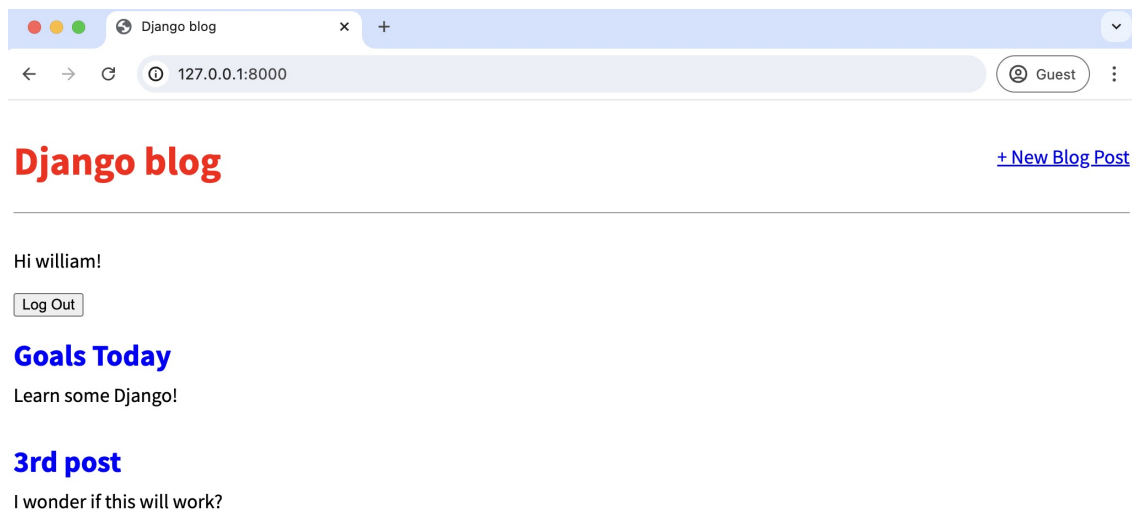
- Your password can't be too similar to your other personal information.
- Your password must contain at least 8 characters.
- Your password can't be a commonly used password.
- Your password can't be entirely numeric.

Password confirmation: Enter the same password as before, for verification.

장고 가입 페이지

장고가 기본적으로 포함하는 많은 추가 텍스트에 주목하세요. 내장된 메시지 프레임워크와 같은 것을 사용하여 이를 사용자 정의할 수 있지만, 지금은 양식을 사용해 보세요.

나는 'william'이라는 새 사용자를 만들었고 제출 후 로그인 페이지로 리디렉션되었습니다. 새 사용자 이름과 비밀번호로 성공적으로 로그인한 후, 개인화된 '안녕하세요 사용자 이름' 인사말과 함께 홈페이지로 리디렉션되었습니다.



Homepage for User William

Our ultimate flow is, therefore: Signup -> Login -> Homepage. And, of course, we can tweak this however we want. The `SignUpView` redirects to login because we set `success_url = reverse_lazy('login')`. The Login page redirects to the homepage because in our `django_project/settings.py` file, we set `LOGIN_REDIRECT_URL = 'home'`.

It may initially be overwhelming to keep track of all the various parts of a Django project, but that's normal. With time, they'll start to make more sense.

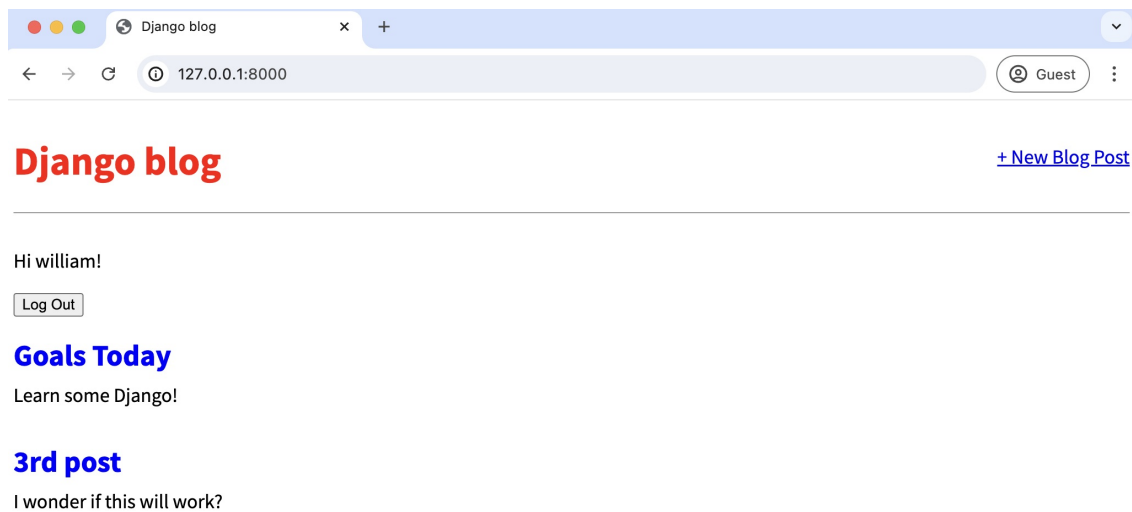
Sign Up Link

One last improvement we can make is to add a signup link to the logged-out homepage. We can't expect our users to know the correct URL after all! How do we do this? Well, we need to figure out the URL name, and then we can pop it into our template. In `accounts/urls.py`, we provided it the name of `signup`, so that's all we need to add to our `base.html` template with the [url template tag](#) just as we've done for our other links.

Add the link for “Sign Up” just below the existing link for “Log In” as follows:

Shell

```
<!-- templates/base.html -->
...
<p>You are not logged in.</p>
<a href="{% url 'login' %}">Log In</a> |
```



사용자 윌리엄의 홈페이지

따라서 우리의 궁극적인 흐름은 다음과 같습니다: 가입 -> 로그인 -> 홈페이지. 물론, 우리는 이것을 원하는 대로 조정할 수 있습니다. SignUpView는 `success_url = reverse_lazy('login')`을 설정했기 때문에 로그인으로 리디렉션됩니다. 로그인 페이지는 `django_project/settings.py` 파일에서 `LOGIN_REDIRECT_URL = 'home'`을 설정했기 때문에 홈페이지로 리디렉션됩니다.

Django 프로젝트의 다양한 부분을 추적하는 것이 처음에는 압도적일 수 있지만, 그것은 정상입니다. 시간이 지나면 더 이해가 될 것입니다.

가입 링크

우리가 할 수 있는 마지막 개선 사항은 로그아웃된 홈페이지에 가입 링크를 추가하는 것입니다. 결국 사용자들이 올바른 URL을 알 것이라고 기대할 수는 없습니다! 우리는 이것을 어떻게 할까요? 음, URL 이름을 알아내야 하고, 그 다음에 템플릿에 넣을 수 있습니다. `accounts/urls.py`에서 우리는 `signup`이라는 이름을 제공했으므로, 그것이 우리가 `base.html` 템플릿에 `url` 템플릿 태그와 함께 추가해야 할 전부입니다. 다른 링크들처럼요.

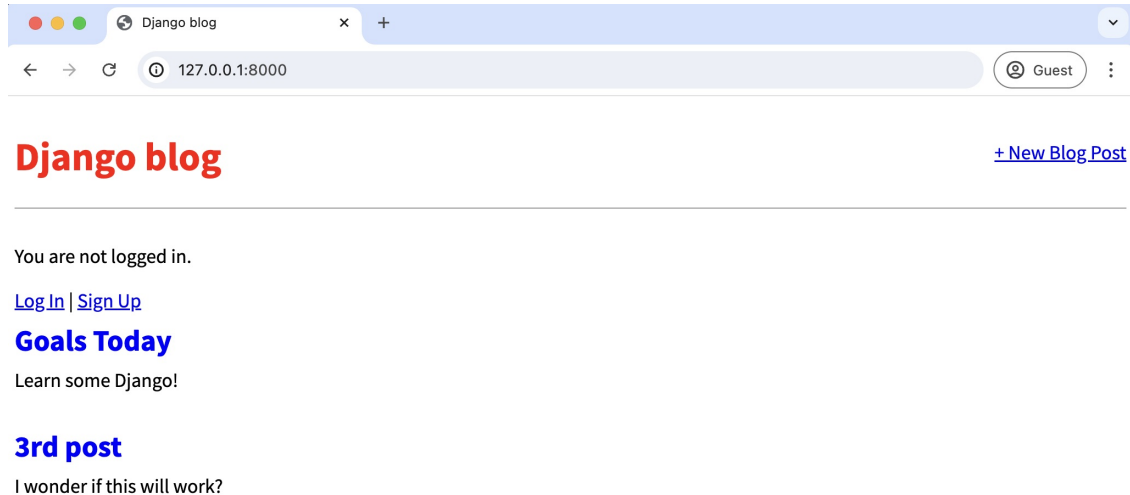
“로그인”이라는 기존 링크 바로 아래에 “가입” 링크를 다음과 같이 추가하세요:

셸

```
<!-- templates/base.html -->
...
<p>로그인하지 않았습니다.</p><a href="{% url 'login' %}">로그인</a> |
```

```
<a href="{% url 'signup' %}">Sign Up</a>
...
```

The signup link will be visible if you navigate to the logged-out homepage. Much better!



Signup Link

GitHub

It's been a while since we made a git commit. Let's do that and then push a copy of our code onto GitHub. First, check all the new work we've done with `git status`.

Shell

```
(.venv) $ git status
```

Then add the new content and enter a commit message.

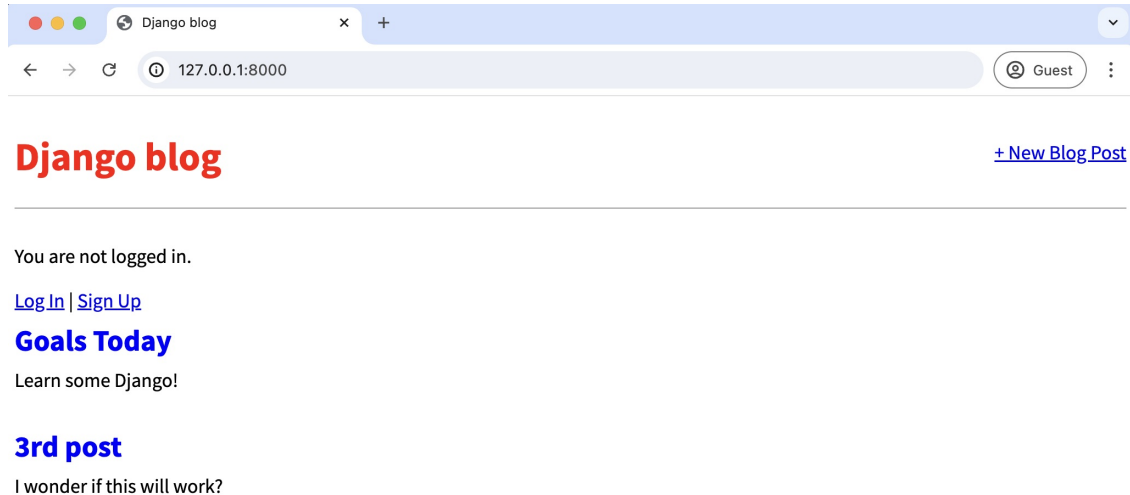
Shell

```
(.venv) $ git add -A
(.venv) $ git commit -m "forms, user accounts, and static files"
```

[Create a new repo](#) on GitHub and follow the recommended steps. I've chosen the name `blog` here; my username is `wsvincent`. Make sure to use your own GitHub username and repo name for the command setting up a remote origin.

가입하기
...

로그아웃된 홈페이지로 이동하면 가입 링크가 표시됩니다. 훨씬 나아졌습니다!



가입 링크

GitHub

우리가 git 커밋을 한 지 꽤 되었네요. 이제 그것을 하고 우리의 코드를 GitHub에 푸시합니다. 먼저, git status로 우리가 한 모든 새로운 작업을 확인하세요.

셸

```
(.venv) $ git status
```

그런 다음 새로운 내용을 추가하고 커밋 메시지를 입력하세요.

셸

```
(.venv) $ git add -A  
(.venv) $ git commit -m "forms, user accounts, and static files"
```

GitHub에 새 리포지토리를 만들고 권장 단계를 따르세요. 여기서는 블로그라는 이름을 선택했습니다; 제 사용자 이름은 wsvincent입니다. 원격 원본을 설정하는 명령어에 자신의 GitHub 사용자 이름과 리포지토리 이름을 사용해야 합니다.

Shell

```
(.venv) $ git remote add origin https://github.com/wsvincent/blog.git  
(.venv) $ git branch -M main  
(.venv) $ git push -u origin main
```

All set!

Conclusion

With a small amount of code, we have added a robust user authentication flow to our website: login, logout, and signup. Under the hood, Django has taken care of the many security gotchas that can crop up if you create a user authentication flow from scratch. The *Blog* website is now complete. It is intentionally minimalist in design, but the combination of user authentication and full CRUD functionality is present in most websites.

In the next chapter, we will start the final project in the book: a *Newspaper* website that incorporates even more features.

셸

```
(.venv) $ git remote add origin https://github.com/wsvincent/blog.git  
(.venv) $ git branch -M main  
(.venv) $ git push -u origin main
```

모든 준비가 완료되었습니다!

결론

조금의 코드로, 우리는 웹사이트에 강력한 사용자 인증 흐름을 추가했습니다: 로그인, 로그아웃, 그리고 가입. 내부적으로, Django는 사용자 인증 흐름을 처음부터 만들 경우 발생할 수 있는 많은 보안 문제를 처리했습니다. 블로그 웹사이트는 이제 완성되었습니다. 디자인은 의도적으로 미니멀리스트이지만, 사용자 인증과 전체 CRUD 기능의 조합은 대부분의 웹사이트에 존재합니다.

다음 장에서는 책의 마지막 프로젝트를 시작할 것입니다: 더 많은 기능을 통합한 신문 웹사이트입니다.

Chapter 9: Newspaper Project

This chapter and the remaining portion of the book will focus on building a production-ready *Newspaper* website. The project choice is an homage to Django's roots as a newspaper CRM. It provides the opportunity to introduce even more features, including advanced user authentication and styling, complex data models, permissions, deployment, and more.

Initial Set Up

The first step is to create a new Django project from the command line. We need to do our familiar steps of creating and navigating to a new directory called `news` and installing and activating a new virtual environment called `.venv`.

Shell

```
# Windows
$ cd onedrive\desktop\code
$ mkdir news
$ cd news
$ python -m venv .venv
$ .venv\Scripts\Activate.ps1
(.venv) $

# macOS
$ cd ~/desktop/code
$ mkdir news
$ cd news
$ python3 -m venv .venv
$ source .venv/bin/activate
(.venv) $
```

Next, install Django and Black, create a new Django project called `django_project`, and make a new app called `accounts`.

Shell

```
(.venv) $ python -m pip install django~=5.0.0
(.venv) $ python -m pip install black
(.venv) $ django-admin startproject django_project .
(.venv) $ python manage.py startapp accounts
```

Note that we **did not** run `migrate` to configure our database. Given how tightly connected the user model is to the rest of Django, it's important to wait until **after** we've created our new custom user model before doing so.

9장: 신문 프로젝트

이 장과 책의 나머지 부분은 생산 준비가 완료된 신문 웹사이트를 구축하는 데 중점을 둡니다. 이 프로젝트 선택은 Django의 뿌리인 신문 CRM에 대한 경의를 표합니다. 이는 고급 사용자 인증 및 스타일링, 복잡한 데이터 모델, 권한, 배포 등 더 많은 기능을 소개할 수 있는 기회를 제공합니다.

초기 설정

첫 번째 단계는 명령줄에서 새로운 Django 프로젝트를 만드는 것입니다. 우리는 news라는 새 디렉토리를 만들고 탐색하는 익숙한 단계를 수행해야 합니다.

그리고 .venv라는 새로운 가상 환경을 설치하고 활성화합니다. 셸

```
# Windows$ cd onedrive\
desktop\code$ mkdir news

$ cd news
$ python -m venv .venv$ .venv\
Scripts\Activate.ps1( .venv) $
```

```
# macOS$ cd ~
/desktop/code$ mkdir
news$ cd news

$ python3 -m venv .venv$ source
.venv/bin/activate( .venv) $
```

다음으로, Django와 Black을 설치하고, django_project라는 새로운 Django 프로젝트를 생성한 후, accounts라는 새로운 앱을 만듭니다.

셸

```
( .venv) $ python -m pip install django~=5.0.0( .venv) $
python -m pip install black
( .venv) $ django-admin startproject django_project .
( .venv) $ python manage.py startapp accounts
```

우리가 데이터베이스를 구성하기 위해 migrate를 실행하지 않았다는 점에 유의하세요. 사용자 모델이 Django의 나머지 부분과 얼마나 밀접하게 연결되어 있는지를 고려할 때, 새로운 사용자 모델을 만든 후에야 migrate를 실행하는 것이 중요합니다.

In your web browser, navigate to `http://127.0.0.1:8000`, and the familiar Django welcome screen will be visible.

Git

The start of a new project is an excellent time to initialize Git and create a repo on GitHub. We've done this several times before, so we can use the same commands to initialize a new local Git repo and check its status.

Shell

```
(.venv) $ git init  
(.venv) $ git status
```

The `.venv` directory, the `__pycache__` directory, and the SQLite database should not be included in Git, so create a project-level `.gitignore` file in your text editor.

`.gitignore`

```
.venv/  
__pycache__/  
db.sqlite3
```

Run `git status` again to confirm the `.venv` directory and SQLite database are not included. Then, add the rest of our work along with a commit message.

Shell

```
(.venv) $ git status  
(.venv) $ git add -A  
(.venv) $ git commit -m "initial commit"
```

[Create a new repo](#) on GitHub and provide a name. I've chosen `news`; my username is `wsvincent`. Make sure to use your repo name and username using the command below.

Shell

```
(.venv) $ git remote add origin https://github.com/wsvincent/news.git  
(.venv) $ git branch -M main  
(.venv) $ git push -u origin main
```

All done!

User Profile vs Custom User Model

웹 브라우저에서 `http://127.0.0.1:8000`로 이동하면 익숙한 Django 환영 화면이 표시됩니다.

Git

새 프로젝트의 시작은 Git을 초기화하고 GitHub에 리포지토리를 만드는 데 훌륭한 시점입니다. 우리는 이전에 여러 번 이 작업을 수행했으므로 동일한 명령을 사용하여 새 로컬 Git 리포지토리를 초기화하고 상태를 확인할 수 있습니다.

셸

```
(.venv) $ git init(.venv)
$ git status
```

`.venv` 디렉토리, `__pycache__` 디렉토리 및 SQLite 데이터베이스는 Git에 포함되지 않아야 하므로 텍스트 편집기에서 프로젝트 수준의 `.gitignore` 파일을 만드세요.

`.gitignore`

```
.venv/__pycache__
__pycache__/db.sqlite3
```

다시 `git status`를 실행하여 `.venv` 디렉토리와 SQLite 데이터베이스가 포함되지 않았는지 확인합니다. 그런 다음 나머지 작업과 커밋 메시지를 추가합니다.

셸

```
(.venv) $ git status
(.venv) $ git add -A
(.venv) $ git commit -m "초기 커밋"
```

[GitHub](#)에 새 리포를 만들고 이름을 제공하세요. 저는 `news`를 선택했습니다; 제 사용자 이름은 `wsvincent`입니다. 아래 명령어를 사용하여 리포 이름과 사용자 이름을 반드시 사용하세요.

셸

```
(.venv) $ git remote add origin https://github.com/wsvincent/news.git(.venv) $ git
branch -M main
(.venv) $ git push -u origin main
```

모두 완료되었
습니다!

사용자 프로필 vs 사용자 정의 모델

Django’s built-in [User model](#) allows us to start working with users right away, as we just did with our *Blog app* in the previous chapters. However, most large projects need a way to add information related to users, such as age or any number of additional fields. There are two popular approaches.

The first is called the “User Profile” approach and [extends the existing User model](#) by creating a [OneToOneField](#) to a separate model containing fields with additional information. The idea is to keep authentication reserved for user and not bundled with non-authentication-related user information.

The second approach is to create a custom user model that extends user but allows for additional user information to be added. The [Django documentation](#) recommends using a custom user model when starting a new project as it makes later customization far easier than using the default user model. A custom user model can be created using [AbstractUser](#), which behaves identically to the default user model but allows for customization.

It is possible to implement a hybrid approach combining a custom user model and a user profile model. But for this project, we will stick to a basic custom user model using `AbstractUser`.

AbstractUser

We can create a custom user model in four steps:

- update `django_project/settings.py`
- add a new `CustomUser` model
- add new forms for `UserCreationForm` and `UserChangeForm`
- update `accounts/admin.py`

In `django_project/settings.py`, we’ll add the `accounts` app to our `INSTALLED_APPS`. Then at the bottom of the file, use the [AUTH_USER_MODEL](#) config to tell Django to use our new custom user model instead of the built-in user model. We’ll call our custom user model `CustomUser`. Since it will exist within our `accounts` app, we should refer to it as `accounts.CustomUser`.

Code

```
# django_project/settings.py
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
```

Django의 내장 User 모델을 사용하면 이전 장에서 Blog 앱과 같이 바로 사용자 작업을 시작할 수 있습니다. 그러나 대부분의 대규모 프로젝트에서는 나이 또는 추가 필드와 같은 사용자 관련 정보를 추가할 방법이 필요합니다. 두 가지 인기 있는 접근 방식이 있습니다.

첫 번째는 "사용자 프로필" 접근법이라고 하며, 추가 정보를 포함하는 필드를 가진 별도의 모델과 OneToOneField를 생성하여 기존 User 모델을 확장하는 방식입니다. 이 아이디어는 인증을 User에만 국한시키고 인증과 관련 없는 사용자 정보를 함께 묶지 않는 것입니다.

두 번째 방법은 User를 확장하지만 추가 사용자 정보를 추가할 수 있는 맞춤형 사용자 모델을 만드는 것입니다. Django 문서는 새 프로젝트를 시작할 때 기본 User 모델을 사용하는 것보다 나중에 맞춤화가 훨씬 쉬워지므로 맞춤형 사용자 모델을 사용할 것을 권장합니다. 맞춤형 사용자 모델은 기본 User 모델과 동일하게 동작하지만 맞춤화를 허용하는 AbstractUser를 사용하여 만들 수 있습니다.

커스텀 사용자 모델과 사용자 프로필 모델을 결합한 하이브리드 방식을 구현하는 것이 가능합니다. 하지만 이 프로젝트에서는 AbstractUser를 사용한 기본 커스텀 사용자 모델에 집중할 것입니다.

추상 사용자

우리는 네 단계로 맞춤 사용자 모델을 만들 수 있습니다:

- django_project/settings.py 업데이트
- 트 새 CustomUser 모델 추가
- UserCreationForm 및 UserChangeForm에 대한 새 양식 추가
- accounts/admin.py 업데이트

django_project/settings.py에서 accounts 앱을 INSTALLED_APPS에 추가합니다. 그런 다음 파일 하단에서 AUTH_USER_MODEL 설정을 사용하여 Django가 내장 User 모델 대신 새로 만든 커스텀 사용자 모델을 사용하도록 지정합니다. 우리의 커스텀 사용자 모델은 CustomUser라고 부를 것입니다. 존재할 것이기 때문에

우리 계정 앱 내에서는 이를 accounts.CustomUser.Code라고 지칭해야 합니다.

```
# django_project/settings.py
INSTALLED_APPS = [
    django.contrib.admin",
    "django.contrib.auth",
```

```
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    "accounts", # new
]
...
AUTH_USER_MODEL = "accounts.CustomUser" # new
```

Next update `accounts/models.py` with a new User model called `CustomUser`, which extends the existing `AbstractUser`. We will also include a custom field for age here.

Code

```
# accounts/models.py
from django.contrib.auth.models import AbstractUser
from django.db import models

class CustomUser(AbstractUser):
    age = models.PositiveIntegerField(null=True, blank=True)
```

If you read the [documentation on custom user models](#), you will see that it recommends using `AbstractBaseUser`, not `AbstractUser`, which complicates things for beginners. Working with Django is far simpler and remains customizable if we use `AbstractUser` instead.

So why use `AbstractBaseUser` at all? If you want a fine level of control and customization, `AbstractBaseUser` *can* be justified. But it requires rewriting a core part of Django. If we want a custom user model that can be updated with additional fields, the better choice is `AbstractUser`, which subclasses `AbstractBaseUser`. In other words, we write much less code and have less opportunity to mess things up. It's the better choice unless you really know what you're doing with Django!

Note that we use both [null](#) and [blank](#) with our age field. These two terms are easy to confuse but quite distinct:

- `null` is **database-related**. When a field has `null=True`, it can store a database entry as `NULL`, meaning no value.
- `blank` is **validation-related**. If `blank=True`, then a form will allow an empty value, whereas if `blank=False`, then a value is required.

```
"django.contrib.contenttypes", "  
django.contrib.sessions", "  
django.contrib.messages", "  
django.contrib.staticfiles", "  
accounts", # new  
]  
...  
AUTH_USER_MODEL = "accounts.CustomUser" # new
```

다음 업데이트에서 accounts/models.py를 기존 AbstractUser를 확장하는 CustomUser라는 새로운 사용자 모델로 업데이트합니다. 여기에 나이 필드를 위한 사용자 정의 필드도 포함할 것입니다.

코드

```
# accounts/models.py  
from django.contrib.auth.models import AbstractUser  
from django.db import models  
  
class CustomUser(AbstractUser):  
    age = models.PositiveIntegerField(null=True, blank=True)
```

사용자 정의 모델에 대한 문서를 읽으면 AbstractUser가 아닌 AbstractBaseUser를 사용하는 것이 권장된다는 것을 알 수 있습니다. 이는 초보자에게는 복잡하게 만들 수 있습니다. Django와 함께 작업하는 것은 AbstractUser를 사용하는 경우 훨씬 간단하고 여전히 사용자 정의가 가능합니다.

그렇다면 왜 AbstractBaseUser를 전혀 사용할까요? 세밀한 제어와 사용자 정의가 필요하다면 AbstractBaseUser를 정당화할 수 있습니다. 하지만 이는 Django의 핵심 부분을 다시 작성해야 합니다. 추가 필드로 업데이트할 수 있는 사용자 정의 모델이 필요하다면, 더 나은 선택은 AbstractBaseUser를 서브클래스하는 AbstractUser입니다. 다시 말해, 우리는 훨씬 적은 코드를 작성하고 실수할 기회가 줄어듭니다. Django를 정말 잘 알고 있지 않는 한 더 나은 선택입니다!

우리는 나이 필드에 null과 blank를 모두 사용한다는 점에 유의하세요. 이 두 용어는 혼동하기 쉽지만 상당히 다릅니다:

- null은 데이터베이스와 관련이 있습니다. 필드에 null=True가 설정되면 데이터베이스 항목을 NULL로 저장할 수 있으며, 이는 값이 없음을 의미합니다.
- blank는 유효성 검사와 관련이 있습니다. blank=True인 경우 양식에서 빈 값을 허용하지만, blank=False인 경우 값이 필요합니다.

In practice, `null` and `blank` are commonly used together in this fashion so that a form allows an empty value, and the database stores that value as `NULL`.

A common mistake is that the **field type** dictates how to use these values. Whenever you have a string-based field like `CharField` or `TextField`, setting both `null` and `blank` as we've done will result in two possible values for "no data" in the database, which is a bad idea. Instead, the Django convention is to use the empty string `""`, not `NULL`.

Forms

If we step back briefly, how would we typically interact with our new `CustomUser` model? One case is when a user signs up for a new account on our website. The other is within the admin app, which allows us, as superusers, to modify existing users. So we'll need to update the two built-in forms for this functionality: [UserCreationForm](#) and [UserChangeForm](#).

Create a new file called `accounts/forms.py` and update it with the following code to extend the existing `UserCreationForm` and `UserChangeForm` forms.

Code

```
# accounts/forms.py
from django.contrib.auth.forms import UserCreationForm, UserChangeForm
from .models import CustomUser

class CustomUserCreationForm(UserCreationForm):
    class Meta:
        model = CustomUser
        fields = UserCreationForm.Meta.fields + ("age",)

class CustomUserChangeForm(UserChangeForm):
    class Meta:
        model = CustomUser
        fields = UserChangeForm.Meta.fields
```

For both new forms, we are using the [Meta class](#) to override the default fields by setting the `model` to our `CustomUser` and using the default fields via `Meta.fields` which includes *all* default fields. To add our custom age field, we simply tack it on at the end, and it will display automatically on our future signup page. Pretty slick, no?

실제로, null과 blank는 일반적으로 이러한 방식으로 함께 사용되어 aform이 빈 값을 허용하고 데이터베이스가 해당 값을 NULL로 저장합니다.

일반적인 실수는 필드 유형이 이러한 값을 사용하는 방법을 결정한다는 것입니다. CharField나 TextField와 같은 문자열 기반 필드가 있을 때, 우리가 한 것처럼 null과 blank를 모두 설정하면 데이터베이스에서 "nodata"에 대해 두 가지 가능한 값이 생성되며, 이는 좋지 않은 아이디어입니다. 대신, Django 관례는 빈 문자열 ""을 사용하는 것입니다, NULL이 아닙니다.

양식

잠시 뒤로 물러서서, 우리의 새로운 CustomUser 모델과 일반적으로 어떻게 상호작용할까요? 한 경우는 사용자가 우리 웹사이트에서 새 계정을 등록할 때입니다. 다른 경우는 관리 앱 내에서, 슈퍼 사용자로서 기존 사용자를 수정할 수 있게 해줍니다. 따라서 이 기능을 위해 두 개의 내장 양식인 UserCreationForm과 UserChangeForm을 업데이트해야 합니다.

accounts/forms.py라는 새 파일을 만들고 다음으로 업데이트합니다. 기존 UserCreationForm 및 UserChangeForm 양식을 확장하는 코드입니다. 코드

```
# accounts/forms.py
from django.contrib.auth.forms import UserCreationForm, UserChangeForm from .models
import CustomUser

class CustomUserCreationForm(UserCreationForm):
    class Meta:
        model = CustomUser
        fields = UserCreationForm.Meta.fields + ("age",)

class CustomUserChangeForm
    (UserChangeForm):
    class Meta:
        model = CustomUser
        fields = UserChangeForm.Meta.fields
```

새로운 폼 모두에 대해, 우리는 Meta 클래스를 사용하여 기본 필드를 재정의하고 모델을 CustomUser로 설정하며 기본 필드를 Meta.fields를 통해 사용하고 있습니다. 여기에는 모든 기본 필드가 포함됩니다. 우리의 사용자 정의 나이 필드를 추가하기 위해, 우리는 단순히 끝에 추가하면 되며, 이는 우리의 미래 가입 페이지에 자동으로 표시됩니다. 꽤 멋지죠?

The concept of fields on a form can be confusing at first, so let's take a moment to explore it further. Our CustomUser model contains all the fields of the default User model **and** our additional age field, which we set.

But what are these default fields? It turns out there [are many](#) including username, first_name, last_name, email, password, groups, and more. Yet when a user signs up for a new account on Django, the default form only asks for a username, email, and password, which tells us that the default setting for fields on UserCreationForm is just username, email, and password even though many more fields are available.

Understanding how forms and models interact in Django takes time and repetition. Don't be discouraged if you are slightly confused right now! In the next chapter, we will create our signup, login, and logout pages to tie together our CustomUser model and forms more clearly.

The final step is to update our admin.py file since the admin is tightly coupled to the default User model. We will extend the existing [UserAdmin](#) class to use our new CustomUser model. To control which fields are listed, we use [list_display](#). But to edit new custom fields, like age, we must add [fieldsets](#). And to include a new custom field in the section for creating a new user we rely on `add_fieldsets`.

Here is what the complete code looks like:

Code

```
# accounts/admin.py
from django.contrib import admin
from django.contrib.auth.admin import UserAdmin

from .forms import CustomUserCreationForm, CustomUserChangeForm
from .models import CustomUser

class CustomUserAdmin(UserAdmin):
    add_form = CustomUserCreationForm
    form = CustomUserChangeForm
    model = CustomUser
    list_display = [
        "email",
        "username",
        "age",
        "is_staff",
    ]
    fieldsets = UserAdmin.fieldsets + ((None, {"fields": ("age",)}),)
    add_fieldsets = UserAdmin.add_fieldsets + ((None, {"fields": ("age",)}),)
```

양식의 필드 개념은 처음에는 혼란스러울 수 있으므로, 좀 더 자세히 살펴보도록 하겠습니다. 우리의 CustomUser 모델은 기본 User 모델의 모든 필드와 우리가 설정한 추가적인 나이 필드를 포함하고 있습니다.

하지만 이 기본 필드는 무엇일까요? 사용자 이름, 이름, 성, 이메일, 비밀번호, 그룹 등 많은 필드가 있습니다. 그러나 사용자가 Django에서 새 계정을 등록할 때 기본 양식은 사용자 이름, 이메일 및 비밀번호만 요청합니다. 이는 UserCreationForm의 필드에 대한 기본 설정이 사용자 이름, 이메일 및 비밀번호라는 것을 알려줍니다. 더 많은 필드가 사용 가능하지만 말이죠.

Django에서 양식과 모델이 어떻게 상호작용하는지를 이해하는 데는 시간과 반복이 필요합니다. 지금 약간 혼란스러워도 낙담하지 마세요! 다음 장에서는 우리의 CustomUser 모델과 양식을 더 명확하게 연결하기 위해 가입, 로그인 및 로그아웃 페이지를 만들 것입니다.

마지막 단계는 admin.py 파일을 업데이트하는 것입니다. 관리자는 기본 User 모델과 밀접하게 연결되어 있기 때문입니다. 우리는 기존 UserAdmin 클래스를 확장하여 우리의 새로운 CustomUser 모델을 사용합니다. 어떤 필드가 나열될지를 제어하기 위해 list_display를 사용합니다. 그러나 나이와 같은 새로운 사용자 정의 필드를 편집하려면 fieldsets를 추가해야 합니다. 그리고 새 사용자를 생성하는 섹션에 새로운 사용자 정의 필드를 포함하기 위해 add_fieldsets에 의존합니다.

완전한 코드는 다음과 같습니다:

코드

```
# accounts/admin.py
from django.contrib import admin
from django.contrib.auth.admin import UserAdmin

from .forms import CustomUserCreationForm, CustomUserChangeForm
from .models import CustomUser

class CustomUserAdmin(UserAdmin):
    add_form = CustomUserCreationForm
    change_form = CustomUserChangeForm
    model = CustomUser
    list_display = ["email", "username", "age", "is_staff", ]
    fieldsets = UserAdmin.fieldsets + ((None, {"fields": ("age",)}),)
    add_fieldsets = UserAdmin.add_fieldsets + ((None, {"fields": ("age",)}),)
```

```
admin.site.register(CustomUser, CustomUserAdmin)
```

There are many ways to customize the user admin, and some developers like to add additional options such as [list_filter](#), [search_fields](#), and [ordering](#).

But for this project, we are now done. Type Control+c to stop the local server and go ahead and run makemigrations and migrate for the first time to create a new database that uses the custom user model.

Shell

```
(.venv) $ python manage.py makemigrations accounts
Migrations for 'accounts':
  accounts/migrations/0001_initial.py
    - Create model CustomUser
```

Shell

```
(.venv) $ python manage.py migrate
Operations to perform:
  Apply all migrations: accounts, admin, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0001_initial... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying accounts.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying sessions.0001_initial... OK
```

Superuser

Let's create a superuser account to confirm everything is working as expected. On the command line, type the following command and go through the prompts.

Shell

```
(.venv) $ python manage.py createsuperuser
```

```
admin.site.register(CustomUser, CustomUserAdmin)
```

사용자 관리자를 사용자 정의하는 방법은 여러 가지가 있으며, 일부 개발자는 `list_filter`, `search_fields` 및 `ordering`과 같은 추가 옵션을 추가하는 것을 좋아합니다.

하지만 이 프로젝트에서는 이제 완료되었습니다. `Control+c`를 입력하여 로컬 서버를 중지하고 `makemigrations` 및 `migrate`를 처음으로 실행하여 사용자 정의 사용자 모델을 사용하는 새 데이터베이스를 만드세요.

Shell

```
(.venv) $ python manage.py makemigrations accountsMigrations
for 'accounts':
  accounts/migrations/0001_initial.py- 모델
  CustomUser 생성
```

Shell

```
(.venv) $ python manage.py migrate수행할
작업:
  모든 마이그레이션 적용: accounts, admin, auth, contenttypes, sessionsRunning
migrations:
  contenttypes.0001_initial 적용 중... OK
  contenttypes.0002_remove_content_type_name 적용 중...
  OKauth.0001_initial 적용 중... OK
  auth.0002_alter_permission_name_max_length 적용 중...
  OKauth.0003_alter_user_email_max_length 적용 중...
  OKauth.0004_alter_user_username_opts 적용 중... OK
  auth.0005_alter_user_last_login_null 적용 중...
  OKauth.0006_require_contenttypes_0002 적용 중... OK
  auth.0007_alter_validators_add_error_messages 적용 중...
  OKauth.0008_alter_user_username_max_length 적용 중...
  OKauth.0009_alter_user_last_name_max_length 적용 중...
  OKauth.0010_alter_group_name_max_length 적용 중...
  OKauth.0011_update_proxy_permissions 적용 중... OK
  auth.0012_alter_user_first_name_max_length 적용 중...
  OKaccounts.0001_initial 적용 중... OK
  admin.0001_initial 적용 중... OK
  admin.0002_logentry_remove_auto_add 적용 중... OK
  admin.0003_logentry_add_action_flag_choices 적용 중...
  OKsessions.0001_initial 적용 중... OK
```

슈퍼유저

모든 것이 예상대로 작동하는지 확인하기 위해 슈퍼유저 계정을 생성합니다. 명령줄에 다음 명령을 입력하고 프롬프트를 따라 진행하세요.

셸

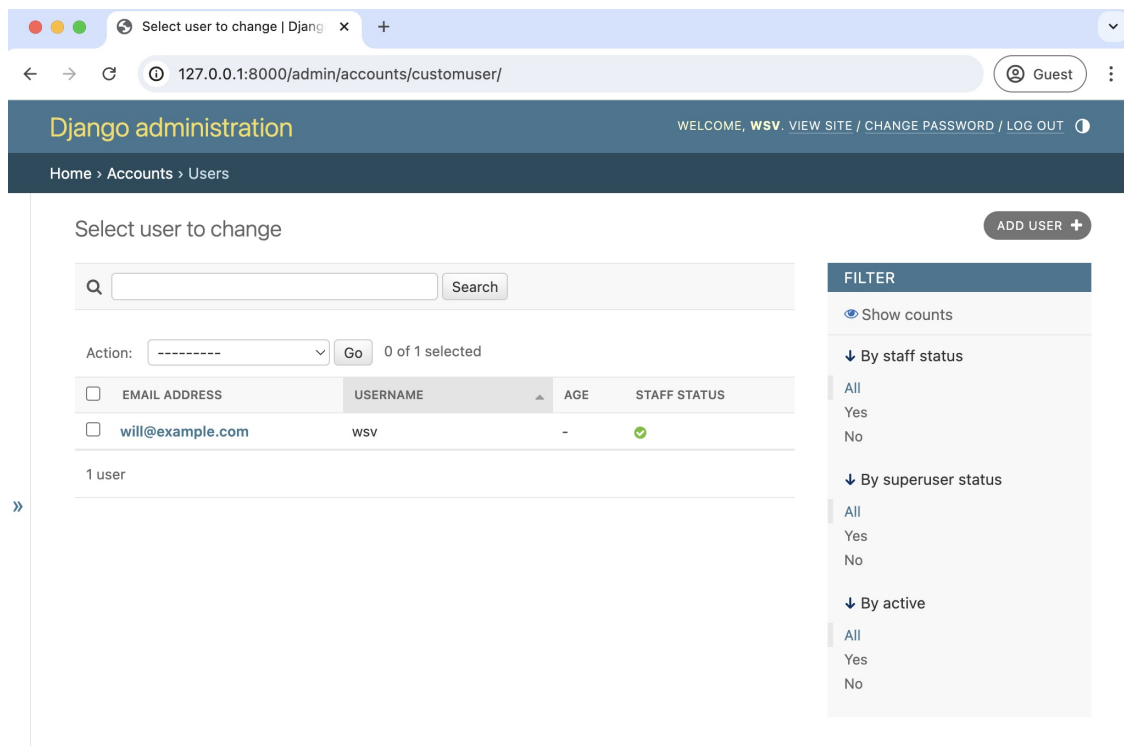
```
(.venv) $ python manage.py createsuperuser
```

Make sure your superuser email account is one that actually works. We will use it later on to verify email integration. But the fact that this flow here works is the first proof our custom user model is set up correctly. Let's view things in the admin, too, to be extra sure. Start up the web server.

Shell

```
(.venv) $ python manage.py runserver
```

Then navigate to the admin at <http://127.0.0.1:8000/admin> and log in. If you click on the link for “Users” you should see your superuser account and the default fields: Email Address, Username, Age, and Staff Status. These were set in `list_display` in our `admin.py` file.



Admin Select User to Change

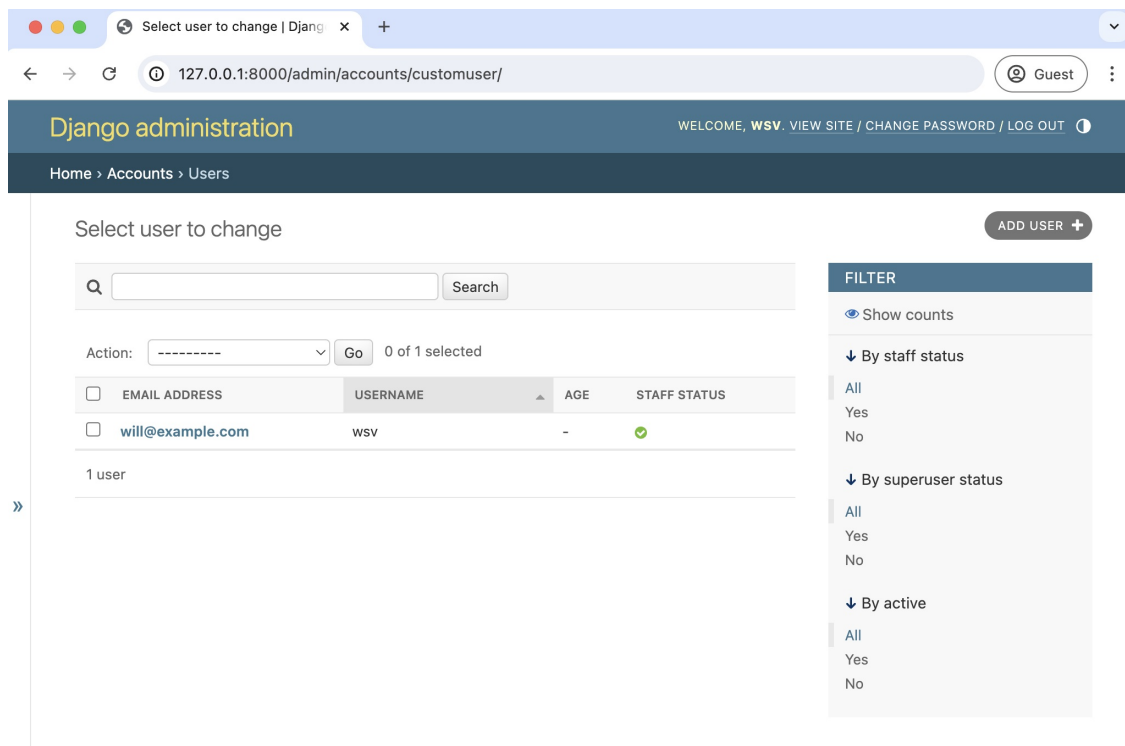
The age field is empty because we have yet to set it. However, you can set your age now because we set the `fieldsets` section. Click on the highlighted link for your superuser's email address to bring up the edit user interface. If you scroll to the bottom, you will see that we added the age field. Go ahead and enter your age. Then click on “Save.”

슈퍼유저 이메일 계정이 실제로 작동하는 계정인지 확인하세요. 나중에 이메일 통합을 확인하는 데 사용할 것입니다. 하지만 이 흐름이 작동한다는 사실은 우리의 사용자 모델이 올바르게 설정되었다는 첫 번째 증거입니다. 추가로 확인하기 위해 관리자에서 내용을 확인해 봅시다. 웹 서버를 시작하세요.

셸

```
(. venv) $ python manage.py runserver
```

그런 다음 <http://127.0.0.1:8000/admin>에서 관리자 페이지로 이동하여 로그인하세요. '사용자' 링크를 클릭하면 슈퍼유저 계정과 기본 필드인 이메일 주소, 사용자 이름, 나이 및 직원 상태가 표시됩니다. 이 필드는 `admin.py` 파일의 `list_display`에 설정되었습니다.



관리자 사용자 변경 선택

나이 필드는 아직 설정하지 않았기 때문에 비어 있습니다. 그러나 필드셋 섹션을 설정했기 때문에 지금 나이를 설정할 수 있습니다. 슈퍼유저의 이메일 주소에 하이라이트된 링크를 클릭하여 사용자 편집 인터페이스를 불러옵니다. 아래로 스크롤하면 나이 필드를 추가한 것을 볼 수 있습니다. 나이를 입력하세요. 그런 다음 '저장'을 클릭하세요.

wsv | Change user | Django | x

127.0.0.1:8000/admin/accounts/customuser/1/change/ Guest

Filter

- Accounts | user | Can add user
- Accounts | user | Can change user
- Accounts | user | Can delete user
- Accounts | user | Can view user
- Administration | log entry | Can add log entry
- Administration | log entry | Can change log entry
- Administration | log entry | Can delete log entry
- Administration | log entry | Can view log entry
- Authentication and Authorization | group | Can add
- Authentication and Authorization | group | Can chan
- Authentication and Authorization | group | Can dele
- Authentication and Authorization | group | Can view

Choose all Remove all

Specific permissions for this user. Hold down "Control", or "Command" on a Mac, to select more than one.

Important dates

Last login: Date: 2024-07-01 Today Time: 17:54:08 Now
Note: You are 4 hours behind server time.

Date joined: Date: 2024-07-01 Today Time: 17:53:57 Now
Note: You are 4 hours behind server time.

Age:

SAVE Save and add another Save and continue editing Delete

Admin Edit Age

It will redirect back to the main Users page listing our superuser. Note that the age field is now updated.

wsv | Change user | Django | 127.0.0.1:8000/admin/accounts/customuser/1/change/ Guest

Filter

Accounts | user | Can add user
Accounts | user | Can change user
Accounts | user | Can delete user
Accounts | user | Can view user
Administration | log entry | Can add log entry
Administration | log entry | Can change log entry
Administration | log entry | Can delete log entry
Administration | log entry | Can view log entry
Authentication and Authorization | group | Can add
Authentication and Authorization | group | Can chan
Authentication and Authorization | group | Can dele
Authentication and Authorization | group | Can view

Choose all Remove all

Specific permissions for this user. Hold down "Control", or "Command" on a Mac, to select more than one.

Important dates

Last login: Date: 2024-07-01 Today Time: 17:54:08 Now
Note: You are 4 hours behind server time.

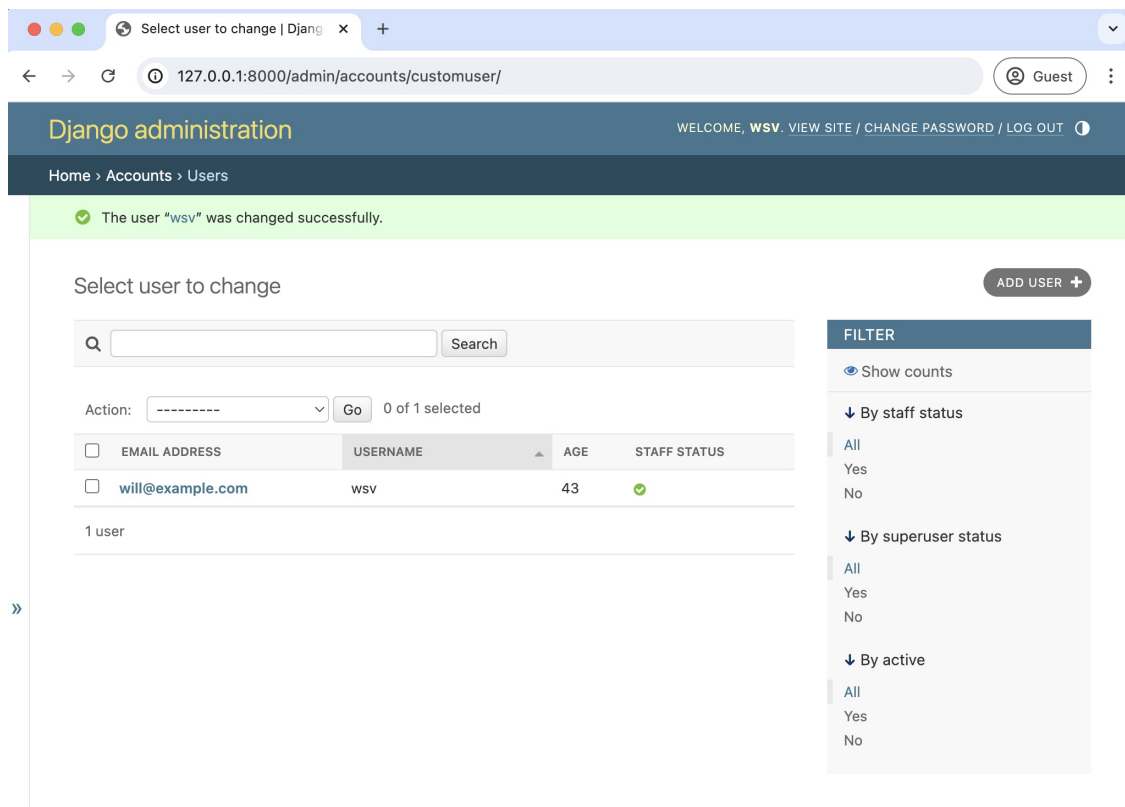
Date joined: Date: 2024-07-01 Today Time: 17:53:57 Now
Note: You are 4 hours behind server time.

Age:

SAVE Save and add another Save and continue editing Delete

관리자 나이 수정

주 슈퍼유저 목록이 있는 기본 사용자 페이지로 다시 리디렉션됩니다.
나이 필드가 이제 업데이트되었음을 유의하세요.



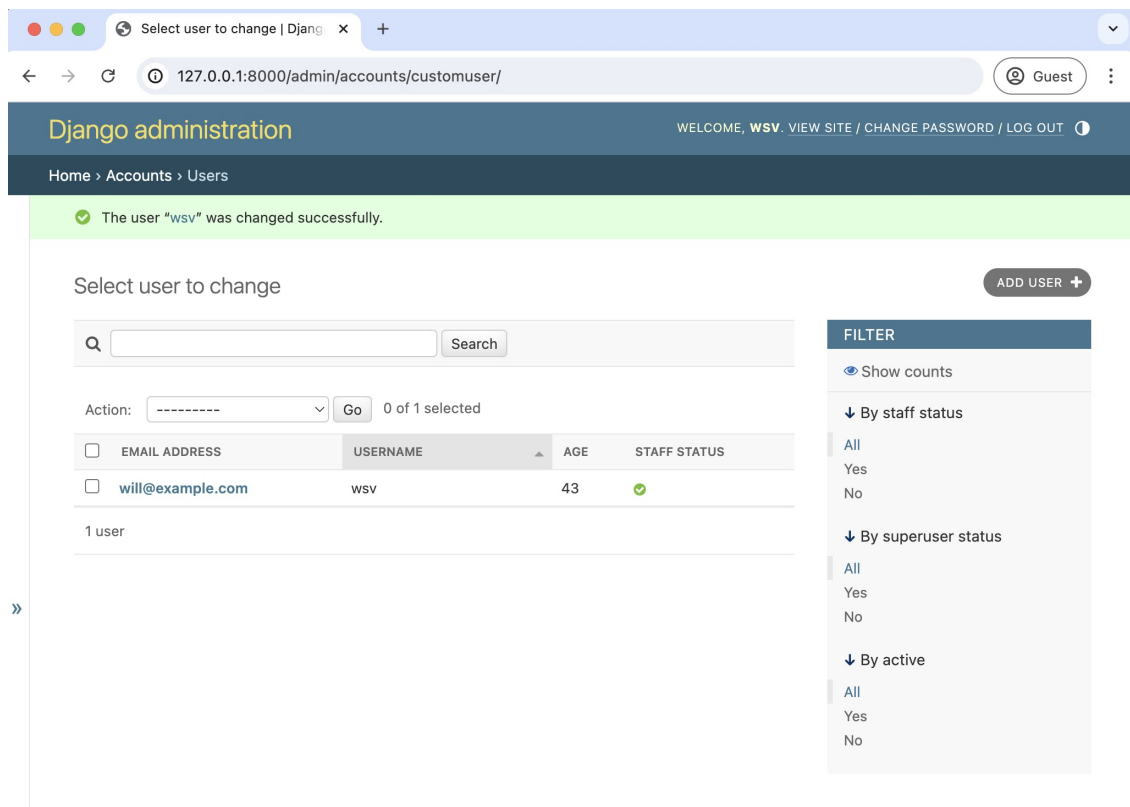
Admin Updated Age

Tests

It is a good idea to add tests every time we make code changes that alter core functionality. While all our manual actions trying out the custom user worked just now, we may break something in the future. Adding tests for new code and regularly running the entire test suite helps spot errors early.

At a high level, we want to ensure that both a regular user and a superuser can be created and have the proper field permissions. Suppose you look at the [official documentation](#) on `models.User`, which our custom user model inherits from. In that case, it comes with several built-in fields: `username`, `first_name`, `last_name`, `email`, `password`, `groups`, `user_permissions`, `is_staff`, `is_active`, `is_superuser`, `last_login`, and `date_joined`. It is also possible to add any number of custom fields, as we have seen by adding the `age` field.

Being “staff” means a user can access the admin site and view models for which they are given permission; a “superuser” has full access to the admin and all its



관리자 업데이트 나이

테스트

코어 기능을 변경하는 코드 변경을 할 때마다 테스트를 추가하는 것은 좋은 생각입니다. 사용자 정의 사용자로 시도하는 모든 수동 작업이 지금은 잘 작동했지만, 미래에 무언가를 망가뜨릴 수 있습니다. 새로운 코드에 대한 테스트를 추가하고 전체 테스트 스위트를 정기적으로 실행하는 것은 오류를 조기에 발견하는 데 도움이 됩니다.

높은 수준에서, 우리는 일반 사용자와 슈퍼 사용자가 모두 생성되고 적절한 필드 권한을 가질 수 있도록 보장하고자 합니다. 사용자 정의 사용자 모델이 상속하는 `models.User`에 대한 공식 문서를 살펴보면, 여러 개의 내장 필드가 함께 제공됩니다: `username`, `first_name`, `last_name`, `email`, `password`, `groups`, `user_permissions`, `is_staff`, `is_active`, `is_superuser`, `last_login`, `date_joined`. 나이 필드를 추가함으로써 보았듯이, 원하는 만큼의 사용자 정의 필드를 추가하는 것도 가능합니다.

“스태프”라는 것은 사용자가 관리 사이트에 접근하고 그들이 권한을 부여받은 모델을 볼 수 있다는 것을 의미합니다; “슈퍼 사용자”는 관리 및 모든 것에 대한 전체 접근 권한을 가집니다.

models. A regular user should have `is_active` set to `True`, `is_staff` set to `False`, and `is_superuser` to `False`. A superuser should have everything set to `True`.

Here is one way to add tests to our custom user model:

Code

```
# accounts/tests.py
from django.contrib.auth import get_user_model
from django.test import TestCase

class UsersManagersTests(TestCase):
    def test_create_user(self):
        User = get_user_model()
        user = User.objects.create_user(
            username="testuser",
            email="testuser@example.com",
            password="testpass1234",
        )
        self.assertEqual(user.username, "testuser")
        self.assertEqual(user.email, "testuser@example.com")
        self.assertTrue(user.is_active)
        self.assertFalse(user.is_staff)
        self.assertFalse(user.is_superuser)

    def test_create_superuser(self):
        User = get_user_model()
        admin_user = User.objects.create_superuser(
            username="testsuperuser",
            email="testsuperuser@example.com",
            password="testpass1234",
        )
        self.assertEqual(admin_user.username, "testsuperuser")
        self.assertEqual(admin_user.email, "testsuperuser@example.com")
        self.assertTrue(admin_user.is_active)
        self.assertTrue(admin_user.is_staff)
        self.assertTrue(admin_user.is_superuser)
```

At the top, we import `get_user_model()`, so we can test our user registration. We also import `TestCase` since these tests touch the database.

Our class of tests is called `UsersManagersTests` and extends `TestCase`. The first unit test, `test_create_user`, checks that a regular user displays expected behavior. The second unit test, `test_create_superuser`, does the same, albeit for a superuser account.

Now run the tests; they should pass without any issues.

Shell

모델. 일반 사용자는 is_active가 True로 설정되고, is_staff가 False로 설정되며, is_superuser가 False로 설정되어야 합니다. 슈퍼유저는 모든 것이 True로 설정되어야 합니다.

사용자 정의 사용자 모델에 테스트를 추가하는 한 가지 방법은 다음과 같습니다:

코드

```
# accounts/tests.py
from django.contrib.auth import get_user_model
from django.test import TestCase

class UsersManagersTests(TestCase):
    def test_create_user(self):
        User = get_user_model()
        user = User.objects.create_user(
            username="testuser", email="testuser@example.com",
            password="testpass1234", )

        self.assertEqual(user.username, "testuser")
        self.assertEqual(user.email, "testuser@example.com")
        self.assertTrue(user.is_active)
        self.assertFalse(user.is_staff)
        self.assertFalse(user.is_superuser)

    def test_create_superuser(self):
        User = get_user_model()
        admin_user = User.objects.create_superuser(
            username="testsuperuser", email="testsuperuser@example.com",
            password="testpass1234", )

        self.assertEqual(admin_user.username, "testsuperuser")
        self.assertEqual(admin_user.email, "testsuperuser@example.com")
        self.assertTrue(admin_user.is_active)
        self.assertTrue(admin_user.is_staff)
        self.assertTrue(admin_user.is_superuser)
```

맨 위에서 `get_user_model()`을 가져와서 사용자 등록을 테스트할 수 있습니다. 또한 데이터베이스에 접근하기 때문에 `TestCase`도 가져옵니다.

우리의 테스트 클래스는 `UsersManagersTests`라고 하며 `TestCase`를 확장합니다. 첫 번째 단위 테스트인 `test_create_user`는 일반 사용자가 예상되는 동작을 표시하는지 확인합니다. 두 번째 단위 테스트인 `test_create_superuser`는 슈퍼유저 계정에 대해 동일한 작업을 수행합니다.

이제 테스트를 실행하세요. 문제 없이 통과해야 합니다.

Shell

```
(.venv) $ python manage.py test
Found 2 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
..
-----
Ran 2 tests in 0.114s

OK
Destroying test database for alias 'default'...
```

Git

We've completed a bunch of new work, so it's time to add a Git commit.

Shell

```
(.venv) $ git status
(.venv) $ git add -A
(.venv) $ git commit -m "custom user model"
```

Conclusion

We started our new project by adding a custom user model and an age field. We also explored how to add tests whenever core Django functionality is changed and can now focus on building the rest of our *Newspaper* website. In the next chapter, we will implement advanced authentication and registration by customizing the signup, login, and logout pages.

```
(.venv) $ python manage.py testFound
2 test(s).
'default'에 대한 테스트 데이터베이스를 생성하는 중... 시스템
검사가 문제를 식별하지 않았습니다 (0 무시됨)...
```

-2개의 테스트를 0.114초에 실행했습니다

확인

'default'에 대한 테스트 데이터베이스를 파괴하는 중...

Git

우리는 많은 새로운 작업을 완료했으므로 Git 커밋을 추가할 시간입니다.

셸

```
(.venv) $ git status
(.venv) $ git add -A
(.venv) $ git commit -m "custom user model"
```

결론

우리는 사용자 정의 사용자 모델과 나이 필드를 추가하여 새로운 프로젝트를 시작했습니다. 또한 핵심 Django 기능이 변경될 때마다 테스트를 추가하는 방법을 탐색했으며 이제 나머지 신문 웹사이트를 구축하는데 집중할 수 있습니다. 다음 장에서는 가입, 로그인 및 로그아웃 페이지를 사용자 정의하여 고급 인증 및 등록을 구현할 것입니다.

Chapter 10: User Authentication

Now that we have a working custom user model, we can add the functionality every website needs: the ability for users to sign up, log in, and log out. Django provides everything we need for users to log in and out, but we must create our own form to allow new users to sign up. We'll also build a basic homepage with links to all three features so users don't have to type in the URLs by hand every time.

Templates

By default, the Django template loader looks for templates in a nested structure within each app. For example, the structure `accounts/templates/accounts/home.html` would be needed for a `home.html` template within the `accounts` app. However, a single project-level templates directory approach is cleaner and scales better, so that's what we'll use.

Create a new project-level templates directory and a registration directory within where Django will look for templates related to logging in and signing up.

Shell

```
(.venv) $ mkdir templates
(.venv) $ mkdir templates/registration
```

We need to tell Django about this new directory by updating the configuration for "DIRS" in `django_project/settings.py`.

Code

```
# django_project/settings.py
TEMPLATES = [
    {
        ...
        "DIRS": [BASE_DIR / "templates"], # new
        ...
    }
]
```

If you think about what happens when you log in or out of a site, you are immediately redirected to a subsequent page. We need to tell Django where to send users in each case. The `LOGIN_REDIRECT_URL` and `LOGOUT_REDIRECT_URL` settings do that. We'll configure both to redirect to our homepage with the